



ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)

בנושא

מתכנן משלוחים ומסלולים

Delivery & Route Planner



שם הסטודנט: יהב רבינוביץ

ת"ז: 328456348

מנחה: אילן פרץ

תאריך הגשה: י"ח סיוון תשפ"ו 03/06/2026

שם המכללה: כנפי רוח קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

3	1. הצעת הפרויקט שאושרה ע"י משרד החינוך
4	2. מבוא
5	2.1 הרקע לפרויקט
6	2.2 תהליך המחקר
7	2.3 רקע תיאורטי וסקירת ספרות מקצועית
7	2.4 אתגרים מרכזיים במחקר
9	3. מטרות ויעדים לפיתוח המערכת
10	4. אתגרים בבניית המערכת
10	5. הגדרת מדדי הצלחה למערכת
11	6. תיאור פתרונות קיימים לבעיה וניתוח חלופות
12	7. תיאור החלופה הנבחרת ונימוקים לבחירתה
13	8. אפיון המערכת המוצעת
13	8.1 ניתוח הדרישות מהמערכת
15	8.2 מודלי המערכת
15	8.3 אפיון פונקציונלי וביצועים עיקריים
16	8.4 אילוצי המערכת
17	9. תיאור הארכיטקטורה של המערכת
17	9.1 ארכיטקטורת המערכת
18	9.2 מודל שרת לקוח
18	9.3 תהליכים של מע' ההפעלה
19	9.4 תיאור פרוטוקולי תקשורת
20	9.5 שירותים חיצוניים
21	10. ניתוח תרחישים וזרימת המידע במערכת המוצעת
21	10.1 תרשימי תרחיש Use case Diagram
23	10.2 תרשימי רצף Sequence Diagram
26	11. תיאור מסכים וממשק משתמש
26	11.1 תרשימי היררכיית המסכים
29	11.2 תיאור המסכים
48	12. תיאור התוכנה
48	12.1 סביבת הפיתוח של התוכנה
50	12.2 תיאור מבנה הפרויקט והמחלקות
74	12.3 מבני נתונים בהם נעשה שימוש
76	12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים
83	13. תיאור מסד הנתונים
86	14. מדריך למשתמש
86	15. בדיקות והערכה
87	16. מסקנות
88	17. פיתוחים עתידיים
89	18. ביבליוגרפיה

1. הצעת הפרויקט שאושרה ע"י משרד החינוך

להלן הצעת הפרויקט שאושרה, מתומצתת ומוסברת בקצרה.

תיאור נושא הפרויקט

המערכת המפותחת במסגרת פרויקט זה היא פלטפורמה חכמה לניהול ואופטימיזציה של מערכי משלוחים, המיועדת לעסקים המפעילים צי רכב להפצת סחורות בסביבה עירונית. המערכת משמשת ככלי עזר למנהלי לוגיסטיקה וסדרני הפצה, ומאפשרת להם להפוך את תהליך תכנון המסלולים הידני והמורכב לתהליך אוטומטי, מהיר ומדויק.

השימוש במערכת מתבצע באמצעות ממשק חכם אליו מוזנים נתוני ההזמנות, כתובות הלקוחות ומשאבי הצי הזמינים, ובתמורה מפיקה המערכת תוכנית עבודה אופטימלית המחלקת את המשימות בין הנהגים השונים בצורה מאוזנת. הרכיבים העיקריים של המערכת מנקודת מבט המשתמש כוללים מודול קליטת נתונים, וממשק תצוגה גיאוגרפי.

המערכת פותרת את האתגרים הלוגיסטיים והתפעוליים על ידי החלפת התכנון הידני בחישוב המבוסס על מודלים מתמטיים מתקדמים, היא מצמצמת משמעותית את הנסועה המיותרת, מבטיחה עמידה באילוצי זמן קשיחים ומונעת מצבים של עומס יתר על נהג בודד בעוד אחרים נותרים ללא פעילות.

הגדרת הבעיה האלגוריתמית

הבעיה האלגוריתמית העומדת בליבת הפרויקט מוגדרת כבעיית אופטימיזציה מסוג ניתוב כלי רכב עם מגבלות קיבולת, או בשמה המקצועי - Vehicle Routing Problem (VRP) הגדרה זו מהווה תרגום מתמטי לאתגרים הלוגיסטיים שתוארו בסעיף הראשון, ובפרט לצורך בניהול יעיל של צי רכב בסביבה עירונית צפופה. הבעיה דורשת תכנון של קבוצת מסלולים אופטימלית היוצאת מנקודת מוצא אחת, כך שכל לקוח יקבל שירות בדיוק פעם אחת על ידי רכב בודד, וכל זאת תוך מזעור פונקציית המטרה הכוללת את המרחק המצטבר, זמן הנסיעה וצריכת המשאבים.

כדי להבין את עומק הבעיה, יש לבחון את המגבלות המרכזיות המשפיעות על האלגוריתם, כאשר הראשונה שבהן היא מגבלת הקיבולת. כל משאית בצי הרכב מחזיקה בנפח או במשקל מקסימלי שהיא יכולה לשאת, מה שמאלץ את המערכת "לחתוך" מסלולים ולחזור למחסן להעמסה מחדש, גם אם גיאוגרפית היה משתלם להמשיך ללקוח הבא. לצד זאת, מגבלת חלונות הזמן מוסיפה רובד של דחיפות, שכן הגעה ללקוח מחוץ לטווח השעות שהוגדר נחשבת לכישלון של המסלול. הקשר הישיר בין הגדרה אלגוריתמית זו לאתגרים שהוצגו בסעיף 1 טמון במורכבות הריאליסטית של עולם הלוגיסטיקה, שבו המערכת נדרשת לתמרן בין צרכים סותרים, כמו רצון למזער נסועה מול הכרח להגיע ללקוחות מרוחקים בשעות ספציפיות.

מבחינה חישובית, הבעיה מוגדרת כבעיית NP קשה - הגדרה שמשמעותה היא שאין כיום אלגוריתם שמסוגל למצוא את הפתרון המושלם ביותר בזמן סביר ככל שמספר הלקוחות גדל. מרחב הפתרונות גדל באופן מעריכי עם הוספת כל לקוח או רכב חדש לצי, מה שהופך סריקה מלאה של כל האפשרויות לבלתי אפשרית מבחינה טכנולוגית. אפיון זה מחייב את המערכת להשתמש בגישות מבוססות חיפוש חכם, כגון האלגוריתם הגנטי. אלגוריתם זה מחקה תהליכים מהטבע כמו ברירה טבעית ומוטציות כדי לגדל פתרונות טובים יותר מדור לדור. במקום לחפש את הפתרון המושלם, המערכת מספקת פתרונות איכותיים וקרובים מאוד לזמן האופטימלי בזמן אמת, מה שמאפשר למנהל המערכת להריץ את האופטימיזציה ולקבל מסלולים מוכנים לעבודה באופן מיידי.

תהליך פיתוח הפתרון כלל מחקר ויישום של אלגוריתמים אבולוציוניים לשיפור הדרגתי של פתרונות, לצד הטמעת מנגנונים לבדיקת אילוצים לוגיסטיים וניהול מסלולים אופטימלי. האלגוריתם שהמערכת תשתמש בו תתייחס לכל פתרון כאל רשימת מסלולים מלאה לכל הרכבים במערכת. כלומר, מערך שמכיל את סדר הלקוחות שכל רכב צריך לבקר בהם. זהו האובייקט שהאלגוריתם מקבל כקלט בכל שלב, ועליו הוא מבצע פעולות של שיפור, סינון, ויצירה מחדש. הפלט מכל שלב הוא שוב פתרון משופר, או קבוצת פתרונות שממשיכה לשלב הבא.

2. מבוא

עולם הלוגיסטיקה עוסק בניהול, תכנון ובקרה של תהליכי הובלה והפצה של סחורות ממחסנים ומרכזים לוגיסטיים אל לקוחות הקצה. בשנים האחרונות תחום זה הפך למורכב במיוחד עקב גידול משמעותי בכמות ההזמנות, דרישה לאספקה מהירה, תנועה עירונית עמוסה ואילוצים תפעוליים רבים.

הצורך ההנדסי המרכזי בתחום הלוגיסטיקה הוא תכנון מסלולי משלוחים בצורה אופטימלית. יש צורך במערכת חישובית שתדע לחלק הזמנות בין מספר רכבי משלוח, לקבוע סדר ביקורים אצל לקוחות, ולעמוד במגבלות כמו קיבולת רכב, זמני הגעה נדרשים, מרחקי נסיעה וזמני עבודה.

כיום, למרות ההתקדמות הטכנולוגית, ארגונים רבים עדיין נתקלים בקשיים משמעותיים בשל הסתמכות על תכנון ידני או על כלים פשוטים שאינם מסוגלים לעבד את כלל האילוצים בזמנית. חוסר היכולת לבצע אופטימיזציה מלאה של המסלולים מוביל לבזבז דלק, שחיקה מואצת של כלי הרכב, עומס על הנהגים ופגיעה ברמת השירות הניתנת ללקוח. מצב זה מדגיש את הפער הקיים בין הדרישה הגוברת למשלוחים מהירים לבין היכולת המעשית לנהל את המערך הלוגיסטי בצורה יעילה וחסכונית.

2.1 הרקע לפרויקט

בשנים האחרונות, עם ההתפתחות המהירה של תחום המסחר האלקטרוני והעלייה המתמדת בדרישה למשלוחים מהירים, תחום הלוגיסטיקה הפך לאחד המרכיבים החשובים והמורכבים ביותר בניהול עסקים. חברות רבות נדרשות להתמודד מדי יום עם עשרות ואף מאות משלוחים, כאשר כל טעות בתכנון המסלולים עלולה לגרום לעיכובים, לבזבז משאבים ולהפסדים כספיים משמעותיים.

במערכות מסורתיות, תכנון מסלולי החלוקה מתבצע לעיתים בצורה ידנית או באמצעות שיקול דעת אנושי בלבד. שיטה זו יוצרת בעיות רבות, כגון נסיעות מיותרות, חוסר איזון בחלוקת העבודה בין הרכבים, צריכת דלק גבוהה ואי-עמידה בזמני אספקה. בנוסף, כאשר מספר ההזמנות גדל, קשה מאוד לאדם לחשב בצורה יעילה את סדר התחנות האופטימלי ואת חלוקת ההעמסה המתאימה לכל רכב.

בעיה מרכזית נוספת היא מגבלת הקיבולת של כלי הרכב. לכל רכב יש מגבלת משקל ונפח שאסור לחרוג ממנה, ולכן נדרש מנגנון חכם שיוכל לחלק את ההזמנות בצורה מאוזנת בין כלל רכבי החלוקה. ללא תכנון נכון, עלולים להיווצר מצבים שבהם רכב אחד עמוס יתר על המידה בעוד רכבים אחרים אינם מנוצלים באופן מלא.

כדי להתמודד עם אתגרים אלו פותחה מערכת Smart Delivery מטרת המערכת היא להפוך את תהליך ניהול המשלוחים לאוטומטי, חכם ויעיל יותר. המערכת מקבלת את כלל ההזמנות הממתינות, מחשבת את המרחקים בין הכתובות באמצעות שירותי מפות מתקדמים, ומתכננת מסלולי חלוקה אופטימליים בהתאם למיקום התחנות ולקיבולת הרכבים.

2.2 תהליך המחקר

בשלב הראשון של הפרויקט בוצע מחקר מקדים במטרה להבין כיצד עסקים וחברות הפצה מנהלים כיום את תהליך המשלוחים והלוגיסטיקה שלהם. במסגרת המחקר נבחנו תהליכי עבודה קיימים וזוהו הקשיים המרכזיים בתכנון ידני של מסלולי חלוקה. בנוסף, הוגדרו הצרכים של שני סוגי המשתמשים במערכת: מנהלי המערכת, הזקוקים לכלי ניהול ושליטה מתקדמים כמו צפייה במסלולים על גבי מפה וניהול צי הרכבים, ולקוחות הקצה, הזקוקים לממשק פשוט להזנת הזמנות ומעקב אחר סטטוס המשלוחים שלהם. בשלב השני נערכה סקירה של פתרונות קיימים ושל שיטות מתמטיות לפתרון בעיית ניתוב הרכבים. במהלך המחקר התברר כי ניסיון לבדוק את כל אפשרויות המסלולים בצורה מלאה אינו מעשי, משום שכמות השילובים גדלה בצורה עצומה ככל שמספר ההזמנות עולה. לכן, המחקר התמקד בשיטות אופטימיזציה חכמות.

בשלב השלישי נבחנה התשתית הטכנולוגית המתאימה למימוש המערכת. נבחרה סביבת פיתוח המבוססת על Java ו-Spring Boot בצד השרת יחד עם Vaadin בצד הממשק, כדי לאפשר פיתוח אחיד, יציב ומאובטח ללא צורך בהפרדה בין טכנולוגיות Frontend ו-Backend שונות. בנוסף, נבחר מסד הנתונים, MongoDB Atlas, המספק גמישות גבוהה בניהול מידע בצורה של מסמכים עצמאיים ומאפשר עבודה נוחה מול נתונים משתנים כמו הזמנות, משתמשים ורכבים. לצורך חישוב מרחקים ומסלולים נבחר Google Maps API, המאפשר לבצע חישובים המבוססים על כבישים אמיתיים ותנאי נסיעה בפועל.

2.3 רקע תיאורטי וסקירת ספרות מקצועית

בעיית ניתוב כלי הרכב, המוכרת בספרות המקצועית כ-Vehicle Routing Problem או בקיצור VRP, מהווה את אחת מאבני היסוד בתחום חקר הביצועים והאופטימיזציה הקומבינטורית. הבעיה, מגדירה אתגר תכנוני שבו צי רכבים נדרש לספק שירות לקבוצת לקוחות המפוזרים גיאוגרפית, תוך התחלה וסיום בנקודת מוצא אחת או יותר המכונה מחסן לוגיסטי. המטרה המרכזית ב-VRP היא מזעור העלות הכוללת של התפעול, הנמדדת לרוב במונחי מרחק נסיעה כולל, מספר כלי הרכב המופעלים או זמן העבודה הכולל, וכל זאת תחת עמידה במכלול של אילוצים תפעוליים. מבחינה מבנית, ה-VRP נחשבת להרחבה מורכבת של בעיית הסוכן הנוסע, שכן בעוד שהסוכן הנוסע מתמקד במסלול יחיד העובר בכל הנקודות, ה-VRP מחייב חלוקה חכמה של המשימות בין מספר רכבים, מה שיוצר בעיית הקצאה ובעיית סדר בו-זמנית.

המורכבות החישובית של ה-VRP מסווגת כבעיית NP-Hard, מה שמעיד על כך שלא קיים אלגוריתם המסוגל למצוא פתרון אופטימלי המובטח בזמן ריצה יעיל עבור בעיות בקנה מידה רחב.

ככל שמספר הצמתים והרכבים גדל, מספר השילובים האפשריים של המסלולים צומח בקצב מהיר במיוחד, מה שהופך סריקה מלאה של כלל האפשרויות לבלתי ישימה מבחינה חישובית. בשל כך, פותחו לאורך השנים וריאציות רבות של הבעיה המשקפות אילוצים מהעולם האמיתי, כמו VRP עם מגבלות קיבולת שבו לכל רכב יש מטען מקסימלי, או VRP עם חלונות זמן הדורש הגעה ללקוחות בטווח שעות מוגדר. אילוצים אלו הופכים את מרחב הפתרונות האפשריים למצומצם וקשה יותר לניווט, שכן פתרון שנראה קצר מבחינת מרחק עלול להיפסל עקב איחור ללקוח או חריגה ממשקל המטען המותר.

2.4 אתגרים מרכזיים במחקר

מחקר ופיתוח של מערכת אופטימיזציה לניהול משלוחים דורשים התמודדות עם אתגרים אלגוריתמיים ומעשיים מורכבים. הפיכת בעיה מתמטית תיאורטית לכלי יישומי ואפקטיבי עבור חברות לוגיסטיקה מחייבת פתרון של מספר בעיות ליבה, הנעות בין סיבוכיות המחשוב לבין הצורך לייצג במדויק את המציאות בשטח. בעיית ניתוב כלי רכב (VRP) מסווגת מבחינה אלגוריתמית כבעיה קשה לפתרון. משמעות הדבר היא שעם כל לקוח או חבילה נוספים שמתווספים למערכת, מספר המסלולים האפשריים גדל בצורה מעריכית. האתגר המחקרי המרכזי באלגוריתם הגנטי הוא ניווט יעיל במרחב עצום זה.

2.4.1 התמודדות עם הבעיה

התמודדות עם בעיית ניתוב כלי הרכב מציבה אתגר אלגוריתמי מורכב במיוחד, היות והיא מסווגת כבעיית אופטימיזציה קשה לפתרון (NP-Hard). ככל שמספר הלקוחות והחבילות גדל, כמות השילובים

האפשריים למסלולים צומחת בקצב מעריכי (אקספוננציאלית), מה שמונע מראש את האפשרות לבצע סריקה מלאה וממצה של כלל האפשרויות בזמן סביר. כדי להתמודד עם קושי זה, המערכת מיישמת מנוע חישוב אלגוריתמי חכם, כדי לשפר את איכות הפתרונות.

2.4.2 הסיבות והמוטיבציה לבחירת הנושא

המוטיבציה המרכזית לבחירת הנושא הגיעה מתוך התבוננות בחיים האמיתיים ובפער הגדול שקיים כיום בשוק הלוגיסטיקה והמשלוחים. כולנו נתקלים ביום-יום בנושא המשלוחים, בין אם אנחנו מחכים בבית לחבילה שתגיע בזמן, ובין אם אנחנו רואים ברחוב משאיות חלוקה שמנסות לתמרן בכבישים צפופים. בעוד שלחברות ענק יש תקציבי עתק למערכות ניהול מתוחכמות, עסקים קטנים ובינוניים רבים עדיין נאלצים לתכנן את מסלולי הנסיעה של הנהגים שלהם בצורה ידנית. המחשבה על מנהל עבודה או סדרן שיושב מדי יום שעות ארוכות ומנסה לנחש איזה נהג ייסע לאיזו כתובת, מבלי שיש לו דרך אמיתית לחשב משקלים, שעות הגעה או מרחקים, יצרה רצון חזק לפתח פתרון שיהיה נגיש, פשוט ויעיל עבורם. מעבר לצורך המעשי של העסקים, הבחירה בנושא זה נבעה מהרצון להתמודד עם אתגר מקצועי אמיתי ומקיף בתור מפתח תוכנה. בעיית ניתוב כלי הרכב היא אחת הבעיות המפורסמות והקשות ביותר לפיצוח בעולם המחשבים.

המוטיבציה הייתה לא רק לפתור תרגיל תיאורטי על הנייר, אלא ליצור מוצר שלם וידידותי. הרעיון הוא לקחת את כל החישובים המסובכים, המרחקים והאילוצים, ולהחביא אותם מאחורי הקלעים, כדי שעסקים קטנים ומשתמשי קצה יקבלו חוויה פשוטה לחלוטין.

2.4.3 על איזה צורך הפרויקט עונה ?

מערכת תכנון המשלוחים פותחה כדי לתת מענה לכמה צרכים אנושיים, תפעוליים וכלכליים מרכזיים שקיימים בכל עסק שמנהל מערך הפצה:

- **הצורך בחיסכון בזמן ומניעת שגיאות אנוש:** במקום שסדרן העבודה או מנהל העסק ישבו שעות ארוכות בכל יום מול רשימות כתובות, וינסו לשבור את הראש מי נוסע לאן, ויבזבזו זמן ניהולי יקר, המערכת עונה על הצורך בפתרון מהיר. היא מחליפה את התכנון הידני המתיש ומאפשרת לקבל תוכנית עבודה שלמה, מסודרת והגיונית בלחיצת כפתור אחת ותוך שניות ספורות.
- **הצורך בצמצום הוצאות הדלק והנסיעות המיותרות:** נסיעות ארוכות, מסלולים מפותלים או מצבים שבהם שני נהגים שונים מגיעים במקרה לאותו רחוב בדיוק, יוצרים לעסק הפסדים כספיים כבדים. המערכת עונה על הצורך הכלכלי לקצץ בהוצאות התפעול. על ידי בניית מסלול נסיעה חכם ויעיל שמתחשב ברשת הכבישים האמיתית, היא חוסכת קילומטרים רבים של נסועה מיותרת, שומרת על כלי הרכב מפני שחיקה, ומורידה משמעותית את חשבון הדלק של החברה.

- **הצורך בלקוחות מרוצים ובשירות אמין:** בעולם של היום, לקוחות לא מוכנים לחכות "כל היום" בבית שהשליח יגיע; הם מבקשים לדעת שעה מדויקת ורוצים שהעסק יעמוד במילה שלו. המערכת עונה בדיוק על הצורך הזה בשירות איכותי. היא מוודאת שהנהגים יגיעו לכל יעד בדיוק בטווח השעות שהלקוח ביקש, מונעת איחורים מתסכלים ועוזרת לעסק לשמור על מוניטין מקצועי ואמין.
- **הצורך בחלוקת עבודה הוגנת ושמירה על הנהגים:** אחד האתגרים הכי פחות מטופלים בתחום המשלוחים הוא הרווחה של הנהגים עצמם. פעמים רבות קורה שנהג אחד מקבל קו עמוס ומתיש ומסיים את היום שחוק ועייף, בזמן שנהג אחר מקבל קו קצר וקל ונשאר ללא פעילות. המערכת עונה על הצורך החברתי והניהולי באיזון; היא דואגת לחלק את עומס החבילות בצורה שוויונית והגיונית בין כל הנהגים הזמינים, דבר שמונע שחיקת עובדים, תורם לאווירה טובה בעסק ומייעל את השימוש בכל המשאבים.

3. מטרות ויעדים לפיתוח המערכת

- מטרות העל של הפרויקט מתמקדות בפתרון הכשלים של תכנון המסלולים הידני והפיכתו לתהליך דיגיטלי, מחושב וחכם:
- **ממשק ניהול פשוט ואינטראקטיבי:** פיתוח סביבת עבודה ידידותית ונוחה למשתמש, המאפשרת להזין ולנהל בקלות את רשימת המשלוחים היומית, את פרטי כלי הרכב הזמינים ואת מיקומי מחסני היציאה.
 - **מנוע חישוב חכם מבוסס אילוצים מרובים:** בניית מנגנון אלגוריתמי שמסוגל לעבד בו-זמנית מערכת מורכבת של מגבלות: כגון משקל ונפח החבילות מול הקיבולת המקסימלית של כל רכב, יחד עם שעות האספקה המבוקשות.
 - **תכנון מסלולים מבוסס כבישים אמיתיים:** שילוב נתוני מפה ריאליסטיים כך שחישוב המרחקים וזמני ההגעה יתבססו על רשת הדרכים, הפניות וחוקי התנועה הממשיים בשטח, ולא על מרחקים אוויריים תיאורטיים בקו ישר.
 - **הצגה חזותית וברורה על גבי מפה:** יצירת רכיב ויזואלי המציג בצורה צבעונית וברורה את מסלולי הנסיעה שנבחרו עבור כל רכב, כולל סדר התחנות המדויק, כדי לאפשר הבנה מהירה ונוחה של תוכנית החלוקה.
 - **מהירות תגובה וביצועים בזמן אמת:** הבטחה שמנוע האופטימיזציה יצליח לעבד את כלל הנתונים ולהפיק את תוכנית המסלולים המשופרת בתוך שניות בודדות, כדי להעניק לעסק גמישות תפעולית מלאה.
 - **איזון משאבים וחלוקת עבודה הוגנת:** ליצור סביבת עבודה הוגנת ושוויונית עבור הנהגים בצי, על ידי מניעת מצבים בהם נהג אחד קורס תחת עומס משלוחים חריג בעוד נהג אחר נותר כמעט ללא פעילות.

4. אתגרים בבניית המערכת

מעבר לאתגרי המחקר התיאורטיים, תרגום הרעיון לכדי מערכת עובדת, אינטראקטיבית ושלמה הציב בפנינו מספר אתגרים מעשיים במהלך הבנייה והפיתוח:

- **בניית ממשק ניהול פשוט לפעולה מורכבת:** מאחר שהרצת האלגוריתם וניהול מערך המשלוחים מתבצעים על ידי משתמש האדמין בלבד, האתגר הגדול היה לעצב עבורו סביבת עבודה שלא תעמיס עליו. היינו צריכים לפצח איך לרכז עבור המנהל את כל המידע הרגיש, הזנת הנהגים, ניהול החבילות והגדרת האילוצים - במסכים נקיים ואינטואיטיביים, כך שיוכל להפעיל את המנוע החישוב הכבד בלחיצת כפתור פשוטה וללא צורך בידע טכני מוקדם.
- **חיבור וסנכרון בין מנוע החישוב למפה הוויזואלית:** אחד האתגרים המרכזיים בבנייה היה הזרמת הנתונים בצורה חלקה. ברגע שמשתמש האדמין נותן את הפקודה לחשב מסלולים, המערכת צריכה לקחת את הפתרון המתמטי שנוצר מאחורי הקלעים, לתרגם אותו לקואורדינטות גיאוגרפיות מדויקות, ולהציג אותו למנהל באופן מיידי כקווים צבעוניים וברורים על גבי המפה האינטראקטיבית.
- **ניהול זיכרון וביצועים בזמן הרצת האלגוריתם:** מאחר שהאלגוריתם מבצע המון פעולות חישוב ובדיקות בשניות בודדות, בניית המערכת דרשה תכנון קפדני של זרימת המידע במחשב. האתגר היה לוודא שזמן החישוב יישאר קצר ככל הניתן ושמסך הניהול של האדמין לא "יקפא" או יקרוס בזמן שהתוכנה מעבדת את האפשרויות השונות, אלא יציג חוויית שימוש רציפה ומהירה.
- **שמירה על סדר ועקביות בנתוני העסק:** המערכת נדרשת לנהל ישויות רבות שמשתנות כל הזמן (כמו חבילות שמתווספות). האתגר בבנייה היה ליצור מבנה נתונים גמיש ויציב שימנע כפילויות או סתירות. למשל, לוודא שחבילה לא תשוך בטעות לשני נהגים שונים, או שהמערכת לא תאפשר לאדמין לשלוח רכב שאינו זמין באותו יום.

5. הגדרת מדדי הצלחה למערכת

כדי להעריך את יעילות המערכת ולוודא שהיא אכן עומדת בציפיות ובצרכים שלשםם פותחה, הוגדרו חמישה מדדי הצלחה מרכזיים, הניתנים לבחינה מעשית בשטח:

- **מדד זמן תכנון המסלולים (עבור האדמין):** המערכת תיחשב להצלחה אם היא תצליח לקצר את זמן העבודה של משתמש האדמין בצורה דרמטית. המטרה היא שתהליך תכנון של מערך משלוחים יומי, שלוקח בדרכ כלל שעות ארוכות של עבודה ידנית ומתישה, יתבצע באופן אוטומטי לחלוטין ויספק לאדמין תוכנית עבודה מלאה בתוך פחות מדקה מרגע לחיצת הכפתור.
- **מדד החיסכון בנסועה ובעלויות (מרחק כולל):** הצלחת המערכת תימדד ביכולתה להפחית את סך הקילומטרים המצטבר שצי הרכב מבצע בכל יום. המערכת שואפת להציג ירידה משמעותית

במרחקי הנסיעה ובזמני השהות על הכביש בהשוואה לתכנון האנושי הישן, דבר שיתבטא ישירות בחיסכון כספי בהוצאות הדלק ובבלאי של כלי הרכב.

- **מדד חוקיות המסלולים ועמידה באילוצים:** המערכת חייבת להציג 100% הצלחה בהפקת מסלולים חוקיים ותקינים. המשמעות היא שאף מסלול שמיוצר על ידי התוכנה לא יחרוג ממגבלת המשקל או הנפח המותרים של הרכב (אילוץ קיבולת).
- **מדד איזון העומסים בין הנהגים:** הצלחת המערכת תיבחן ברמת ההוגנות והשוויוניות של חלוקת המשימות. המדד ייחשב לחיובי ככל שהמערכת תצליח להימנע ממצבי קיצון (כמו נהג אחד שעובד שעות ארוכות מול נהג אחר שיושב בחוסר מעש), ותדע לפזר את כמות החבילות והעצירות בצורה מאוזנת והגיונית בין כלל הנהגים שמשמש האדמין הגדיר כזמינים באותו יום.
- **מדד חוויית המשתמש ופשטות ההפעלה:** מאחר שניהול המערכת מרוכז כולו בידי האדמין, מדד הצלחה קריטי הוא קלות התפעול. המערכת תיחשב למוצלחת אם האדמין יוכל לנהל את המידע, להפעיל את האלגוריתם, ולקרוז את מפת המסלולים הסופית בצורה חלקה, פשוטה ומהירה, ללא צורך בליווי טכני מתמשך או בהכשרה מורכבת.

6. תיאור פתרונות קיימים לבעיה וניתוח חלופות

כדי להתמודד עם מורכבותה החישובית של בעיית ניתוב כלי הרכב, (VRP) נבחנו לאורך השנים מגוון רחב של גישות ופתרונות קיימים בספרות המקצועית. גישות אלו נעות בין אלגוריתמים מתמטיים מדויקים לבין שיטות היוריסטיות ומטה-היוריסטיות, כאשר כל חלופה מציגה איזון שונה בין איכות הפתרון (אופטימליות) לבין משאבי המחשוב וזמן הריצה הנדרשים.

הקבוצה הראשונה שנבחנה כוללת שיטות מתמטיות מדויקות, ובראשן אלגוריתם ענף וחתך Branch and Cut. גישה זו מבוססת על חלוקת הבעיה לעץ החלטות והוספת חסמים מתמטיים קשיחים לאורך תהליך החיפוש, המאפשרים לקצץ ענפים שלמים במרחב הפתרונות שאינם רלוונטיים. היתרון המרכזי של חלופה זו הוא ההבטחה המתמטית המוחלטת להגעה אל הפתרון האופטימלי והטוב ביותר האפשרי מתוך כלל מרחב המדגם, אך חסרונה המשמעותי טמון בדרישות זיכרון וזמן חישוב קיצוניות במיוחד, ההופכות לבלתי נשלטות ככל שמימדי הבעיה גדלים, דבר המונע את יישומו בזמן אמת. שיטה מדויקת נוספת היא תכנון דינמי (Dynamic Programming) המתבססת על פירוק בעיית הניתוב למשימות קטנות יותר ושמירת תוצאות הביניים של תתי-הבעיות בזיכרון המחשב כדי למנוע חישובים חוזרים ונשנים עבור מסלולים זהים. בעוד שגישה זו מציגה יעילות גבוהה במציאת פתרונות אופטימליים לתת-מסלולים מוגדרים היטב, היא סובלת מתופעת "קללת הממדיות" ומובילה לקריסה חישובית מהירה של המערכת ככל שמספר הלקוחות, האילוצים התפעוליים וחלונות הזמן הולך וגדל. כחלופה מהירה לשיטות המדויקות, נבחנה היוריסטיקת החיסכון של קלארק-רייט Clarke-Wright Savings. זוהי היוריסטיקה קלאסית המתחילה במצב קצה שבו כל לקוח מקבל שירות ברכב ייעודי נפרד, ומספקת איחוד הדרגתי של מסלולים בודדים בהתבסס על מדד של מקסימום חיסכון במרחק

הנסיעה המצטבר. אלגוריתם זה מתאפיין במהירות ביצוע יוצאת דופן וכמעט מיידי, ומסוגל להפיק פתרונות מהירים גם עבור בעיות רחבות ומרובות צמתיים, אך חסרונם המרכזי נובע מהיותו פתרון "תאב בצע", המקבל החלטות מקומיות בלבד, ולכן נוטה פעמים רבות לפספס את האופטימום הגלובלי ולהיתקע בפתרון שאינו אידיאלי למערך כולו.

במעבר לשיטות מטה-היוריסטיות מתקדמות, נבחן אלגוריתם חיפוש טאבו (Tabu Search) המבוסס על חיפוש מקומי הבוחן מסלולים תוך שימוש ברשימת איסור דינמית ("טאבו") החוסמת מהלכים קודמים שבוצעו לאחרונה ומאלצת את האלגוריתם לחקור אזורים חדשים במרחב. היתרון המרכזי בגישה זו הוא מניעה יעילה ביותר של חזרה למעגלים סגורים בחיפוש, ובכך מסייעת למערכת להיחלץ מנקודות אופטימום מקומיות, אך יעילות השיטה מציגה תלות גבוהה מאוד בכיול מדויק של פרמטרי הריצה וגודל רשימת הזיכרון, דבר הדורש משאבי ניסוי וטעייה רבים. חלופה מטה-היוריסטית נוספת היא חיפוש מדומה (Simulated Annealing), השואבת השראה מתהליך הקירור הפיזיקלי של מתכות ומאפשרת באופן מבוקר והסתברותי הרעה זמנית באיכות הפתרון בשלבים הראשוניים של הריצה. הגישה מציגה יכולת חקירה מצוינת ויעילה של מרחבי פתרון מורכבים, מקוטעים ולא רציפים, ומצטיינת בדילוג מעל מכשולי אופטימום מקומי, אך מנגד, משך ההגעה לפתרון איכותי ויציב עלול לקחת זמן רב וממושך עקב תהליך ה"קירור" האיטי וההדרגתי המובנה באלגוריתם.

לבסוף, נבחן האלגוריתם הגנטי (Genetic Algorithm), שהוא מטה-היוריסטיקה המבוססת על עקרונות האבולוציה והברירה הטבעית. אלגוריתם זה מנהל אוכלוסייה של מסלולי הפצה ומפתח אותם לאורך דורות באמצעות תהליכי סלקציה של הפתרונות המבטיחים, הצלבה ביניהם ליצירת פתרונות חדשים, והפעלה מבוקרת של מוטציות אקראיות. היתרון המרכזי והבולט של החלופה הגנטית הוא גמישותה המרתקת ויכולת ההתמודדות היוצאת דופן שלה עם טיפול באילוצים מרובים במקביל, כגון שילוב בין נפחי קיבולת, חלונות זמן קשיחים, דרישה לתכנון על בסיס כבישים אמיתיים וחלוקת עומס מאוזנת ומשובחת בין הנהגים. יחד עם זאת, חסרונם המשמעותי טמון במורכבות התכנותית והמבנית הגבוהה שהוא מציב בעיצוב ה"כרומוזום" והאופרטורים האבולוציוניים, במטרה להבטיח שכל פתרון חדש שנוצר יישאר מסלול חוקי, הגיוני ותקין לחלוטין מבחינה לוגיסטית.

7. תיאור החלופה הנבחרת ונימוקים לבחירתה

הגישה הנבחרת לניהול וניתוח מסלולי המשלוחים במערכת הוא האלגוריתם הגנטי. הבחירה במנגנון זה נעשתה לאחר בחינה של מספר פתרונות קיימים, ולאחר מכן להגיע למסקנה שהאלגוריתם הגנטי מציע את האיזון המושלם בין מהירות הריצה של התוכנה לבין איכות ומציאותיות המסלולים שמתקבלים בסופו של דבר.

במהלך הניתוח נפסלו השיטות המתמטיות המדויקות (כמו ענף וחתך או תכנון דינמי), מכיוון שהן "כבדות" מדי ומיועדות בעיקר לבעיות קטנות; ברגע שמספר המשלוחים והנהגים גדל, המחשב עלול להתעכב שעות ארוכות בחישובים, דבר שאינו מעשי למערכת שאמורה לעבוד בזמן אמת בעולם

האמיתי. מנגד, אלגוריתמים פשוטים מדי (כמו אלגוריתם החיסכון של קלארק-רייט) אמנם פועלים בשניות, אך הם "אינטרסנטיים" וקצרי רואי – הם בוחרים תמיד בצעד הבא שנראה הכי קרוב באותו רגע ומפספסים את התמונה המלאה, מה שמוביל למסלולים מפותלים ולא יעילים. בניגוד לשיטות אחרות שדורשות נוסחאות קשיחות, האלגוריתם הגנטי עובד בצורה הדומה לאבולוציה בטבע. הוא מייצר המון אפשרויות למסלולים, בודק אילו מהן עומדות הכי טוב בדרישות, ומשפר אותן מדור לדור. זה מאפשר לנו "להנחית" עליו בקלות מגוון אילוצים במקביל, כמו מגבלת המשקל שהרכב יכול לסחוב. מבלי לשבור את הגירון של האלגוריתם. בזכות שימוש ב"מוטציות" (שינויים אקראיים קטנים שהאלגוריתם מבצע תוך כדי תנועה), המערכת לא נתקעת על הפתרון הראשון שנראה יעיל. היא ממשיכה לחקור רעיונות חדשים ויצירתיים לאורך הריצה, מה שמבטיח שבסופו של דבר נקבל את תוכנית המשלוחים היעילה והרווחית ביותר עבור העסק.

8. אפיון המערכת המוצעת

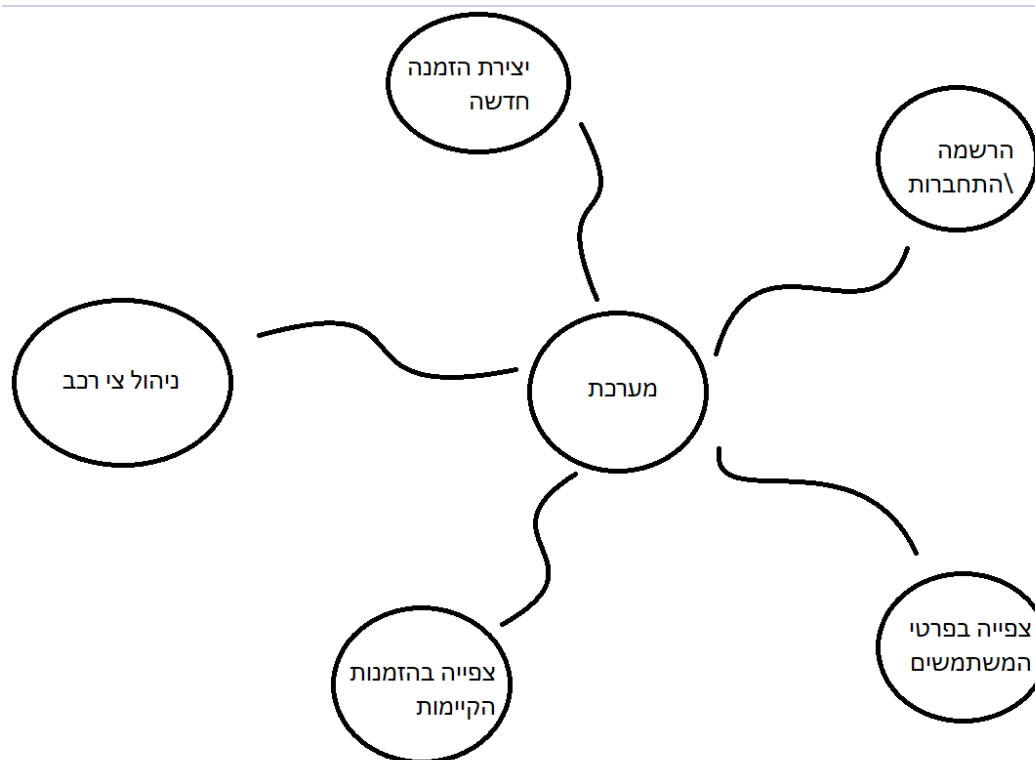
8.1 ניתוח הדרישות מהמערכת

ניתוח הדרישות הוא שלב מרכזי בתכנון המערכת, שבו מגדירים בצורה ברורה מה המערכת צריכה לעשות ואילו תנאים היא חייבת לקיים. הדרישות מתחלקות לשני סוגים: דרישות פונקציונליות, שמתארות את הפעולות והיכולות של המערכת, ודרישות לא-פונקציונליות, שמתארות את איכות הביצועים, האמינות וההתנהגות הכללית שלה.

- המערכת צריכה לתמוך בניהול משתמשים והרשאות, כך שיהיו רמות גישה שונות. משתמשים רגילים כמו נהגים יכולים רק לצפות במסלולים שלהם, בעוד שהאדמין הוא היחיד שיכול לנהל את המערכת, להזין נתונים ולהפעיל את מנוע האופטימיזציה.
- בנוסף, המערכת צריכה לאפשר ביצוע פעולות CRUD (הוספה, קריאה, עדכון ומחיקה) על כל הנתונים המרכזיים: הזמנות ולקוחות עם פרטי משלוח כמו משקל, נפח וחלונות זמן, ניהול צי רכבים כולל קיבולת ושיוך למחסנים, והגדרת מחסנים לוגיסטיים המשמשים כנקודות יציאה וחזרה.
- דרישה מרכזית נוספת היא הפעלת מנוע האופטימיזציה בלחיצת כפתור אחת, כך שהמערכת תעבד את כל הנתונים ותייצר את המסלולים היעילים ביותר לכלל הרכבים בהתאם לאילוצים.
- לבסוף, המערכת צריכה להציג את תוצאות החישוב בצורה ויזואלית וברורה על גבי מפה אינטראקטיבית, כולל מסלולים מסומנים לכל רכב וסדר תחנות מדויק.

- המערכת חייבת לשמור על נתונים בצורה יציבה וקבועה בבסיס נתונים בענן, כך שלא יאבד מידע גם במקרה של כשל או כיבוי של השרת. כל הנתונים צריכים להיות זמינים לשליפה מהירה בכל זמן.
- בנוסף, נדרש זמן תגובה מהיר במיוחד של מנוע החישוב, כך שתוצאות האופטימיזציה יתקבלו תוך שניות בודדות כדי לא לפגוע בעבודה השוטפת של המשתמש.
- המערכת צריכה להיות יעילה מבחינת שימוש במשאבים, כולל ניהול נכון של זיכרון ומעבד בזמן הרצת האלגוריתם, ושחרור משאבים מיד בסיום התהליך כדי למנוע עומסים מיותרים.
- כמו כן, נדרשת זמינות גבוהה של המערכת לאורך כל שעות הפעילות, כך שהאדמין יוכל לגשת למערכת בכל רגע נתון.
- לבסוף, יש חשיבות גבוהה לאבטחת מידע, כך שכל הנתונים יועברו בצורה מאובטחת, ושלא תהיה גישה ישירה לבסיס הנתונים אלא רק דרך שכבת ה-Backend כדי לשמור על שלמות ובידוד המידע.

8.2 מודלי המערכת



8.3 אפיון פונקציונלי וביצועים עיקריים

המערכת כוללת ניהול משתמשים מבוסס תפקידים (RBAC) אשר מגדיר הפרדה ברורה ברמת הקוד בין משתמשים רגילים לבין מנהלי מערכת. הפרדה זו באה לידי ביטוי הן בצד ה-UI והן בצד ה-Backend, כך שלקוח רגיל ניגש רק לפונקציות של יצירת משלוחים ומעקב אחר הזמנות, בעוד שמנהל המערכת מקבל גישה מלאה לניהול תשתיות, צי רכבים והרצת האלגוריתם. גישה זו משפרת את אבטחת המידע ומונעת גישה לא מורשית לפונקציות קריטיות.

בנוסף, קיימת יכולת של אימות כתובות ומיקוד גיאוגרפי (Geocoding) באמצעות שילוב של Google Maps API. המערכת מקבלת כתובות טקסטואליות שהוזנו על ידי המשתמש ומתרגמת אותן לקואורדינטות גיאוגרפיות מדויקות. תהליך זה כולל בדיקה ואימות של הנתונים, כך שכתובות לא תקינות או לא מזוהות נחסמות ואינן נכנסות לתהליך הלוגיסטי, דבר המבטיח אמינות גבוהה של נתוני המשלוחים.

יכולת מרכזית נוספת היא מודול האופטימיזציה הגנטית, המשמש כמנוע החישוב המרכזי של המערכת. מודול זה מבצע סריקה של מספר רב של קומבינציות אפשריות לחלוקת הזמנות בין רכבים ובניית מסלולים, תוך ניסיון להגיע לפתרון האופטימלי ביותר. במהלך התהליך האלגוריתם מתחשב באילוצים שונים כגון קיבולת רכבים, משקל חבילות וחלונות זמן של לקוחות, ובכך מייצר חלוקה יעילה וחכמה של המשימות הלוגיסטיות.

לבסוף, המערכת כוללת המחשה חזותית דינמית של תוצאות האלגוריתם. המסלולים מחושבים ומוצגים על גבי מפה אינטראקטיבית בזמן אמת, כאשר כל מסלול מסומן בצבע ייחודי בהתאם לרכב המשויך אליו. בנוסף לכך מוצג פאנל מידע נלווה המציג את סדר התחנות המדויק עבור כל רכב, מה שמאפשר למשתמש להבין בצורה ברורה את תהליך הביצוע של המערכת ואת תוצאות האופטימיזציה בצורה אינטואיטיבית וחזותית.

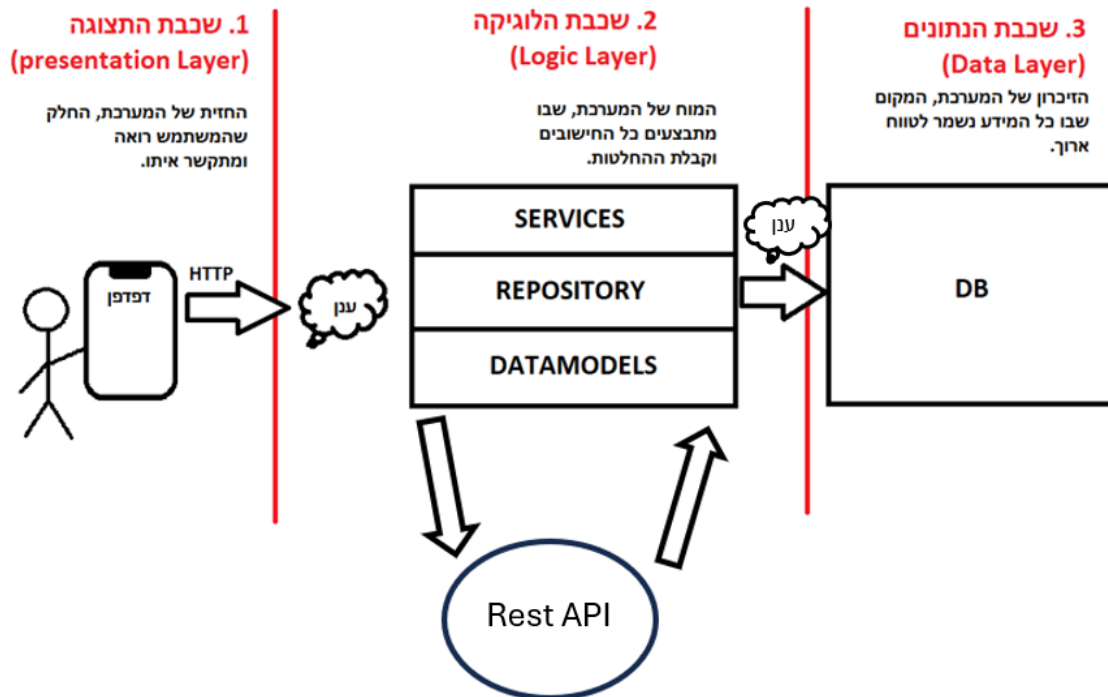
8.4 אילוצי המערכת

המערכת מבוססת על מספר אילוצים טכנולוגיים מרכזיים שמגדירים את ארכיטקטורת הפיתוח שלה ומכתיבים את אופן המימוש בפועל.

- האילוץ הראשון הוא אילוץ שפת הפיתוח והתשתית, לפיו כלל המערכת מבוססת על Java 21 בצד השרת יחד עם Spring Boot כתשתית ה-Backend המרכזית. בצד הלקוח נעשה שימוש ב-Vaadin, כך שהממשק נבנה כחלק אינטגרלי מהשרת ולא באמצעות מסגרות Frontend חיצוניות כגון React. החלטה זו יוצרת ארכיטקטורה אחודה על בסיס JAVA שבה כל שכבות המערכת מנוהלות באותה סביבה, דבר שמפשט את הפיתוח, התחזוקה והאינטגרציה בין רכיבי המערכת.
- האילוץ השני הוא אילוץ מסד הנתונים, לפיו כל ניהול המידע במערכת מתבצע באמצעות מסד נתונים מסוג NoSQL בלבד, ובפרט MongoDB Atlas בענן. שימוש זה שולל לחלוטין שימוש במסדי נתונים רלציוניים-SQL, ולכן מחייב תכנון נתונים מבוסס מסמכים. המשמעות היא שכל ישות במערכת נשמרת כאובייקט עצמאי, ללא שימוש בהצלבות מורכבות (Joins) מה שמאפשר גמישות גבוהה בשינויים והרחבות עתידיות של מבנה הנתונים.
- האילוץ השלישי נוגע לתלות בשירותים חיצוניים-API, כאשר חלקים מרכזיים בלוגיקת המערכת תלויים באופן ישיר בשירות Google Maps API. שירות זה משמש לביצוע גיאוקודינג (המרת כתובות לקואורדינטות), חישוב מרחקים בין נקודות, והצגת מסלולים על גבי מפה אינטראקטיבית. לצורך כך המערכת עושה שימוש במפתח גישה ייעודי (google.maps.api.key) אשר מנוהל בקובצי הקונפיגורציה של השרת באופן מאובטח, כדי להבטיח גישה יציבה ומבוקרת לשירותי המפות החיצוניים.

9. תיאור הארכיטקטורה של המערכת

9.1 ארכיטקטורת המערכת



כדי להבטיח הפרדת סמכויות מלאה, גמישות בתחזוקה ויכולת הרחבה עתידית של המוצר, המערכת תוכננה ונבנתה על בסיס **ארכיטקטורת שלוש השכבות**. מבנה זה מחלק את המערכת לשלושה רבדים עצמאיים הלוגית, המקיימים ביניהם קשרי תקשורת מוגדרים וברורים: שכבת התצוגה, שכבת הלוגיקה העסקית, ושכבת הנתונים.

- שכבת התצוגה והממשק:** החלק במערכת שאיתו המשתמש עובד ישירות דרך דפדפן האינטרנט. תפקידה הוא להציג מידע בצורה ברורה ונוחה, לאפשר הזנה ועדכון של נתונים, להעביר פעולות ובקשות לשכבת הלוגיקה של המערכת, ולהציג את התוצאות שהתקבלו מהאלגוריתם בצורה ויזואלית. בתוך שכבה זו קיימים מסכי ניהול שמאפשרים לאדמין לנהל לקוחות, חבילות, מחסנים וכלי רכב, יחד עם רכיב מפה אינטראקטיבי שמציג את מיקומי היעדים ואת מסלולי הנסיעה של הנהגים בקווים צבעוניים וברורים. בנוסף, השכבה כוללת רכיבי קלט כמו כפתורים, טפסים ותפריטי בחירה, שבעזרתם ניתן לשלוט על פעולות המערכת ולהגדיר אילוצים שונים כמו קיבולת רכבים או חלונות זמן למשלוחים.
- שכבת הלוגיקה העסקית:** החלק המרכזי במערכת ונחשבת למוח שלה. שכבה זו רצה על שרת Tomcat ייעודי ואחראית על כל העיבוד והחישובים שמתבצעים מאחורי הקלעים, בלי קשר לעיצוב או למראה של הממשק. היא מקבלת בקשות מה-Frontend, מעבדת את הנתונים, מתקשרת עם בסיס הנתונים ומחזירה תוצאות מוכנות להצגה. בתוך השכבה פועל בקר היישום שמקבל את הבקשות מהמערכת ומעביר אותן לשירות המתאים. בנוסף נמצא

מנוע האופטימיזציה המבוסס על אלגוריתם גנטי, שתפקידו לחשב את מסלולי החלוקה הטובים ביותר לפי אילוצים כמו קיבולת רכבים וזמני הגעה. קיימת גם מערכת אינטגרציה עם שירותי מפות, שמביאה נתוני מרחקים וזמני נסיעה אמיתיים מהכבישים, ולא לפי מרחק אווירי. מעבר לכך, שכבת השירותים העסקיים אחראית על ניהול ההזמנות, בדיקות תקינות של הנתונים, ואימות שהמסלולים עומדים במגבלות כמו משקל ונפח הרכב.

- **שכבת הנתונים:** המקום שבו נשמר כל המידע של המערכת בצורה קבועה ומאובטחת בענן. התפקיד שלה הוא לשמור את הנתונים בצורה אמינה כך שהם לא יימחקו גם אם המערכת נסגרת או השרת נכבה. הגישה לנתונים מתבצעת רק דרך שכבת ה-Backend, מה שמוסיף אבטחה וסדר למערכת. בתוך בסיס הנתונים קיימים אוספים שונים שמנהלים את המידע: אוסף משתמשים ששומר פרטי התחברות והרשאות, אוסף הזמנות שבו נמצאים פרטי המשלוחים כמו כתובות, זמני הגעה, משקל ונפח החבילות, אוסף רכבים שמכיל מידע על כלי הרכב והקיבולת שלהם, ואוסף מחסנים שבו נשמרים מיקומי נקודות היציאה והחזרה של הנהגים. זרימת המידע במערכת מתחילה כאשר האדמין נכנס למסך הניהול ב-Frontend ולוחץ על כפתור לחישוב מסלול אופטימלי. הבקשה נשלחת אל שכבת ה-Backend שמושכת מבסיס הנתונים את כל המידע הדרוש כמו פרטי החבילות, הרכבים, המחסנים ואילוצי הזמן. לאחר מכן, ה-Backend פונה לשירותי המפות כדי לקבל מרחקים וזמני נסיעה אמיתיים, ומעביר את כל הנתונים למנוע האלגוריתם הגנטי. האלגוריתם מבצע חישובים רבים בזמן קצר ובונה את מסלולי החלוקה היעילים ביותר בהתאם לאילוצים שהוגדרו. בסיום התהליך, ה-Backend שומר את המסלולים החדשים בבסיס הנתונים ושולח את התוצאות ל-Frontend. לבסוף, שכבת התצוגה מציגה לאדמין בצורה ברורה את מסלולי הנסיעה על גבי המפה יחד עם לוחות הזמנים של כל נהג.

9.2 מודל שרת לקוח

מודל שרת-לקוח מהווה את התשתית התקשורתית המאפשרת את פעולתה של מערכת תכנון המסלולים, והוא מגדיר את חלוקת התפקידים בין ממשק המשתמש לבין מנוע החישוב. במודל זה, המערכת מופרדת לשתי ישויות מרכזיות המקיימות ביניהן אינטראקציה מתמדת של בקשה ותגובה. תפקידו של הלקוח (ה-Frontend) הוא לרכז את המידע הגולמי, לנהל את אירועי המשתמש ולשלוח אותם כבקשה מובנית אל השרת, מבלי לבצע את עיבוד הנתונים הכבד בעצמו. השרת (ה-Backend) פועל כיחידת העיבוד המרכזית והעוצמתית של המערכת. עם קבלת הבקשה מהלקוח, השרת מפעיל את האלגוריתמים האבולוציוניים המורכבים ומשתמש במשאבי מחשוב נרחבים כדי לסרוק מיליוני אפשרויות לסידור מסלולים. ההפרדה הזו חיונית מבחינה הנדסית, שכן היא מאפשרת למשתמש קצה להפעיל כלים טכנולוגיים מתקדמים גם ממכשירים ניידים או תחנות עבודה

בעלות כוח עיבוד מוגבל, תוך ריכוז הלוגיקה העסקית, גישה מאובטחת לבסיס הנתונים - MongoDB, והחישובים האינטנסיביים בשרת ייעודי ומאובטח.

התקשורת בין הישויות מתבצעת בפרוטוקול העברת נתונים סטנדרטי: HTTPS/HTTP. בסיום תהליך האופטימיזציה, השרת מנסח תגובה המכילה את הפתרון הנבחר ומעביר אותה חזרה ללקוח לצורך הצגה ויזואלית על גבי רכיב המפה. מבנה זה מבטיח סנכרון מלא בין בסיס הנתונים לבין המשתמשים השונים, ומקנה למערכת יכולת התרחבות וגמישות מבנית, המאפשרת לה להתרחב ולתמוך בכמות לקוחות הולכת וגדלה תוך שמירה על ביצועים גבוהים ויציבות לאורך זמן.

9.3 תהליכים של מע' ההפעלה

כדי שהמערכת תפעל בצורה תקינה ויציבה, היא נשענת על שירותי מערכת ההפעלה שמנהלים את כל המשאבים של השרת ומתווכים בין החומרה לבין התוכנה. מערכת ההפעלה אחראית על שלושה תחומים מרכזיים: תקשורת רשת, ניהול תהליכים וניהול זיכרון.

מערכת ההפעלה מפעילה את שרת היישום כתהליך עצמאי ומבודד בזיכרון ה-Ram, עם מרחב כתובות מוגן משלו. כדי לנצל טוב יותר את המעבד הרב-ליבתי ולשפר ביצועים, המערכת משתמשת במנגנון של ריבוי חוטים (Multithreading).

בשרת היישומים Tomcat מנוהל מנגנון זה באמצעות Thread pool. בעת הפעלת השרת נוצר מאגר קבוע של חוטים, וכשמגיעה בקשת HTTP מהדפדפן Tomcat, מקצה לה חוט פנוי מתוך המאגר. החוט מטפל בבקשה עד סיומה ולאחר מכן חוזר לברירה לשימוש חוזר, במקום להיווצר מחדש בכל פעם. כך מתקבלת עבודה יעילה יותר עם פחות עומס על המערכת.

הארכיטקטורה הזו מאפשרת למערכת לטפל בהרבה בקשות במקביל בצורה חלקה ויציבה. בנוסף, ב-Spring Boot, השירותים בנויים כ-Stateless, כלומר הם לא שומרים מצב משותף בין בקשות, מה שמאפשר לחוטים שונים להריץ את אותן פעולות במקביל בלי התנגשות או דריסת מידע. גם בבסיס הנתונים MongoDB, מנהל גישה מקבילית באמצעות Connection Pool ומנגנונים פנימיים שמבטיחים כתיבה וקריאה תקינות גם תחת עומס, תוך שמירה על שלמות הנתונים.

9.4 תיאור פרוטוקולי תקשורת

- **פרוטוקולי TCP/IP:** הם הבסיס שעליו מתבצעת כל התקשורת ברשת בין חלקי המערכת. הם אחראים לכך שהמידע יעבור בצורה תקינה, מסודרת ואמינה בין הדפדפן של האדמין לבין שרת המערכת. פרוטוקול TCP דואג לקחת את המידע הגדול שהמערכת שולחת, לחלק אותו לחבילות קטנות, ולוודא שכל החבילות מגיעות ליעד שלהן בצורה מלאה ובסדר הנכון. אם חלק מהמידע לא מגיע, TCP שולח אותו מחדש באופן אוטומטי. פרוטוקול IP אחראי על ניתוב החבילות ברשת ועל שליחתן לכתובת הנכונה, בדומה למערכת ניווט שמכוונת את המידע

ליעדו. השילוב בין שני הפרוטוקולים מאפשר למערכת לעבוד בצורה יציבה ואמינה, כך שהעברת הנתונים והרצת האלגוריתם יתבצעו במהירות וללא תקלות תקשורת.

- **REST API:** הוא ממשק תקשורת שמאפשר ל-Backend של המערכת לחשוף שירותים בצורה מסודרת ונגישה לשאר חלקי המערכת. כל פעולה או משאב במערכת מוגדרים כנקודת קצה (Endpoint) עם כתובת ייחודית, שאליה ניתן לשלוח בקשות ולקבל תשובות. כך, פעולות כמו שליפת הזמנות, עדכון נתונים או הפעלת אלגוריתם הניתוב מתבצעות דרך קריאות מוגדרות וברורות לשרת.
- **JSON:** פורמט סטנדרטי להעברת נתונים בין חלקי המערכת דרך ה-Rest API. כל המידע שעובר בין ה-Frontend, ה-Backend, בסיס הנתונים ושירותים חיצוניים כמו Google Maps נארז בצורה של טקסט מסודר וקל לקריאה. היתרון המרכזי של JSON הוא שהוא פשוט, קל משקל ומהיר לעיבוד, ולכן מתאים מאוד למערכות רשת. המבנה שלו בנוי בצורה היררכית של מפתחות וערכים, מה שמאפשר לארגן מידע מורכב בצורה ברורה, כמו פרטי לקוחות, הזמנות, רכבים וחלונות זמן למשלוחים, וגם את תוצאות המסלולים שהאלגוריתם מייצר. בזכות השימוש ב-JSON, כל חלקי המערכת עובדים באותו פורמט נתונים, מה שיוצר סנכרון מלא בין שכבת הנתונים, שכבת הלוגיקה ושכבת התצוגה, ומאפשר העברה יעילה ואחידה של מידע לאורך כל המערכת.

9.5 שירותים חיצוניים

- **Google maps API:** המערכת תתחבר למאגר המידע של גוגל כדי לקבל נתוני מרחק וזמן נסיעה מדויקים, המייצגים את הבסיס לחישוב המסלולים במערכת. נוכל גם להראות ויזואלית בעזרתה את מסלולי הרכבים.

10. ניתוח תרחישים וזרימת המידע במערכת המוצעת

10.1 תרשימי תרחיש Use case Diagram

להלן תרשימי Use Case שמסביר על הפעולות שמשתמש קצה מסוגל לעשות:



תרשים זה מתאר את הפעולות הזמינות למשתמש רגיל במערכת, בדרך כלל לקוח או עובד שטח, ואת האינטראקציה שלו עם מערכת ניהול המשלוחים. להלן הסבר מקרי השימוש עבור משתמש רגיל:

- **התחברות למערכת (Login)**

זהו שלב חובה שמאפשר גישה למערכת. המשתמש מזין פרטי התחברות, והמערכת בודקת אותם מול מסד הנתונים דרך ה Backend-לאחר אימות מוצלח מתקבלת הרשאת גישה שמאפשרת לבצע פעולות בהתאם להרשאות המשתמש.

- **יצירת הזמנת משלוח:**

המשתמש יכול ליצור הזמנה חדשה על ידי הזנת פרטי משלוח כמו כתובת יעד, משקל ונפח החבילה וחלון הזמן הרצוי לקבלת המשלוח. הנתונים נשמרים במערכת ומתווספים לרשימת ההזמנות לטיפול.

- **צפייה בהזמנות האישיות:**

המשתמש יכול לצפות בכל ההזמנות שהוא יצר, כולל סטטוס עדכני לכל הזמנה (למשל: ממתינ לעיבוד, בדרך למשלוח או סופק). פעולה זו מאפשרת מעקב שוטף אחרי מצב המשלוחים שלו.

להלן תרשים נוסף המייצג את פעולות ואופן השימוש אצל משתמש האדמיין:



תרשים זה מתאר את פעולות הניהול המרכזיות במערכת, אשר נגישות אך ורק למנהל המערכת (האדמין). תפקידו של האדמין הוא לשלוט בליבה התפעולית של המערכת, לנהל את המשאבים, ולהפעיל את מנוע האופטימיזציה. להלן הסבר מקרי השימוש עבור מנהל המערכת:

התחברות למערכת: (Login)

בדומה למשתמש רגיל, גם האדמין נדרש לבצע התחברות למערכת. לאחר אימות זהותו והגדרת רמת ההרשאה שלו כמנהל, נפתחות בפניו כלל אפשרויות הניהול המתקדמות של המערכת.

ניהול וצפייה בצי הרכבים:

האדמין יכול לנהל את כלל כלי הרכב במערכת, כולל הוספה של רכבים חדשים, ומחיקת רכבים קיימים. פעולה זו מאפשרת שליטה מלאה במשאבי ההובלה של המערכת.

צפייה בפרטי המשתמשים:

האדמין יכול לצפות בכל המשתמשים הרשומים במערכת, כולל פרטי הזיהוי שלהם ונתונים כלליים. מקרה שימוש זה נועד לפיקוח וניהול תקין של משתמשי המערכת.

הרצת אלגוריתם הניתוב (מקרה שימוש מרכזי):

זהו מקרה השימוש המרכזי של המערכת. בלחיצת כפתור, האדמין מפעיל את מנוע האופטימיזציה. המערכת אוספת את כל ההזמנות והרכבים הזמינים, שולחת בקשה לשירותי מפות לקבלת זמני נסיעה אמיתיים, ומריצה את האלגוריתם הגנטי לחישוב המסלולים האופטימליים. לאחר סיום החישוב, התוצאות נשמרות במסד הנתונים ומוצגות לאדמין בצורה ויזואלית על גבי המפה האינטראקטיבית.

בנוסף, תרשים המתאר את יכולות האורח במערכת:



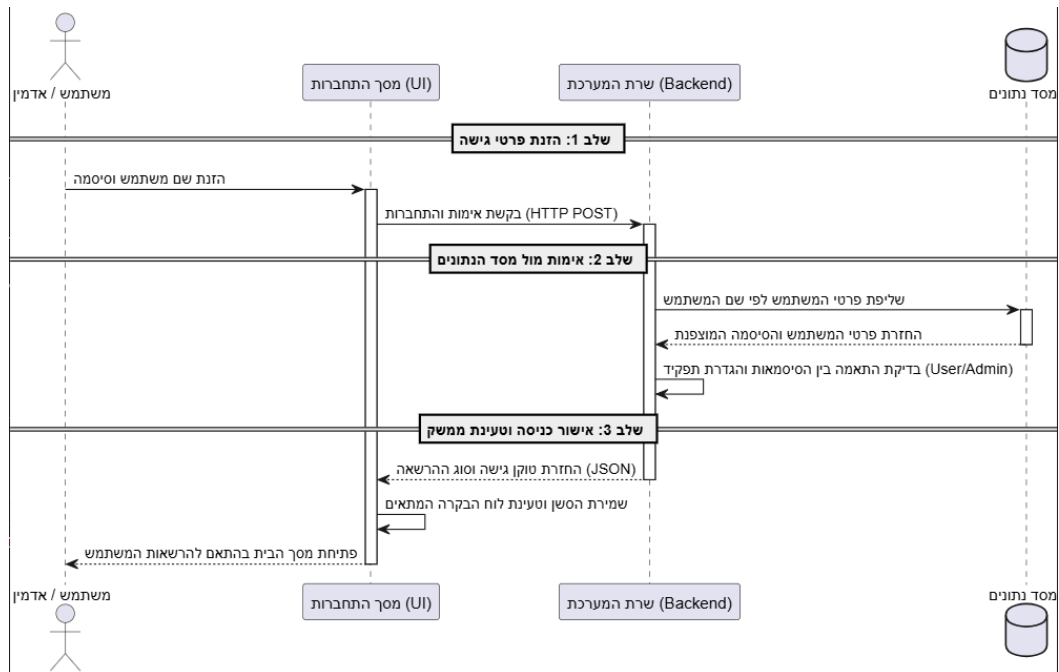
תרשים זה מתאר את הפעולה היחידה הזמינה למשתמשים אשר נכנסים למערכת ללא ביצוע התחברות או רישום מוקדם. להלן הסבר מקרי השימוש עבור האורח:

הרשמה למערכת :

זהו מקרה השימוש היחיד הזמין עבור אורח. המשתמש מזין פרטים בסיסיים כמו שם משתמש וסיסמה, ובמידת הצורך גם פרטים נוספים דרך ממשק ה-Frontend-הנתונים נשלחים ל-Backend-שם מתבצעות בדיקות תקינות כגון בדיקת ייחודיות שם המשתמש ואימות מבנה הסיסמה. לאחר מכן המידע נשמר במסד הנתונים תחת אוסף המשתמשים. בסיום התהליך, האורח נרשם למערכת ומוגדר כמשתמש רגיל בעל הרשאות בסיסיות.

10.2 תרשימי רצף Sequence Diagram

להלן דיאגרמת רצף המתארת את תהליך ההתחברות למערכת.

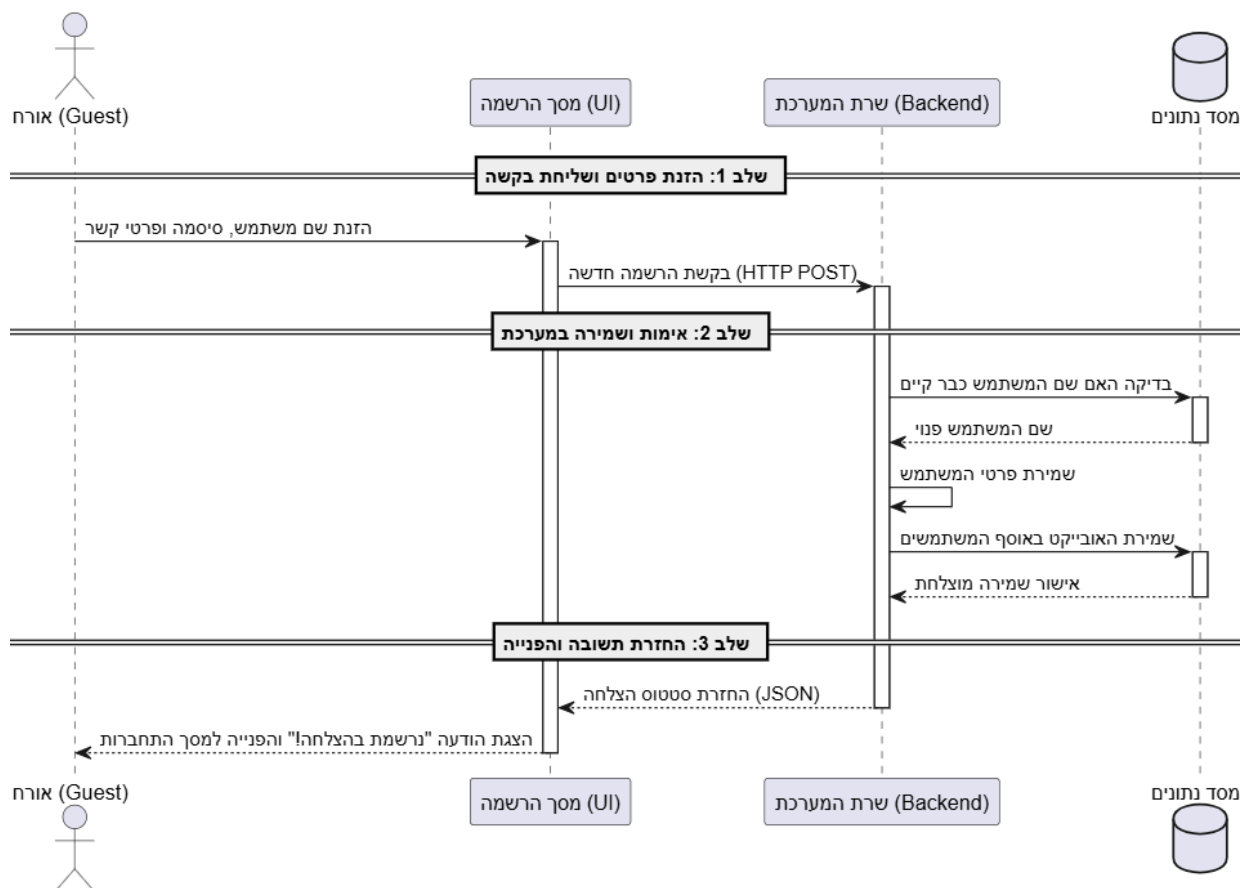


המשתמש מזין את שם המשתמש והסיסמה במסך ההתחברות, והמערכת בודקת שהשדות אינם ריקים לפני שליחת הבקשה לשרת.

השרת מחפש את המשתמש בבסיס הנתונים ומשווה בצורה מאובטחת בין הסיסמה שהוזנה לבין הסיסמה המוצפנת השמורה במערכת. אם הפרטים שגויים, המשתמש מקבל הודעת שגיאה כללית ללא חשיפת מידע רגיש.

אם ההתחברות הצליחה, השרת בודק את סוג המשתמש והרשאותיו במערכת. לאחר מכן נוצר מפתח גישה זמני ונשלחת תשובה לממשק המשתמש. בהתאם לרמת ההרשאה, המערכת פותחת למשתמש את המסכים והאפשרויות המתאימים לו, כמו מסכי ניהול לאדמין או מסכי הזמנות למשתמש רגיל.

להלן דיאגרמת רצף נוספת המתארת את תהליך ההרשמה למערכת:

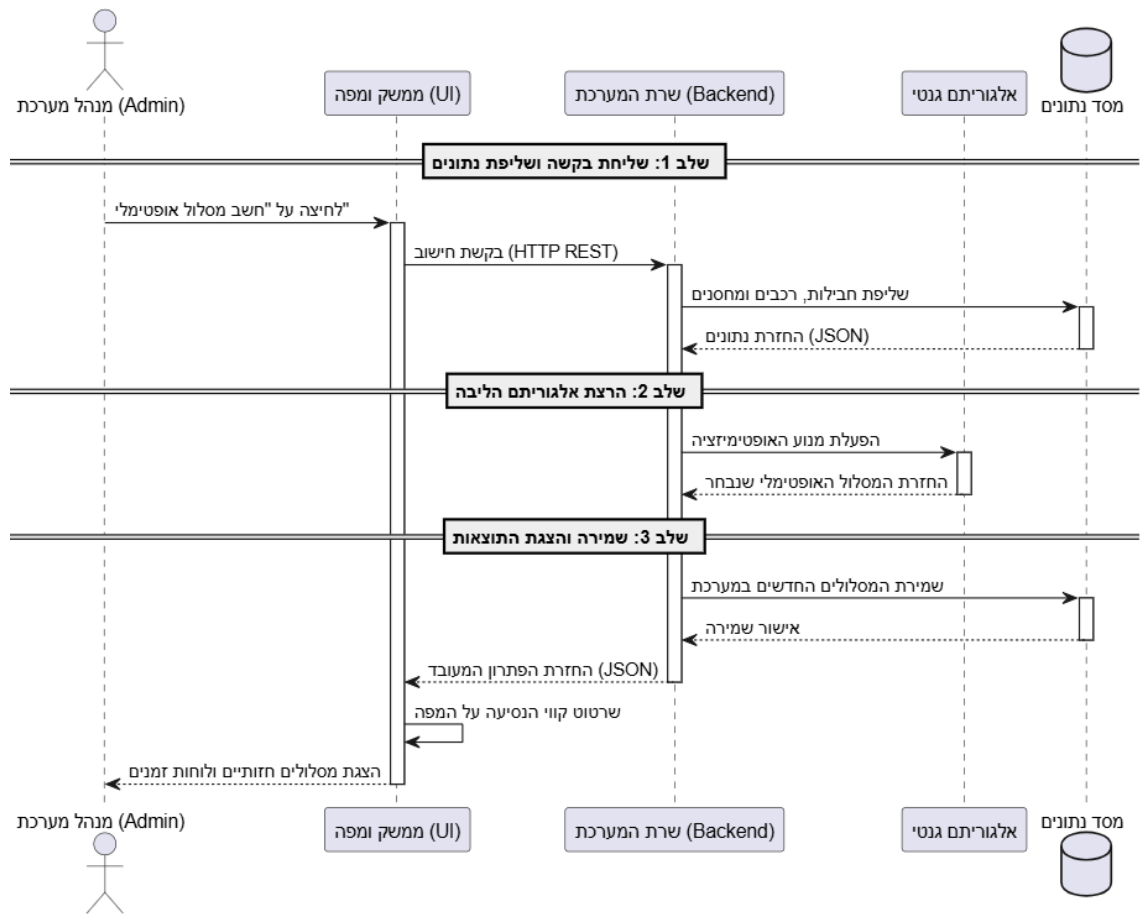


בתחילת התהליך האורח ממלא טופס הרשמה עם שם משתמש, סיסמה ופרטי קשר. המערכת מבצעת בדיקות תקינות כדי לוודא שכל השדות מולאו, שהמידע בפורמט נכון ושהסיסמה עומדת בדרישות האבטחה.

לאחר שליחת הטופס, השרת בודק בבסיס הנתונים האם כבר קיים משתמש עם אותו שם משתמש. אם השם תפוס, המשתמש מקבל הודעה שעליו לבחור שם אחר.

אם כל הנתונים תקינים, הסיסמה עוברת תהליך הצפנה לפני השמירה בבסיס הנתונים, כדי להגן על פרטיות המשתמש. לאחר מכן המשתמש החדש נשמר במערכת ומועבר למסך ההתחברות.

להלן דיאגרמת רצף המתארת את תהליך האלגוריתם המרכזי:

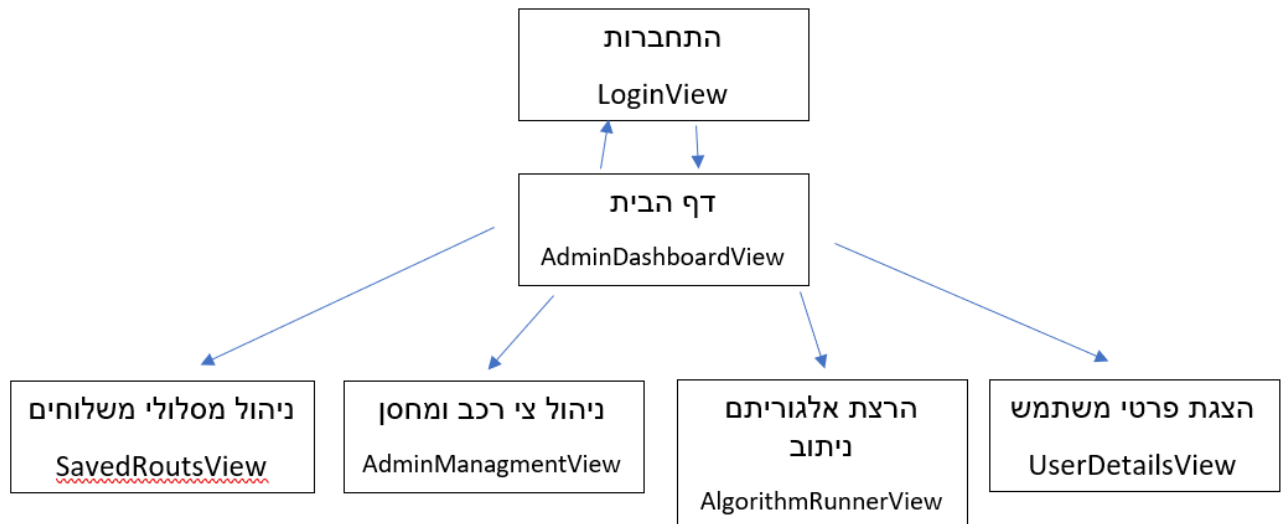


התהליך מתחיל כאשר האדמין לוחץ על כפתור חישוב המסלולים. המערכת חוסמת לחיצות נוספות בזמן החישוב כדי למנוע הפעלה כפולה של האלגוריתם. לאחר מכן השרת שולף מבסיס הנתונים את כל המידע הדרוש: הזמנות פעילות, רכבים זמינים ומיקומי ההזמנות. כדי לבצע חישוב מציאותי, השרת פונה לשירות המפות ומקבל זמני נסיעה ומרחקים אמיתיים לפי הכבישים. הנתונים האלו מוזנים יחד עם אילוצי המערכת לתוך האלגוריתם הגנטי. האלגוריתם בודק אלפי אפשרויות שונות של חלוקת משלוחים ומסלולים, תוך שמירה על מגבלות כמו קיבולת רכבים, זמן נסיעה מינימלי ככל האפשר ואיזון עומסים בין הנהגים. לאחר שנמצא הפתרון היעיל ביותר, הנתונים נשמרים בבסיס הנתונים ונשלחים חזרה לממשק הניהול. לבסוף, המערכת מציגה לאדמין את המסלולים על גבי מפה אינטראקטיבית בצורה ברורה, יחד עם סדר התחנות ולוחות הזמנים של כל נהג.

11. תיאור מסכים וממשק משתמש

11.1 תרשים היררכיית המסכים

תיאור המסכים עבור משתמש אדמין:



- **מסך הגישה וההרשמה - LoginView**

זהו מסך הכניסה הראשי של המערכת, שבו משתמשים יכולים להתחבר או ליצור חשבון חדש. המסך כולל טפסי התחברות והרשמה עם בדיקות תקינות בסיסיות כדי לוודא שהנתונים שהוזנו נכונים. לאחר התחברות מוצלחת, המשתמש מועבר אוטומטית למסך הבית המתאים להרשאות שלו. מסך זה פתוח לכולם: אורחים, משתמשים רגילים ומנהלי מערכת.

- **לוח הבקרה הראשי - AdminDashboardView**

זהו מסך הבית של מנהל המערכת ומשמש כמרכז הניהול הראשי של האפליקציה. ממנו ניתן להגיע לשאר מסכי הניהול במערכת ולקבל תמונת מצב כללית על הפעילות. המסך מחובר למסכי ניהול המשתמשים והרכבים ומאפשר מעבר נוח ביניהם. הגישה למסך זה מותרת רק למנהל מערכת, וכל משתמש אחר שינסה להיכנס אליו ייחסם ויוחזר למסך ההתחברות.

- **מסך בקרת הלקוחות - UserDetailsView**

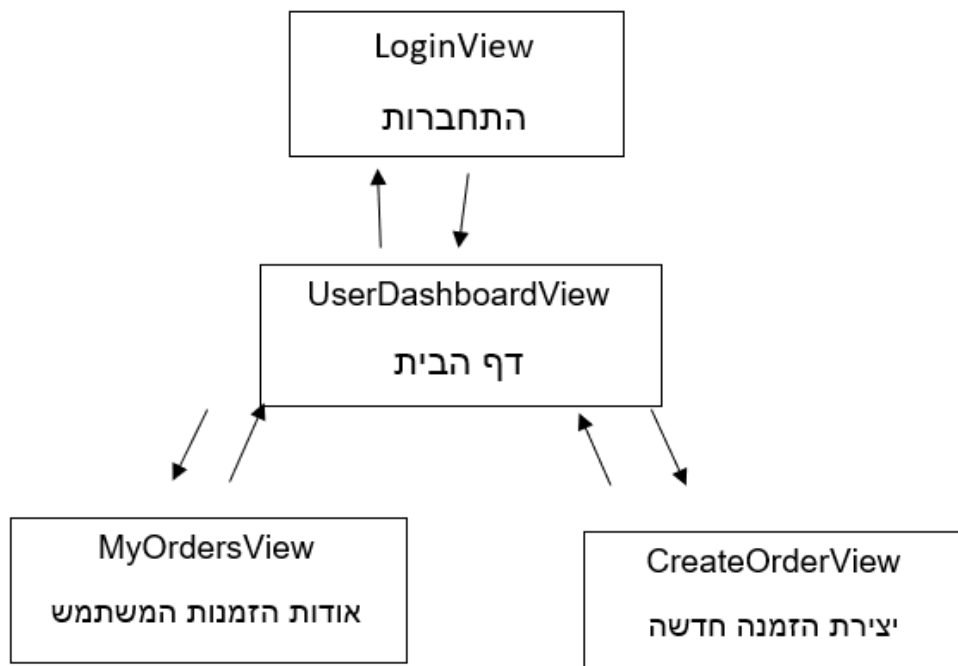
מסך זה מציג למנהל המערכת מידע על המשתמשים, הלקוחות והמשלוחים הקיימים במערכת. הנתונים מוצגים בצורה מסודרת בטבלאות ורשימות וכוללים פרטים כמו כתובות, משקלים, נפחים וחלונות זמן למשלוחים. המסך מחובר גם לדף הבית וגם למסך הרצת האלגוריתם, כדי לאפשר מעבר מהיר בין צפייה בנתונים לבין חישוב המסלולים. הגישה למסך זה מוגבלת למנהל המערכת בלבד.

- **מסך ניהול משאבי התובלה - AdminManagmentView**

זהו מסך הניהול של צי הרכבים. כאן האדמין יכול להוסיף רכבים, לעדכן קיבולת, לשנות זמינות רכבים ולשייך אותם למחסנים שונים. המסך מחובר לדף הבית וגם למסך הרצת האלגוריתם, משום שניהול הרכבים הוא חלק חשוב לפני יצירת המסלולים. גם מסך זה זמין רק למנהל המערכת.

- **מסך הליבה והאלגוריתם הראשי - AlgorithmRunnerView**

זהו המסך המרכזי של המערכת, שבו מופעל האלגוריתם הגנטי לחישוב המסלולים. המסך כולל מפה אינטראקטיבית שמציגה את מסלולי הנסיעה, סדר התחנות ולוחות הזמנים של הנהגים בצורה ויזואלית וברורה. המסך מחובר למסכי ניהול המשתמשים והרכבים כדי לאפשר לאדמין לבצע שינויים ולעדכן נתונים בזמן אמת לפני או אחרי הרצת האלגוריתם. הגישה למסך זה מותרת רק למנהל המערכת, מכיוון שהוא אחראי על ניהול מערך ההפצה כולו.

תיאור המסכים עבור משתמש קצה:

תרשים זה מתאר את זרימת המסכים והניווט עבור המשתמש הרגיל במערכת. הממשק בנוי בצורה פשוטה וממוקדת, כך שהמשתמש יכול לבצע רק את הפעולות שקשורות למשלוחים האישיים שלו, בלי גישה לאזורי הניהול או למנוע החישוב של המערכת.

- מסך הגישה וההרשמה - LoginView**

זהו מסך הכניסה הראשי של המערכת. המשתמש מזין שם משתמש וסיסמה, והמערכת בודקת את הנתונים מול בסיס הנתונים. אם ההתחברות מצליחה והמשתמש מוגדר כמשתמש רגיל, הוא מועבר אוטומטית לדף הבית האישי שלו. מסך זה פתוח לכל סוגי המשתמשים – אורחים, משתמשים רגילים ומנהלי מערכת.

- מרכז פעילות המשתמש - UserDashboardView**

זהו דף הבית של המשתמש הרגיל ומשמש כמרכז הניווט הראשי שלו במערכת. מתוך מסך זה ניתן לעבור למסך יצירת הזמנה חדשה או למסך מעקב ההזמנות האישיות. המעבר בין המסכים מתבצע בצורה דו־כיוונית, כך שהמשתמש יכול לחזור בקלות לדף הבית בכל רגע. מסך זה זמין למשתמש רגיל וגם למנהל המערכת.

- מסך מעקב ההזמנות - MyOrdersView**

מסך זה מציג למשתמש את כל ההזמנות שלו בצורה מסודרת וברורה. המשתמש יכול לראות את סטטוס המשלוחים בזמן אמת, לדוגמה אם ההזמנה בטיפול, שובצה למסלול או כבר סופקה. הגישה למידע מוגבלת להזמנות של אותו משתמש בלבד, בעוד שלמנהל המערכת קיימת גישה פיקוח מלאה.

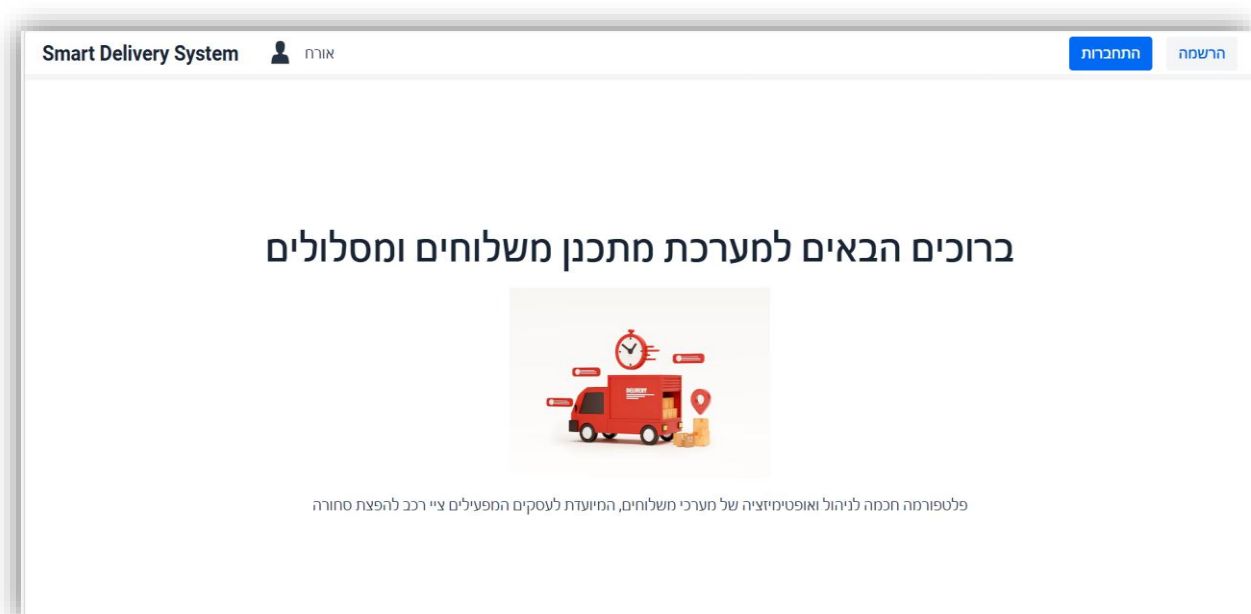
• מסך יצירת משלוח חדש - CreateOrderView

זהו המסך שבו המשתמש יוצר הזמנה חדשה למשלוח. המשתמש מזין כתובת יעד, משקל ונפח החבילה וחלון הזמן המבוקש למשלוח. במהלך מילוי הטופס המערכת מבצעת בדיקות תקינות כדי למנוע טעויות בהזנת הנתונים. לאחר שההזמנה נשמרת בהצלחה, המשתמש חוזר לדף הבית או למסך המתאים להמשך העבודה. מסך זה זמין למשתמש רגיל וגם למנהל המערכת.

11.2 תיאור המסכים

תיאור המסכים עבור משתמש קצה

דף LoginView



המחלקה משמשת כדף הנחיתה הראשי (Landing Page) של המערכת, והיא מהווה את נקודת המפגש הראשונה של המשתמש עם האפליקציה. מטרתה המרכזית היא להציג בצורה ויזואלית וברורה את ייעוד המערכת, ליצור רושם ראשוני מקצועי, ולהכווין את המשתמש אל סביבת העבודה הלוגיסטית של האפליקציה. הדף מעוצב תחת תבנית המבנה המשותפת לאורחים GuestLayout (נסביר עליו בהמשך)

פירוט רכיבי הדף:

- **VerticalLayout**: זהו הרכיב הראשי שמחזיק את כל האלמנטים בדף ומסדר אותם בצורה אנכית. הוא גם דואג שכל התוכן יהיה ממורכז באמצע המסך גם אופקית וגם אנכית.
- **H1**: רכיב טקסט גדול שמציג את שם המערכת. הוא משמש לזיהוי מהיר של האפליקציה.
- **Span**: בתחתית הדף מוצגת פסקת הסבר קצרה באמצעות רכיב Span. טקסט זה מתאר את ייעוד המערכת כפלטפורמה חכמה לניהול ואופטימיזציה של משלוחים עבור עסקים וצי רכב, ובכך מספק למשתמש הקשר פונקציונלי ברור עוד לפני תחילת העבודה בפועל.

- **Image:** המחלקה משלבת רכיב Image המציג תמונה גרפית של רכב משלוחים. אלמנט זה מוסיף ממד חזותי ומחזק את הזהות הלוגיסטית של המערכת. בנוסף, התמונה מוגדרת ברוחב קבוע, מה שמבטיח שמירה על פרופורציות ותצוגה אחידה בממשק המשתמש.

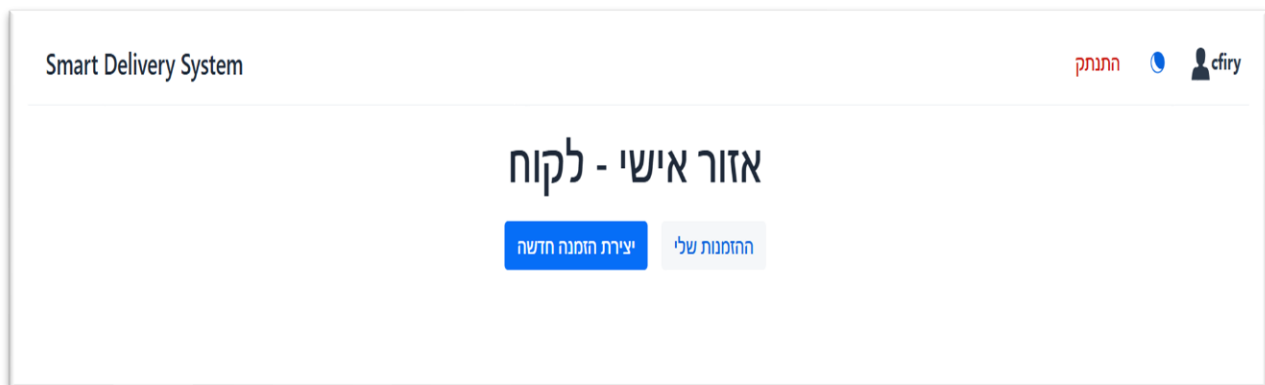
דף RegisterView

הדף RegisterView מיועד למשתמשים מסוג אורח (Guest) שמעוניינים ליצור חשבון חדש במערכת, והוא נגיש דרך הנתביב. /register/ מטרתו היא לאפשר רישום משתמש חדש בצורה מסודרת ובטוחה, תוך ביצוע בדיקות תקינות על הנתונים כבר בזמן ההזנה. המערכת מוודאת שכל השדות מלאים ושהמידע עומד בדרישות בסיסיות כמו אורך שם משתמש תקין. לאחר שהרישום מצליח, החשבון נשמר בבסיס הנתונים והמשתמש מועבר למסך ההתחברות.

פירוט רכיבי הדף

- **VerticalLayout:** זהו הרכיב הראשי של הדף שמסדר את כל האלמנטים בצורה אנכית. הוא אחראי על מבנה הדף ועל כך שכל הרכיבים יוצגו בצורה מסודרת ונוחה לקריאה.
- **H1:** כותרת עליונה שמציגה את שם המסך "RegisterView". היא משמשת לזיהוי ברור של הדף וליצירת מבנה ויזואלי מסודר.
- **HorizontalLayout:** רכיב שמסדר את שדות הקלט וכפתור ההרשמה בשורה אחת. הוא עוזר לשמור על ממשק נקי ומאורגן מבחינה עיצובית.
- **TextField:** כולל שני שדות עיקריים: שם משתמש וסיסמה. שדות אלו משמשים להזנת פרטי המשתמש החדש לפני יצירת החשבון.
- **Button:** כפתור שמפעיל את תהליך הרישום. בלחיצה עליו הנתונים נבדקים ונשלחים לשרת ליצור חשבון משתמש חדש במערכת.
- **Notification:** רכיב שמציג הודעות למשתמש, כמו הצלחה בהרשמה או שגיאות בטופס. ההודעות מופיעות לזמן קצר ונעלמות אוטומטית.

דף UserDashboardView



הדף UserDashboardView משמש כמרכז הפעילות הראשי עבור משתמשים בעלי הרשאה רגילה במערכת דף זה משויך לנתיב הניווט user-dashboard ומנוהל תחת תבנית הפריסה המשותפת MainLayout (הבר שנמצא מעל לאזור האישי, ניתן עליו הסבר בהמשך).

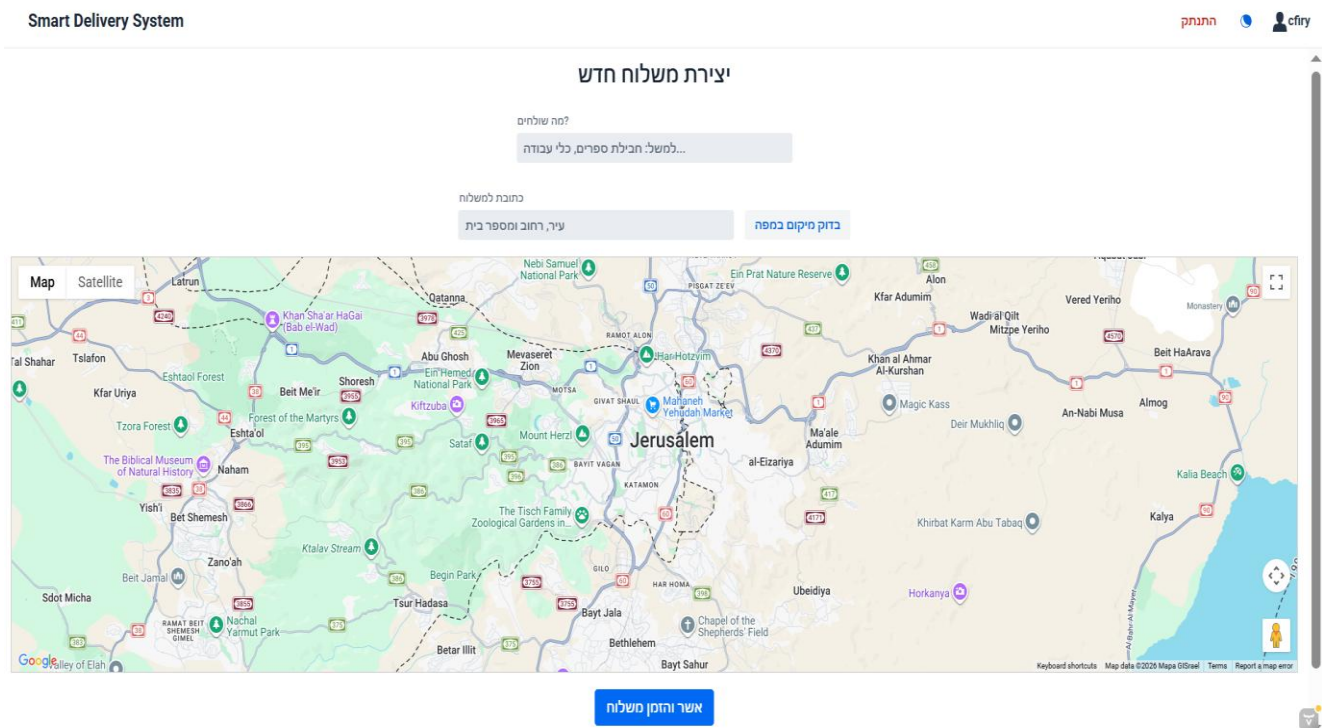
מטרתו המרכזית של דף זה היא לרכז עבור המשתמש את כל הפעולות העסקיות האפשריות שלו במקום אחד ברור ופשוט, תוך יצירת חוויית שימוש אינטואיטיבית ומהירה. דרך מסך זה המשתמש יכול לגשת לשתי הפעולות המרכזיות במערכת: יצירת הזמנה חדשה ומעקב אחר הזמנות קיימות. הדף אינו חושף מידע ניהולי או נתונים מערכתיים של האדמין, ובכך שומר על הפרדת הרשאות מלאה ומבנה אבטחה ברור בין משתמשי קצה לבין שכבת הניהול והאלגוריתמים.

המסך בנוי כנקודת מעבר מרכזית בתוך הזרימה של המערכת, ולכן הוא מהווה צומת ניווט שממנו המשתמש ממשיך לכל אחת מהפעולות העיקריות שלו בצורה פשוטה וללא עומס מידע.

פירוט רכיבי הדף

- **VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.
- **H1:** כותרת הדף מוגדרת כאלמנט טקסט מרכזי המייצג את שם המסך. תפקידה הוא להעניק למשתמש התמצאות מיידית ולחזק את ההבנה שהוא נמצא באזור האישי שלו בתוך המערכת. הכותרת גם יוצרת היררכיה ויזואלית ברורה בין שם המסך לבין שאר רכיבי הממשק.
- **Button:** במסך קיימים שני כפתורי פעולה מרכזיים, כפתור יצירת הזמנה המוביל למסך יצירת הזמנה חדשה. הוא מסומן כפעולה המרכזית בדף ולכן מעוצב בצורה בולטת יותר, ומאפשר למשתמש להתחיל תהליך הזנה של משלוח חדש בצורה מהירה וישירה. וכפתור ההזמנות שלי המוביל למסך הצגת ההזמנות הקיימות של המשתמש. דרך מסך זה המשתמש יכול לעקוב אחר סטטוס המשלוחים שלו ולצפות בהיסטוריית הפעילות שלו במערכת.

דף CreateOrderView



הדף CreateOrderView מהווה את ממשק קליטת הנתונים העסקי המרכזי של הלקוח במערכת, נתיב הדף במערכת הוא create-order.

מטרתו העיקרית של דף זה היא לאפשר למשתמש להזין משלוח חדש בצורה מסודרת ומאובטחת, תוך שילוב מנגנון אימות נתונים בזמן אמת ואינטגרציה עם רכיב מפה אינטראקטיבי. במהלך העבודה עם המסך המשתמש מזין את פרטי המשלוח והכתובת, והמערכת מבצעת בדיקה מול שירות המפות החיצוני (Geocoding) כדי לתרגם את הכתובת לנקודת ציון גיאוגרפית אמיתית. רק לאחר אימות מלא של הנתונים ווידוא שהמשתמש מחובר דרך Session תקין, ההזמנה נשמרת במסד הנתונים ונכנסת לסטטוס המתנה לעיבוד על ידי מנוע האופטימיזציה של המערכת.

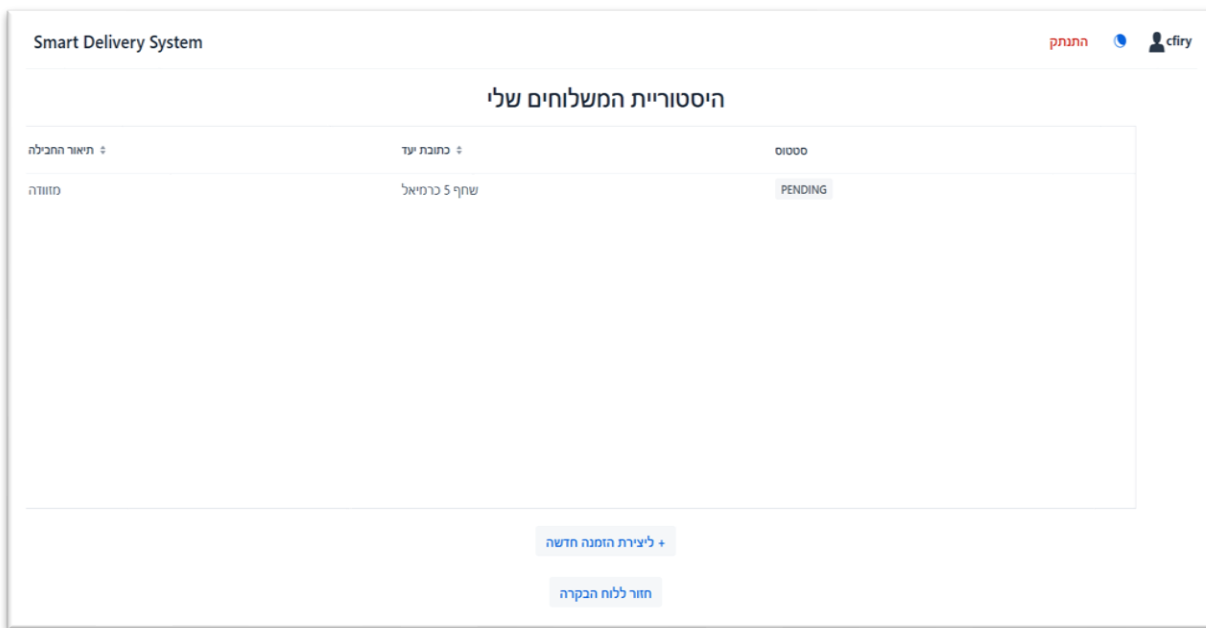
המסך משלב בין הזנה טקסטואלית לבין משוב ויזואלי, ובכך מאפשר למשתמש להבין באופן מידי האם הכתובת שהוזנה תקינה והיכן היא ממוקמת בפועל.

פירוט רכיבי הדף

- VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.

- **H2:** כותרת המסך מוגדרת כאלמנט טקסט מרכזי בשם "יצירת משלוח חדש". תפקידה הוא להציג למשתמש את מטרת הדף בצורה ברורה ולשמור על היררכיית עיצוב תקינה מול שאר רכיבי הממשק.
- **Button:** במסך קיימים שלושה כפתורי פעולה מרכזיים: כפתור בדיקת מיקום שמבצע בדיקה של הכתובת שהוזנה מול שירות המפות ומעדכן את המפה בהתאם לתוצאה, כפתור שמירה השומר את ההזמנה במערכת לאחר ביצוע כל בדיקות האימות הנדרשות, כולל בדיקת Session ואימות כתובת, וכפתור חזרה שמאפשר למשתמש לחזור למסך הקודם מבלי לשמור את הנתונים.
- **TextField:** במסך קיימים שני שדות קלט מרכזיים להזנת פרטי המשלוח: שדה תיאור החבילה המאפשר למשתמש להזין מהותית מה נשלח, ומשמש לזיהוי ההזמנה במערכת. ושדה כתובת היעד להזנת כתובת המשלוח. נתון זה נשלח בהמשך לאימות מול שירות המפות לצורך קבלת קואורדינטות מדויקות.
- **Notification:** רכיב שמציג הודעות למשתמש, כמו הצלחה בהרשמה או שגיאות בטופס. ההודעות מופיעות לזמן קצר ונעלמות אוטומטית.
- **GoogleMapComponent:** רכיב זה מציג מפה אינטראקטיבית מבוססת Google Maps API. תפקידו הוא לספק למשתמש משוב חזותי על הכתובת שהוזנה באמצעות מיקום נקודה מדויקת על גבי המפה, ובכך לאפשר לו לוודא שהיעד שנבחר אכן נכון לפני שמירת ההזמנה.
- **HorizontalLayout:** רכיב שמסדר את שדות הקלט וכפתור ההרשמה בשורה אחת. הוא עוזר לשמור על ממשק נקי ומאורגן מבחינה עיצובית.

דף MyOrdersView



הדף MyOrdersView משמש כמסך המעקב, הבקרה וניהול המידע האישי של הלקוח במערכת הוא משויך לנתיב הניווט my-orders.

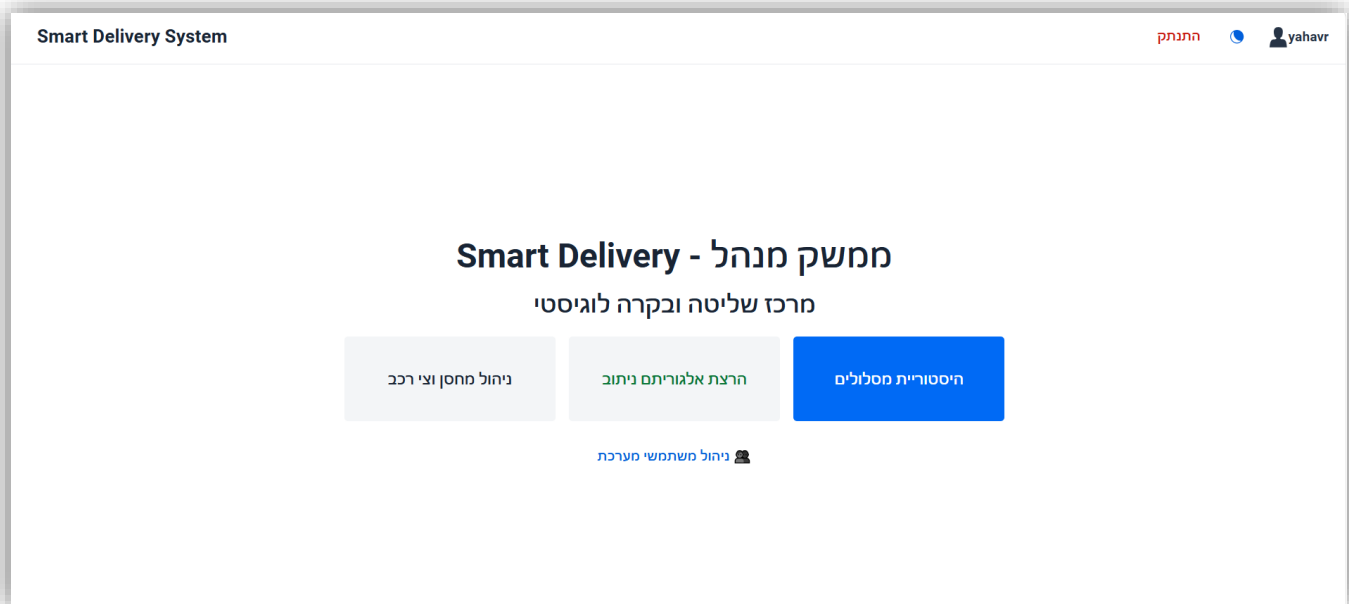
מטרתו המרכזית של דף זה היא לשלוח בצורה מאובטחת ולהציג באופן מרוכז וטבלאי את כל ההזמנות שנוצרו על ידי המשתמש הנוכחי בלבד. עם טעינת הדף מתבצעת בדיקת אבטחה ברמת ה-VaadinSession כדי לוודא שזהות המשתמש מאומתת, ולאחר מכן מתבצעת פנייה לשירות הליבה לצורך שליפת הנתונים המסוננים ממסד הנתונים. בנוסף להצגת הנתונים, המסך מספק למשתמש תמונת מצב ברורה של התקדמות כל משלוח בזמן אמת, כולל סטטוסים כמו המתנה, שיבוץ למסלול או אספקה ללקוח, ובכך מאפשר מעקב פשוט ונגיש אחר כלל הפעילות הלוגיסטית האישית שלו.

פירוט רכיבי הדף:

- **VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.
- **H2:** כותרת המסך מוגדרת כאלמנט טקסט מרכזי בשם "היסטוריית המשלוחים שלי". תפקידה הוא להציג למשתמש את מטרת הדף בצורה ברורה ולשמור על היררכיית עיצוב תקינה מול שאר רכיבי הממשק.
- **GRID:** רכיב זה מהווה את מרכז התצוגה של הדף, ומציג את כלל ההזמנות של המשתמש בצורה טבלאית מסודרת. ה-Grid מוגדר עבור אובייקטי Order וכולל עמודות המציגות את תיאור ההזמנה, כתובת היעד וסטטוס המשלוח. בנוסף, חלק מהעמודות תומכות במיון, מה שמאפשר למשתמש לסדר את הנתונים בצורה נוחה לפי הצורך.

- **Span:** רכיב זה משמש להצגת סטטוס ההזמנה בתוך הטבלה בצורה חזותית. הוא מוצג כתגית המעוצבת דינמית בהתאם למצב ההזמנה, כך שכל סטטוס מקבל ייצוג צבעוני שונה (למשל המתנה, בדרך או סופק). בצורה זו המשתמש יכול להבין במהירות את מצב כל משלוח ללא צורך בקריאת טקסט ארוך.
- **Button:** במסך קיימים שני כפתורי פעולה מרכזיים: כפתור יצירת הזמנה חדשה- כפתור ניווט המוביל את המשתמש ישירות למסך יצירת הזמנה חדשה, וכפתור חזרה המאפשר למשתמש לחזור למסך הראשי של האזור האישי שלו (User Dashboard) בצורה פשוטה ומאובטחת.
- **Notification:** רכיב שמציג הודעות למשתמש, כמו הצלחה בהרשמה או שגיאות בטופס. ההודעות מופיעות לזמן קצר ונעלמות אוטומטית.

תיאור המסכים עבור משתמש אדמין (לאחר LoginView – כיוון וצוין בתיאור המסכים למשתמש)



מסך AdminDashboardView

הדף AdminDashboardView מהווה את מרכז השליטה, הניהול הלוגיסטי הראשי ודף הבית עבור משתמשים בעלי הרשאת ניהול משויך לנתיב הניווט admin-dashboard ומנוהל תחת תבנית המבנה המשותפת MainLayout.

מטרתו המרכזית של דף זה היא לרכז את כלל פעולות הניהול של הארגון במסך אחד מאובטח, ברור ונוח לתפעול. מתוך ממשק זה האדמין יכול לגשת לניהול צי הרכב והתשתיות, לצפות בנתוני המשתמשים וההזמנות, ולהפעיל את מנוע האופטימיזציה של המערכת. המסך משמש כצומת הניווט המרכזי של סביבת הניהול, ולכן הוא בנוי בצורה רחבה וממוקדת שמאפשרת גישה מהירה לפעולות הקריטיות ביותר במערכת.

בנוסף, הדף מיישם הפרדה מלאה בין ממשק הניהול לבין ממשק המשתמש הרגיל, ובכך שומר על אבטחת המידע ועל שלמות הפעולות הלוגיסטיות והחשוביות של הארגון.

פירוט רכיבי הדף

- **VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.

- **H2:** כותרת משנית המוצגת מתחת לכותרת הראשית עם הטקסט "מרכז שליטה ובקרה לוגיסטי". רכיב זה מוסיף תיאור קצר לגבי מטרת המסך ותורם להיררכיה הוויזואלית ולחלוקת התוכן בצורה מסודרת וברורה.
- **H1:** כותרת הדף מוגדרת כאלמנט טקסט מרכזי בשם: "מנהל ממשק - Smart Delivery" תפקידה הוא להציג באופן ברור את סביבת הניהול של המערכת וליצור נקודת עוגן חזותית ראשית עבור המשתמש המנהל.
- **Button:** במסך קיימים שלושה כפתורי פעולה מרכזיים: כפתור ניהול תשתית שמוביל את המנהל למסך ניהול המחסנים וכלי הרכב של הארגון, כפתור הרצת האלגוריתם, המוביל למסך הרצת מנוע האופטימיזציה והמפה האינטראקטיבית, וכפתור ניהול משתמשים שמוביל למסך הצגת המשתמשים ונתוני המערכת.
- **HorizontalLayout:** רכיב זה מאגד את כפתורי הפעולה הראשיים של המערכת ומציג אותם זה לצד זה בשורה אחת. תפקידו הוא ליצור אזור ניווט מרכזי ונוח לשימוש, תוך שמירה על מרווחים אחידים ומראה נקי ומאוזן בין כפתורי הפעולה הגדולים.

דף ניהול כלי רכב AdminManagementView

Smart Delivery System

ניהול תשתית לוגיסטית - Smart Delivery

הגדרת מחסן מרכזי

נחבית מכלה

הורב פתק 18 ירושלים

בדיקת סיוקום בחסן

שמור/עזבן מחסן

Map Satellite

ניהול צי רכב

מספר רישוי

קיבולת (קט)

הוסף רכב

License Plate	Max Capacity	סטטוס
20987654	5.0	מחנ
39485735	3.0	מחנ
29840373	3.0	מחנ
39467382	4.0	מחנ

הדף AdminManagementView משמש כמרכז ניהול התשתית הלוגיסטית של מנהל המערכת במערכת Smart Delivery. מתוך מסך זה האדמין מגדיר את המחסן המרכזי שממנו יוצאים המשלוחים ומנהל את צי הרכבים של החברה, כולל פרטי הרכב וקיבולת ההעמסה שלו.

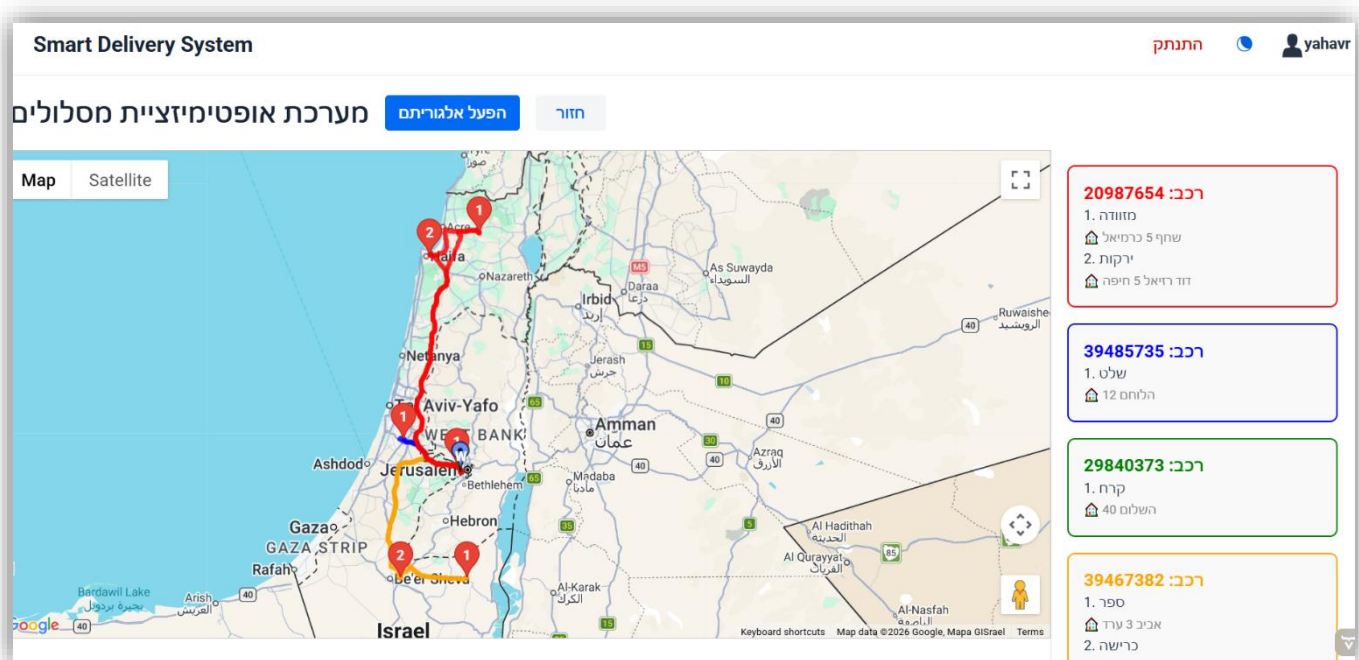
בנוסף, הדף משלב מנגנון אימות כתובות באמצעות מפה אינטראקטיבית וטבלת ניהול דינמית, ובכך מאפשר שליטה מלאה במשאבים הפיזיים שעליהם מבוסס מנוע הניתוב והאופטימיזציה של המערכת.

פירוט רכיבי הדף:

- VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.
- H2\H3:** רכיבי הכותרת משמשים להגדרת מבנה היררכי ברור בדף. כותרת ה-H2 מציגה את נושא המסך הראשי, ואילו כותרות ה-H3 מחלקות את הדף לאזורי עבודה נפרדים כמו ניהול המחסן וניהול צי הרכב.
- Button:** במסך קיימים מספר רכיבי Button המבצעים פעולות שונות במערכת. ישנו כפתור המאפשר לבצע בדיקה גיאוגרפית של כתובת המחסן ולהציג את המיקום על גבי המפה מבלי לשמור אותו במסד הנתונים. עוד כפתור השומר או מעדכן את מיקום המחסן במערכת, כפתור המשמש להוספת רכב חדש לצי, כולל בדיקת כפילויות של מספר רישוי ואיפוס שדות הקלט לאחר ההוספה. ובנוסף, לכל שורת רכב בטבלה מוזרק כפתור מחיקה קטן ואדום המאפשר להסיר רכבים מהמערכת בצורה ישירה.

- **HorizontalLayout**: רכיב זה מאגד את כפתורי הפעולה הראשיים של המערכת ומציג אותם זה לצד זה בשורה אחת. תפקידו הוא ליצור אזור ניווט מרכזי ונוח לשימוש, תוך שמירה על מרווחים אחידים ומראה נקי ומאוזן בין כפתורי הפעולה הגדולים.
- **Notification**: מספק למשתמש משוב מיידי על פעולות שבוצעו במערכת. הודעות הצלחה מוצגות כאשר פעולה מסתיימת בצורה תקינה, כמו שמירת מחסן או הוספת רכב, בעוד שהודעות שגיאה מוצגות במקרים של כשל, לדוגמה כאשר הכתובת אינה תקינה או כאשר מוזן מספר רישוי שכבר קיים במערכת.
- **TextField**: שדות אלו מאפשרים למנהל להזין נתונים טקסטואליים כמו כתובת מלאה ומספר רישוי של רכב. הם משמשים כחלק מטופסי ההזנה והעדכון של נתוני המערכת.
- **NumberField**: רכיב זה משמש להזנת קיבולת ההעמסה של הרכב בקילוגרמים. השימוש בשדה מספרי מבטיח שהמערכת תקבל ערכים תקינים בלבד ללא טקסט לא חוקי.
- **GoogleMapComponent**: רכיב זה מטמיע מפה אינטראקטיבית מבוססת Google Maps ומציג בצורה חזותית את מיקום המחסן שנבחר. לאחר אימות הכתובת, המפה מתעדכנת ומציגה סמן במיקום המדויק.
- **Grid**: רכיב הטבלה מציג את צי הרכבים של החברה בצורה מרוכזת ומאפשר צפייה, עדכון ומחיקה של רכבים קיימים. הטבלה כוללת מידע כמו מספר רישוי, קיבולת וסטטוס פעילות.

דף הרצת האלגוריתם הראשי AlgorithmRunnerView



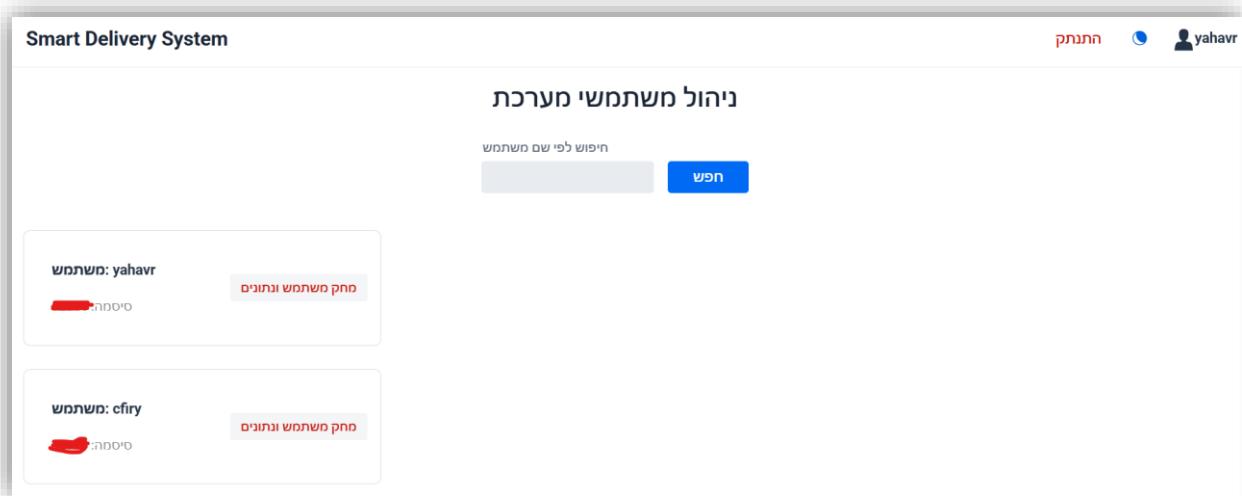
הדף AlgorithmRunnerView מהווה את מרכז הבקרה החישובי והוויזואלי של מערכת Smart Delivery. מתוך מסך זה מנהל המערכת מפעיל את מנוע האופטימיזציה הלוגיסטי, אשר מחשב מסלולי נסיעה יעילים עבור צי הרכבים בהתאם להזמנות הפעילות, מרחקי הדרך ומגבלות הקיבולת. לאחר סיום החישוב, המערכת מציגה את תוצאות האלגוריתם בזמן אמת על גבי מפה אינטראקטיבית ובמקביל בונה פאנל מידע מפורט המציג את סדר התחנות והמשלוחים שהוקצו לכל רכב. הדף מרכז את תהליך התכנון, הניתוח וההמחשה של מערך ההפצה כולו במקום אחד.

פירוט רכיבי הדף

- **VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.
- **HorizontalLayout:** רכיב זה מאגד את כפתורי הפעולה הראשיים של המערכת ומציג אותם זה לצד זה בשורה אחת. תפקידו הוא ליצור אזור ניווט מרכזי ונוח לשימוש, תוך שמירה על מרווחים אחידים ומראה נקי ומאוזן בין כפתורי הפעולה הגדולים.
- **H2\H3:** רכיבי הכותרת משמשים להגדרת מבנה היררכי ברור בדף. כותרת ה-H2 מציגה את נושא המסך הראשי, ואילו כותרות ה-H3 מחלקות את הדף לאזורי עבודה נפרדים כמו ניהול המחסן וניהול צי הרכב.

- **Notification:** רכיב זה מציג למנהל הודעות אזהרה ושגיאה במקרה שחסרים נתונים הדרושים להרצת האלגוריתם, כמו רכבים, מחסן או הזמנות פעילות.
- **Button:** בדף קיימים כפתורים להפעלת האלגוריתם ולחזרה ללוח הבקרה הראשי. כפתור ההפעלה מפעיל את תהליך החישוב והאופטימיזציה, ואילו כפתור החזרה מאפשר ניווט בטוח חזרה למסך הניהול.
- **GoogleMapComponent:** רכיב זה מטמיע מפה אינטראקטיבית מבוססת Google Maps ומציג בצורה חזותית את מסלולי הנסיעה של הרכבים. המפה כוללת סמנים לתחנות, סימון המחסן המרכזי וקווי מסלול צבעוניים עבור כל רכב.
- **Span:** רכיבי Span משמשים להצגת מידע קצר ומעוצב בתוך כרטיסיות הרכב, כגון תיאור החבילה, מספר התחנה וכתובת היעד.
- **Scroller:** רכיב זה מאפשר גלילה אנכית של פאנל המידע הצדדי במקרה שמספר הרכבים או התחנות גדול. כך ניתן לשמור על תצוגה מסודרת מבלי לפגוע באזור המפה המרכזי.

דף ניהול המשתמשים UserDetailsView



הדף UserDetailsView משמש כמסך ניהול ופיקוח על כלל המשתמשים הרשומים במערכת עבור מנהל המערכת בלבד. מתוך מסך זה האדמין יכול לחפש משתמשים, לצפות בפרטי החשבון שלהם ולבצע פעולות ניהוליות על נתוני המשתמשים וההזמנות המשויות אליהם. בנוסף, הדף כולל מנגנון מחיקה משורשת, כך שמחיקת משתמש מסירה באופן אוטומטי גם את כלל ההזמנות והמידע המשויך אליו ממסד הנתונים, ובכך שומרת על תקינות וסדר במאגר המידע הארגוני.

פירוט רכיבי הדף

- VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.
- HorizontalLayout:** רכיב זה מאגד את כפתורי הפעולה הראשיים של המערכת ומציג אותם זה לצד זה בשורה אחת. תפקידו הוא ליצור אזור ניווט מרכזי ונוח לשימוש, תוך שמירה על מרווחים אחידים ומראה נקי ומאוזן בין כפתורי הפעולה הגדולים.
- H2:** רכיב הכותרת מציג את שם המסך בראש העמוד ומסייע להגדיר בצורה ברורה את אזור ניהול המשתמשים במערכת.
- Span:** רכיבי Span מציגים את פרטי המשתמש בתוך הכרטיסיות, כגון שם המשתמש והסיסמה, תוך שימוש בעיצוב שונה ליצירת היררכיה חזותית ברורה.
- Notification:** רכיב זה מציג למנהל הודעות אזהרה ושגיאה במקרה שחסרים נתונים הדרושים להרצת האלגוריתם, כמו רכבים, מחסן או הזמנות פעילות.

- **Button:** בדף קיימים כפתורים לביצוע חיפוש ולמחיקת משתמשים. כפתור המחיקה מפעיל תהליך מחיקה מלא של המשתמש ושל כלל הנתונים המשויכים אליו במערכת.
- **TextField:** שדה זה מאפשר למנהל להזין שם משתמש או חלק ממנו לצורך חיפוש וסינון דינמי של רשימת המשתמשים הקיימים במערכת.

דף ניהול המסלולים SavedRoutesView

Smart Delivery System			
			התנתק yahavr
היסטוריית מסלולים שמורים			
רכב	מרחק	כמות הזמנות	פעולות
20987654	ק"מ 302.09	2	מחק
39485735	ק"מ 272.66	1	מחק
29840373	ק"מ 90.37	1	מחק
39467382	ק"מ 501.55	2	מחק

הדף SavedRoutesView משמש כמסך ההיסטוריה של המסלולים שנבנו במערכת. הוא מאפשר למנהל לצפות בכל תוכניות ההפצה שבוצעו בעבר, לראות נתוני ביצוע מרכזיים (כמו מרחק ומספר הזמנות), ולנהל את המידע ההיסטורי באמצעות מחיקה של מסלולים שאינם רלוונטיים עוד.

פירוט רכיבי הדף:

- VerticalLayout:** מחלקת הדף יורשת ישירות מרכיב זה, המשמש כבסיס המבני של כל האלמנטים במסך. רכיב זה אחראי על סידור כל רכיבי הדף בצורה אנכית, כך שהכותרת מופיעה בראש המסך ומתחתיה אזור הכפתורים. בנוסף, מוגדר יישור מרכזי של כל הרכיבים, כך שהתוכן מוצג באופן סימטרי ואחיד במרכז הדף, מה שמייצר ממשק נקי ונוח לקריאה ולניווט.
- H2:** כותרת אחת בראש המסך: "היסטוריית מסלולים שמורים". תפקידה הוא להגדיר בצורה ברורה את מטרת המסך ולתת הקשר למידע שמוצג בו.
- Grid:** זהו הרכיב המרכזי במסך, המציג את כל המסלולים ההיסטוריים. הטבלה כוללת כמה עמודות עיקריות: מזהה הרכב שביצע את המסלול, המרחק הכולל של הנסיעה, כמה נקודות יבקר המסלול, וכפתורי ניהול לכל מסלול.
- Button:** בכל שורה בטבלה מופיע כפתור מחיקה. שמאפשר למחוק מסלול ספציפי מהמערכת.
- HorizontalLayout:** רכיב זה מאגד את כפתורי הפעולה הראשיים של המערכת ומציג אותם זה לצד זה בשורה אחת. תפקידו הוא ליצור אזור ניווט מרכזי ונוח לשימוש, תוך שמירה על מרווחים אחידים ומראה נקי ומאוזן בין כפתורי הפעולה הגדולים.
- Notification:** רכיב שמציג הודעה קצרה למשתמש אחרי פעולה. לדוגמה אישור שהמסלול נמחק בהצלחה.

הסבר על הבר MainLayout

Smart Delivery System



הרכיב MainLayout מהווה את תבנית המעטפת המרכזית של כלל מערכת ה SmartDelivery, ואינו מסך עצמאי אלא שכבת תשתית ארכיטקטונית (Master Layout) שעוטפת את כל ה- Views במערכת. כל מסך משתמש בו באמצעות הגדרת `layout = MainLayout.class` כך שכל מעבר בין דפים מתבצע בתוך אותה מסגרת ויזואלית אחידה.

מטרתו המרכזית של הרכיב היא ליצור מבנה ניווט קבוע ואחיד לכל חלקי המערכת, תוך שמירה על חוויית משתמש רציפה, מיתוג אחיד של המערכת, וניהול זהות משתמש גלובלית. בנוסף, הרכיב אחראי על הצגת הבר העליון (Header) הכולל מידע על המשתמש המחובר, אפשרות התנתקות מאובטחת, והחלפת מצב תצוגה מצב אור/חושך כאשר כל אלה מנוהלים ברמת התשתית ולא ברמת מסך בודד.

רכיבי התבנית

- **AppLayout:** משמש כשלד המבני של כל המערכת. הוא מגדיר אזור קבוע בחלק העליון של המסך שבו מוצבים רכיבי השליטה, ומתחתיו אזור תוכן דינמי שבו נטענים כל הדפים. בצורה זו, המעבר בין מסכים אינו פוגע במבנה הכללי של המערכת, אלא רק מחליף את התוכן המרכזי.
- **H3:** תיאור ומאפיינים בקוד: רכיב טקסט מסוג H3 המאותחל כ- "Smart Delivery System" ומעוצב עם ריווח אופקי.
- **Span:** מציג בזמן אמת את זהות המשתמש המחובר או "אורח" במידה ואין Session פעיל. שילוב האייקון עם הטקסט מייצר אינדיקציה ויזואלית ברורה למצב ההתחברות, ומהווה חלק ממנגנון שקיפות ואבטחת שימוש.
- **VaddinIcon:** הם אוסף אייקונים מובנה של ספריית Vaddin, שמאפשר להציג סמלים גרפיים סטנדרטיים כחלק מממשק המשתמש ללא צורך בקבצי תמונה חיצוניים. ברכיב זה האייקונים משמשים להעברת משמעות מהירה ואינטואיטיבית: אייקון המשתמש (USER) מוצג לצד שם המשתמש המחובר ומספק אינדיקציה ברורה לזהות הפעילה במערכת, אייקון הירח (MOON) משמש כסמל למצב תצוגה כהה ומופיע כברירת מחדל בכפתור שינוי התצוגה, בעוד שאייקון השמש (SUN_O) מופיע באופן דינמי כאשר המערכת נמצאת במצב כהה ומסמן חזרה למצב תצוגה בהיר. השימוש באייקונים אלו מאפשר למשתמש להבין באופן מיידי את מצב המערכת ואת פעולות השליטה האפשריות, מבלי להסתמך על טקסט בלבד.

- **Button:** משמש כרכיב אינטראקטיבי מרכזי לביצוע פעולות מערכת קריטיות. שני הכפתורים המרכזיים ב-`MainLayout` הם כפתור ההתנתקות וכפתור החלפת התצוגה. כפתור ההתנתקות תפקידו לבצע סגירה מלאה של ה-`Session` הפעיל ולהעביר את המשתמש למסך הכניסה, מה שמבטיח שמירה על אבטחת מידע ומניעת גישה לא מורשית. לעומתו, כפתור שינוי התצוגה מאפשר מעבר דינמי בין מצב בהיר לכהה של המערכת. בעת לחיצה עליו מתבצע שינוי של ערכת העיצוב הגלובלית של האפליקציה, והאייקון שבו מתעדכן בהתאם למצב הנוכחי כדי לספק משוב חזותי מיידי למשתמש.
- **HorizontalLayout:** משמש כסרגל הניווט העליון של כל המערכת. באמצעות שימוש במנגנון `expand` על רכיב הלוגו, האלמנטים מחולקים כך שהלוגו נשאר בצד אחד בעוד שכפתורי הפעולה ופרטי המשתמש מיושרים לצד השני. בנוסף, מוגדרים מאפייני עיצוב גלובליים כמו יישור אנכי למרכז, צבע רקע דינמי לפי התמה הפעילה, ופס הפרדה תחתון שמבדיל בין הניווט לתוכן הדף.

הסבר על הבר GuestLayout



הרכיב GuestLayout משמש כתבנית האב של אזור האורחים במערכת, והיא מיועדת לעטוף את כלל הדפים הציבוריים של האפליקציה לפני ביצוע התחברות. דפים כמו HomeView מקבלים באופן אוטומטי את סרגל הניווט העליון ואת חוויית המשתמש האחידה של מצב אורח.

פירוט רכיבי הדף

- AppLayout:** משמש כשלד המבני של כל המערכת. הוא מגדיר אזור קבוע בחלק העליון של המסך שבו מוצבים רכיבי השליטה, ומתחתיו אזור תוכן דינמי שבו נטענים כל הדפים. בצורה זו, המעבר בין מסכים אינו פוגע במבנה הכללי של המערכת, אלא רק מחליף את התוכן המרכזי.
- H3:** בחלקו השמאלי של הסרגל מוצגת כותרת מסוג H3 עם הטקסט "Smart Delivery System". רכיב זה משמש כלוגו טקסטואלי רשמי של הפרויקט ומחזק את הזהות הוויזואלית של המערכת בכל מסכי האורח.
- Span+Icon:** המחלקה משלבת רכיב Icon יחד עם רכיב Span המכיל את המילה "אורח". שני הרכיבים מאוגדים יחד בתוך HorizontalLayout ייעודי בשם guestInfo, ומטרתם לספק למשתמש חיווי ברור לכך שהוא פועל כרגע ללא התחברות וללא הרשאות מערכת מתקדמות.
- Button:** בסרגל העליון ממוקמים שני כפתורי פעולה אינטראקטיביים: כפתור ההתחברות מוגדר כפעולה הראשית ומעוצב כדי לבלוט ויזואלית. בעת לחיצה, הוא מפעיל ניווט אל מסך ההתחברות, וכפתור ההרשמה המשמש כפעולה משנית ומאפשר מעבר ישיר למסך יצירת משתמש חדש. הוא נשאר בעיצוב הניטרלי של כדי לשמור על היררכיה ויזואלית ברורה מול כפתור ההתחברות.
- HorizontalLayout:** המחלקה עושה שימוש במספר רכיבי HorizontalLayout כדי לארגן את הסרגל העליון בצורה יפה:
 - הצד השמאלי מאגד את לוגו המערכת יחד עם אזור סטטוס האורח. הצד הימני מרכז את כפתורי ההתחברות וההרשמה. השלישי מאגד את שני החלקים ומגדיר פריסה מלאה לרוחב המסך, הדוחף את שני האזורים לקצוות הנגדיים של הסרגל.

12. תיאור התוכנה

פרק זה מציג את הטכנולוגיות, מסגרות העבודה וכלי הפיתוח ששימשו למימוש המערכת. הבחירות נעשו מתוך מחשבה על יציבות, מודולריות, אבטחה ויכולת הרחבה עתידית.

12.1 סביבת הפיתוח של התוכנה

12.1.1 מערכת הפעלה

Windows 11: מערכת זו שימשה כתשתית העבודה המרכזית לכל תהליך הפיתוח, והעניקה סביבה יציבה ונוחה להרצת כלל כלי הפיתוח שנעשה בהם שימוש בפרויקט, כולל שפת Java, סביבת הפיתוח, הדפדפנים וכלי הבדיקה השונים.

Windows 11 נבחרה משום שהיא מספקת תמיכה רחבה בכלי פיתוח מודרניים, אינטגרציה נוחה עם סביבת עבודה גרפית, וניהול פשוט של קבצים, פרויקטים וחיבורים לרשת. בנוסף, המערכת מאפשרת עבודה רציפה ויעילה עם סביבות פיתוח נפוצות ללא צורך בהגדרות מורכבות, דבר התורם לזרימת עבודה חלקה ולפיתוח מהיר ויציב של המערכת כולה.

12.1.2 סביבות הפיתוח (IDEs)

Visual Studio Code (VS Code): משמש כסביבת העבודה המרכזית לפיתוח הפרויקט. הוא מאפשר כתיבת קוד נוחה, ניפוי שגיאות וניהול פרויקט באמצעות הרחבות שונות, כולל חיבור ל-Maven, עבודה עם MongoDB וכלי עזר לכתיבת דיאגרמות.

12.1.3 מסגרות לפיתוח התוכנה (Frameworks)

Spring Boot 3.5.8: משמש כתשתית המרכזית של צד השרת. הוא מנהל את כל שכבות המערכת: בקרים, שירותים, וגישה למסד הנתונים. בנוסף, הוא מפעיל שרת פנימי (Tomcat) ומטפל בבקשות HTTP בצורה אוטומטית ומסודרת.

Vaadin 24.9.5: הממשק Vaddin משמש לבניית ממשק המשתמש (Frontend) בשפת Java בלבד. הוא מאפשר יצירת מסכים אינטראקטיביים כמו טפסים ודשבורדים, כאשר כל הלוגיקה רצה בצד השרת והעדכונים נשלחים לדפדפן בזמן אמת, מה שמפשט את פיתוח הממשק ושומר על אחידות.

12.1.4 שפות תכנות

- **Java 21:** שפת Java היא השפה המרכזית שבה פותחה המערכת מקצה לקצה. היא מאפשרת פיתוח מונחה עצמים (OOP) בצורה מסודרת וברורה, ומספקת ביצועים גבוהים וניהול זיכרון יעיל. השימוש בגרסת LTS עדכנית (Java 21) מבטיח יציבות לאורך זמן ותמיכה בטכנולוגיות מודרניות. באמצעות Java מומשה גם לוגיקת השרת, גם שכבת הממשק וגם מנוע האלגוריתם.

12.1.5 ספריות/מערכות/מנגנוני עזר נוספים

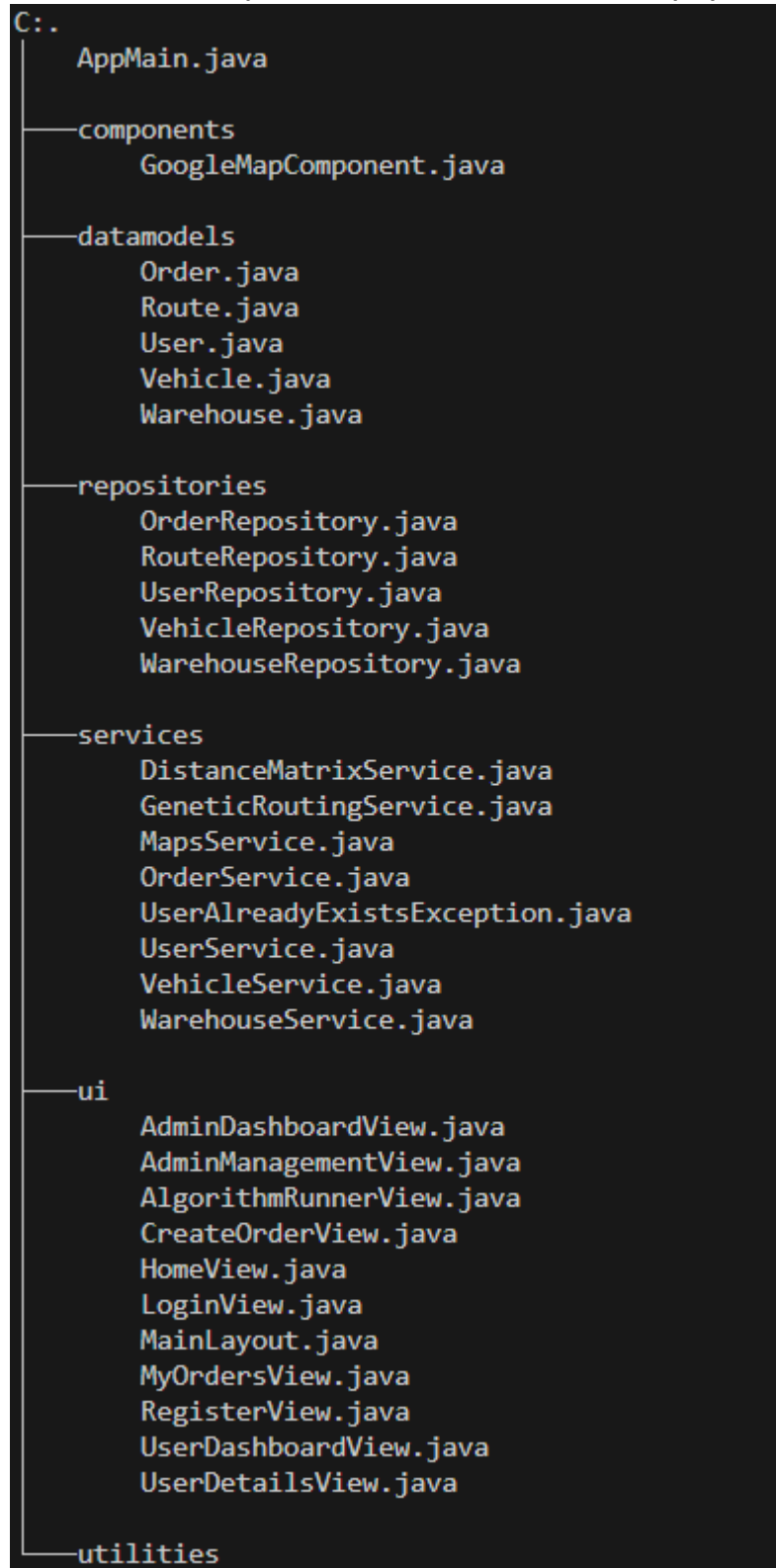
MongoDB Atlas: היא מערכת לניהול מסדי נתונים בענן מסוג NoSQL, אשר שימשה כבסיס הנתונים המרכזי של המערכת. הבחירה בה נובעת מהצורך בגמישות גבוהה בשמירת מידע, כך שניתן לאחסן נתונים כאובייקטים (Documents) במקום מבנה טבלאי קשיח. גישה זו מתאימה במיוחד למערכת מסוג זה, שבה קיימות ישויות דינמיות כמו משתמשים, הזמנות ורכבים, אשר עשויות להשתנות ולהתרחב לאורך זמן ללא צורך בשינויים מבניים במסד הנתונים. בנוסף, העבודה עם MongoDB Atlas מאפשרת גישה נוחה ומאובטחת דרך הענן מכל מקום, דבר התורם לפיתוח, בדיקות ותחזוקה יעילים יותר של המערכת.

Maven: הוא כלי לניהול ובנייה של פרויקטים בשפת Java, אשר שימש בפרויקט לניהול התלויות (Dependencies) והספריות החיצוניות בצורה אוטומטית ומסודרת. במקום להגדיר כל ספרייה באופן ידני Maven, מאפשר הורדה, עדכון וניהול גרסאות באופן מרכזי ואחיד, ובכך מונע בעיות תאימות בין רכיבי המערכת. בנוסף, הוא אחראי על תהליך הבנייה של הפרויקט, כולל קומפילציה והרצה, כך שכל סביבת פיתוח מקבלת מבנה עבודה עקבי ויציב. השימוש ב-Maven-תרם לפישוט תהליך הפיתוח, לשיפור הסדר בפרויקט ולהקטנת הסיכוי לשגיאות תשתית.

12.2 תיאור מבנה הפרויקט והמחלקות

12.2.1 עץ המודולים של הפרויקט

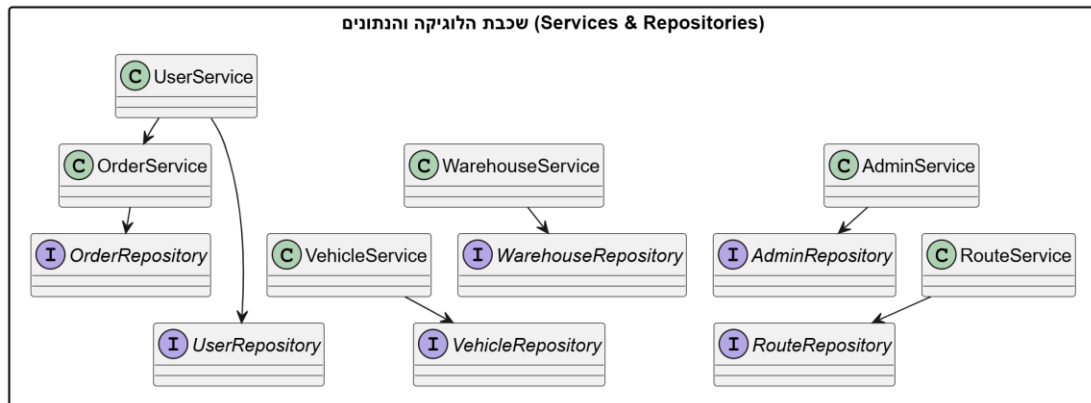
להלן עץ המודולים שמתאר את סדר הפרויקט במישור המבני:



12.2.2 תרשים המחלקות UML Class Diagram

על מנת שנוכל לראות את הכל בצורה מסודרת ושלא יראה מסורבל לעין, חילקתי את תרשים ה UML לכמה תרשימים קטנים יותר, שבעזרתם נבין את התמונה הכללית.

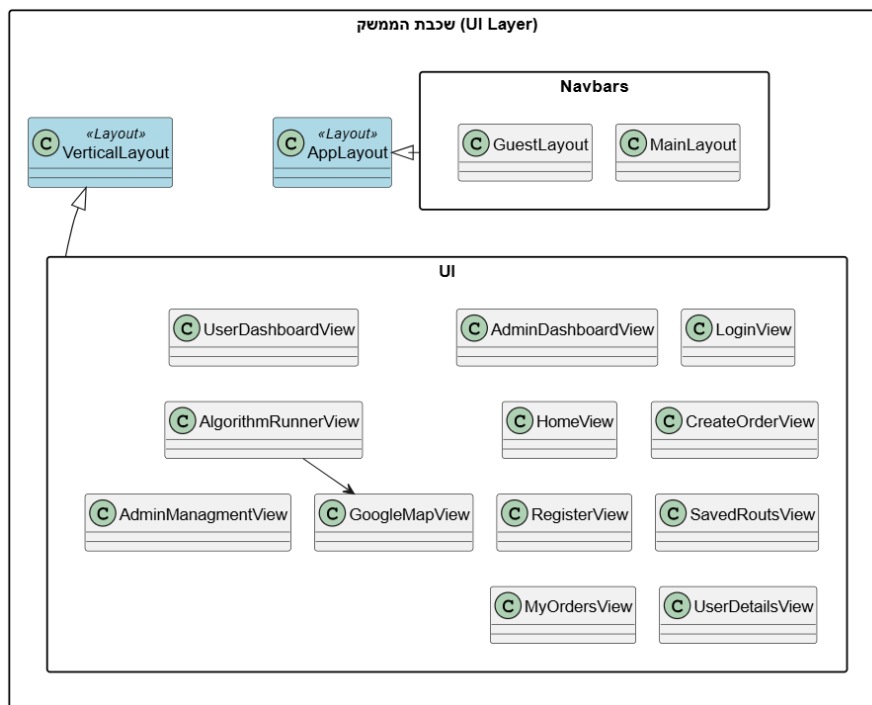
להלן תרשים המייצג תלויות בין Service ל Repository:



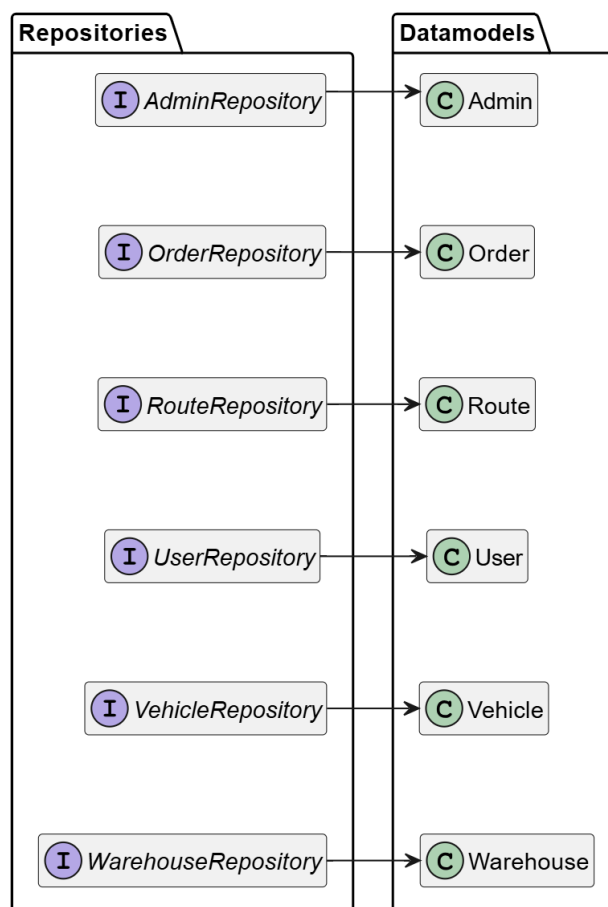
להלן תרשים נוסף המייצג תלויות בין UI ל Service:

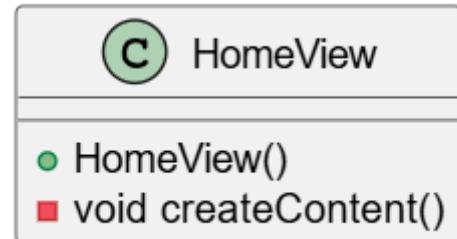


להלן תרשים המציג את תלויות דפי ה-UI וברי הניווט אל מול VerticalLayout ו-AppLayout



להלן תרשים נוסף המתאר את תלות ה-Repository ב Datamodels

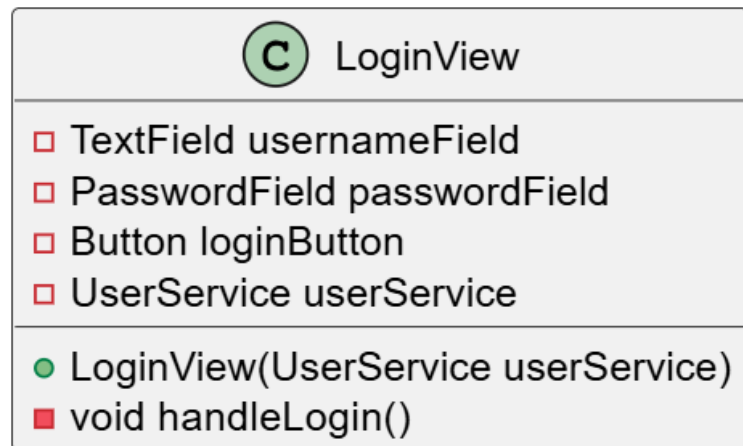


12.2.3 תיאור מחלקות מפורט**להלן תיאור מחלקות ראשיות:****:Views****HomeView**

הפונקציה createContent היא מתודת עזר פנימית שאחראית על בנייה והרכבה של ממשק המשתמש של העמוד בצורה מסודרת ומאורגנת. בפועל, היא משמשת כשכבה שמרכזת את כל הלוגיקה הוויזואלית של המסך ומגדירה אילו רכיבים יוצגו וכיצד הם ייראו. בשלב הראשון, הפונקציה יוצרת את רכיבי הממשק עצמם, כלומר מאתחלת את כל האובייקטים הוויזואליים שיופיעו בדף כגון כותרות, פסקאות טקסט, כפתורים ולעיתים גם רכיבים מורכבים יותר. כל רכיב נבנה בנפרד בהתאם לצורך של המסך ולתפקיד שלו בממשק. לאחר יצירת הרכיבים, הפונקציה מטפלת בשלב העיצוב והסידור שלהם. בשלב זה מוגדרים מאפייני התצוגה כמו יישור לאמצע, ריווחים בין רכיבים, גדלים, וסידור היררכי נכון של האלמנטים על המסך. שלב זה קובע למעשה את החוויה הוויזואלית של המשתמש ואת מבנה העמוד מבחינת UI.

לבסוף, כל הרכיבים שנוצרו ומעוצבים נאספים ומתווספים אל הקונטיינר הראשי של העמוד, כך שהם הופכים לחלק פעיל מהממשק ומוצגים בפועל בדפדפן.

LoginView

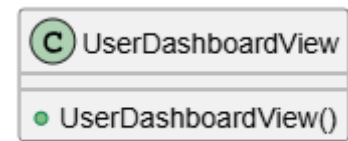


הפונקציה `handleLogin` היא הפונקציה המרכזית בתהליך ההתחברות של המשתמש למערכת, והיא מופעלת ברגע שהמשתמש לוחץ על כפתור ההתחברות במסך הכניסה. תפקידה הוא לנהל את כל תהליך האימות והניתוב של המשתמש בצורה מאובטחת ומבוססת הרשאות.

בשלב הראשון, הפונקציה מבצעת שליפה של הנתונים שהמשתמש הזין בזמן אמת, כלומר היא קוראת את הערכים משדות הקלט של שם המשתמש והסיסמה. ערכים אלו משמשים כבסיס לתהליך האימות מול שכבת הלוגיקה של המערכת.

לאחר מכן מתבצעת לוגיקת החלטה המבוססת על הרשאות משתמש. בשלב זה הפונקציה בודקת האם מדובר במנהל מערכת (Admin) באמצעות תנאי ייעודי, ובמידה ולא, היא פונה ל-`UserService` כדי לבצע אימות מול מסד הנתונים של המשתמשים הרגילים. כך מתבצע למעשה בידול בין סוגי משתמשים שונים כבר בשלב הכניסה למערכת.

בהתאם לתוצאת האימות, הפונקציה מבצעת ניתוב דינמי של המשתמש באמצעות `UI.navigate`. משתמש שאומת בהצלחה מנותב למסך המתאים לו, בין אם זה לוח הבקרה של המנהל או ממשק הלקוח, בעוד שמשתמש שלא עבר אימות נחסם מגישה למערכת ומקבל הודעת שגיאה מתאימה. בצורה זו הפונקציה מרכזת גם את האבטחה, גם את הבקרה הלוגית וגם את ניהול הניווט של המערכת בנקודה אחת.

UserDashboardView

המחלקה `UserDashboardView` משמשת כלוח הבקרה האישי ועמוד הבית של משתמש קצה רגיל במערכת, והיא מוצגת מיד לאחר תהליך התחברות מוצלח כאשר המשתמש אינו מוגדר כמנהל מערכת. מחלקה זו מהווה למעשה את נקודת הכניסה המרכזית של המשתמש לאזור האישי שלו בתוך המערכת.

המחלקה מייצרת הפרדה ברורה בין סוגי המשתמשים ומבטיחה שכל משתמש מגיע לסביבת עבודה המותאמת להרשאות שלו בלבד.

בנוסף, המחלקה מהווה את סביבת העבודה המרכזית של המשתמש, ובה מוצגים השירותים והפעולות הרלוונטיות עבורו. מתוך מסך זה המשתמש יכול לגשת לפעולות הליבה המותרות לו במערכת, כגון יצירת משלוחים חדשים, צפייה בהזמנות קיימות או מעבר למסכי מעקב שונים, בהתאם להרשאות שהוגדרו לו מראש.

מעבר לכך, המחלקה משתלבת כחלק אינטגרלי מתבנית המערכת הכללית (`MainLayout`) מה שמבטיח אחדות עיצובית וניווטית בכל רחבי האפליקציה. המשמעות היא שבכל מצב, מעל תוכן המסך יוצג הבר העליון הקבוע הכולל את פרטי המשתמש המחובר, אפשרויות התנתקות והחלפת תצוגה, וכך נשמרת חוויית משתמש עקבית, מאובטחת ואחידה לאורך כל השימוש במערכת.

CreateOrderView

 CreateOrderView
-mapsService : MapsService -orderService : OrderService -map : GoogleMapComponent -currentLat : double -currentLng : double -isLocationVerified : boolean
+CreateOrderView(MapsService, OrderService, String) -btnCheckAddress_Click() : void -btnSave_Click() : void

המחלקה CreateOrderView משמשת כממשק העבודה המרכזי ליצירת הזמנות חדשות במערכת, והיא מהווה נקודת מפגש בין הזנת נתונים רגילה לבין אימות גיאוגרפי בזמן אמת. דרך מחלקה זו המשתמש מזין את פרטי המשלוח, והמערכת דואגת להפוך אותם לישות לוגיסטית תקפה ומאמתת שניתן לעבד בהמשך על ידי מנוע האופטימיזציה.

אחת הפעולות המרכזיות במחלקה היא תהליך האימות וההמרה הגיאוגרפית btnCheckAddress_Click. כאשר המשתמש מזין כתובת טקסטואלית, המערכת פונה לשירות MapsService כדי לבצע Geocoding כלומר המרה של הכתובת לקואורדינטות גיאוגרפיות מדויקות. לאחר קבלת התוצאות, המיקום מוצג על גבי רכיב המפה האינטראקטיבי GoogleMapsComponent, כך שהמשתמש מקבל חיווי חזותי מיידי ויכול לאשר שהמיקום שהתקבל אכן נכון ומדויק.

בנוסף לכך, המחלקה מנהלת מנגנון בקרה פנימי באמצעות דגל מצב בשם isLocationVerified, מטרתו להבטיח שלמות ואמינות נתונים בתהליך יצירת ההזמנה. כל עוד הכתובת לא אומתה בהצלחה, המשתמש אינו יכול להשלים את תהליך השמירה, מה שמונע הכנסת נתונים שגויים או פיקטיביים למערכת.

לאחר השלמת שלב האימות והזנת כלל הנתונים הנדרשים, מופעלת פונקציית השמירה btnSave_click, פונקציה זו מרכזת את כל המידע שהוזן, כולל הקואורדינטות שאותרו, ומעבירה אותו לשכבת השירות OrderService לצורך שמירה במסד הנתונים. בשלב זה ההזמנה הופכת לחלק ממאגר הנתונים הפעיל של המערכת, ובהמשך תשמש כקלט מרכזי עבור האלגוריתם הגנטי אשר יבצע אופטימיזציה וחלוקת מסלולים לרכבים.

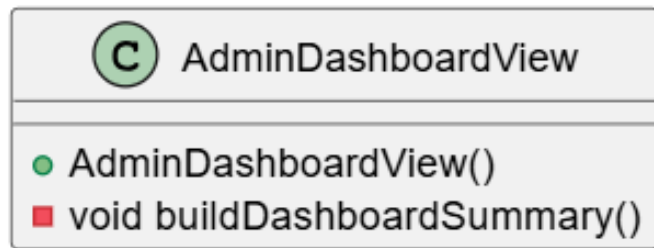
MyOrdersView

 MyOrdersView
-orderService : OrderService -grid : Grid<Order>
+MyOrdersView(OrderService) -updateGrid(String) : void -createStatusBadge(Order) : Span

המחלקה MyOrdersView משמשת כמסך המעקב והניהול האישי של המשתמש, ומאפשרת לו לצפות בצורה מרוכזת בכל ההזמנות ונקודות החלוקה שהוזנו על ידו למערכת. המסך בנוי במבנה טבלאי ברור ונקי, שמטרתו להציג כמויות מידע גדולות בצורה קריאה, מאורגנת ונגישה, תוך שמירה על חוויית משתמש פשוטה ואינטואיטיבית.

אחת הפעולות המרכזיות במחלקה היא תצוגת הנתונים הדינמית באמצעות הפונקציה `updateGrid`. פונקציה זו אחראית לרענון מתמיד של טבלת ההזמנות (`Grid<Order>`) על בסיס המידע העדכני ביותר. היא פונה ל- `OrderService` שולפת את רשימת ההזמנות המשויות למשתמש הנוכחי ממסד הנתונים, ומעדכנת את רכיב הטבלה בהתאם. כך מתקבלת תצוגה חיה ודינמית המשקפת בכל רגע את מצב ההזמנות בפועל.

בנוסף לכך, המחלקה כוללת מנגנון חיווי ויזואלי מתקדם באמצעות הפונקציה `createStatusBadge`. פונקציה זו יוצרת רכיב גרפי מסוג `Badge` המבוסס על `Span` אשר מציג את סטטוס ההזמנה בצורה צבעונית וברורה. כל סטטוס (כגון "ממתין לשיבוץ", "שובץ למסלול" או "נמסר") מיוצג בצבע שונה, כך שהמשתמש יכול להבין את מצב כלל ההזמנות שלו במבט מהיר, ללא צורך בקריאת טקסט מפורט.

AdminDashboardView

המחלקה משמשת כלוח הבקרה הראשי של מנהל המערכת ומהווה את נקודת הכניסה המרכזית לסביבת הניהול. זהו עמוד נחיתה ייעודי עבור משתמשים בעלי הרשאות אדמין בלבד, והוא נטען מיד לאחר תהליך ההתחברות, בהתאם לזיהוי התפקיד של המשתמש במערכת. למערכת מספר פעולות:

מרכז שליטה וניווט: המחלקה מהווה ממשק ניווט מרכזי שמרכז את כל הכלים הניהוליים במקום אחד. ממנה המנהל יכול לעבור למסכי המערכת הקריטיים כגון ניהול משתמשים, צפייה וניהול הזמנות, ניהול תשתיות לוגיסטיות והרצת מנוע האופטימיזציה הגנטי. בכך היא משמשת כ"שער כניסה" לכל שכבת הניהול של המערכת.

הפרדת הרשאות וארכיטקטורת מערכת: עצם קיומו של עמוד נפרד למנהל, בנפרד מממשק המשתמש הרגיל, מיישם עקרון של הפרדה בין שכבות (Separation of Concerns) וניהול הרשאות מבוסס תפקידים. המשמעות היא שמשתמש רגיל כלל לא נחשף לרכיבי הניהול, מה שמחזק את אבטחת המערכת ומונע גישה לא מורשית לפעולות קריטיות.

AlgorithmRunnerView

 AlgorithmRunnerView
❑ OrderService orderService ❑ VehicleService vehicleService ❑ WarehouseService warehouseService ❑ DistanceMatrixService matrixService ❑ GeneticRoutingService geneticService ❑ GoogleMapComponent map ❑ VerticalLayout sidebarContent
● AlgorithmRunnerView(...) ■ void executeAlgorithm() ■ void drawResults(Map<String, List<Order>>, LatLng depotLoc) ■ void createVehicleCard(String vehicleId, List<Order> orders, String color)

המחלקה AlgorithmRunnerView משמשת כנקודת השיא החישובית והוויזואלית של המערכת, ומהווה את הממשק המרכזי שבו מתבצע התהליך המלא של תכנון ואופטימיזציה של מסלולי חלוקה. זהו המסך שבו כל שכבות המערכת מתחברות יחד: נתוני הבסיס (משתמשים, הזמנות, רכבים ומחסן) מוזרמים למנוע החישוב, וממנו מתקבלת תוצאה גרפית של פתרון המסלולים האופטימלי על גבי מפה אינטראקטיבית.

הפעולות המרכזיות שלה

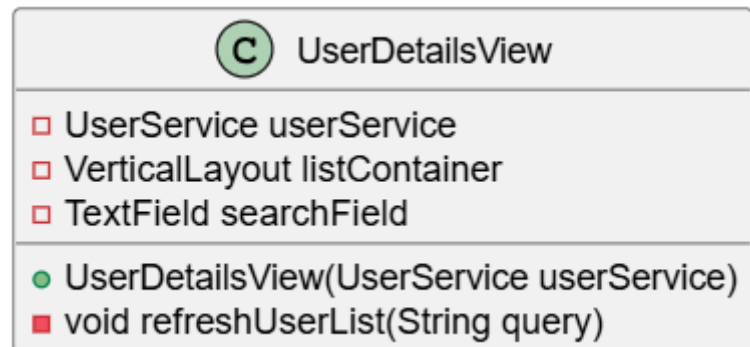
תזמור והפעלת מנוע החישוב: ExecuteAlgorithm
פונקציה זו משמשת כנקודת ההפעלה המרכזית של המערכת. עם לחיצת המשתמש, היא מבצעת איסוף של כלל הנתונים הרלוונטיים מהמערכת (הזמנות ממתנות, צי רכבים פעיל ומיקום המחסן), ולאחר מכן מפעילה את DistanceMatrixService לצורך בניית מטריצת מרחקים מדויקת. בהמשך היא מעבירה את המידע המעובד ל-GeneticAlgorithmService, שם מתבצעת הרצה של האלגוריתם הגנטי למציאת חלוקת מסלולים אופטימלית תחת אילוצי קיבולת ומרחק.

המחשה ויזואלית של תוצאות: drawResults
לאחר קבלת הפתרון מהמנוע החישובי, פונקציה זו אחראית על תרגום הנתונים המספריים לתצוגה גרפית אינטראקטיבית. היא משתמשת ברכיב GoogleMapComponent כדי למקם את המחסן כנקודת מוצא, להציב סמנים על כל תחנות החלוקה, ולצייר מסלולי נסיעה (Polylines) על בסיס כבישים אמיתיים. כל רכב מקבל צבע ייחודי, מה שמאפשר הבחנה ברורה בין המסלולים השונים בזמן אמת.

בניית תצוגת צד דינמית: createVehicleCard
במקביל לתצוגת המפה, המחלקה בונה פאנל צד דינמי שבו מוצגות כרטיסיות מידע עבור כל רכב. כל

כרטיס מציג את סדר התחנות המדויק שהוקצה לרכב, כולל פרטי ההזמנות והיעדים. כך המשתמש מקבל ייצוג כפול של אותה תוצאה: גם גרפי על המפה וגם טקסטואלי ברור, מה שמחזק את ההבנה והבקרה על תוצאות האלגוריתם.

UserDetailsView



המחלקה משמשת כממשק הניהול המרכזי של משתמשי המערכת, והיא מיועדת למנהל המערכת בלבד. זהו מסך תפעולי שמרכז את כל היכולות הנדרשות לצפייה, איתור וניהול של משתמשים רשומים, תוך שמירה על תצוגה פשוטה, מהירה ודינמית שמותאמת לעבודה שוטפת של מנהל מערכת.

הפעולות המרכזיות שלה:

חיפוש וסינון בזמן אמת:

ליבת הפעולה של המחלקה מבוססת על שדה חיפוש טקסטואלי שמאפשר איתור משתמשים לפי שם. בכל שינוי או הפעלה של החיפוש, הפונקציה שולחת את ערך החיפוש ל-UserService שמבצע שאילתה מול מסד הנתונים ומחזיר את רשימת המשתמשים התואמת. לאחר מכן מתבצע רענון מיידי של התצוגה, כך שהתוצאות משתקפות על המסך בזמן אמת ללא צורך ברענון עמוד.

רינדור דינמי של רשימת משתמשים:

המחלקה משתמשת בקונטיינר אנכי המשמש כשטח תצוגה דינמי. בכל חיפוש מחדש, הקונטיינר מתנקה מהתוכן הקודם ונבנה מחדש בהתאם לתוצאות שהתקבלו. עבור כל משתמש נוצרת כרטיסיית מידע נפרדת, מה שמאפשר תצוגה ברורה, מבודדת ונוחה לקריאה. גישה זו גם שומרת על ביצועים טובים, שכן התצוגה נוצרת לפי הצורך בלבד.

הפרדת לוגיקה והזרקת תלויות:

המחלקה מיישמת ארכיטקטורה נקייה בכך שהיא אינה ניגשת ישירות למסד הנתונים. במקום זאת, היא מקבלת את UserService דרך הבנאי, ומעבירה אליו את כל פעולות הלוגיקה העסקית. כך נשמרת הפרדה ברורה בין שכבת ה-UI לבין שכבת השירותים, והמחלקה מתמקדת אך ורק בניהול אינטראקציות המשתמש והצגת הנתונים.

AdminManagementView

 AdminManagementView
-warehouseService : WarehouseService -vehicleService : VehicleService -mapsService : MapsService -map : GoogleMapComponent -vehicleGrid : Grid<Vehicle>
+AdminManagementView(WarehouseService, VehicleService, MapsService, String) -btnSaveWarehouse_Click() : void -btnAddVehicle_Click() : void -updateVehicleGrid() : void

המחלקה משמשת כממשק התשתית וההגדרות המרכזי של מנהל המערכת, ומהווה את השלב המקדים והחיוני להפעלת כל מנגנון האופטימיזציה של המערכת. דרך מסך זה המנהל מגדיר את המשאבים הפיזיים שעליהם מתבסס האלגוריתם הגנטי: מיקום המחסן המרכזי (Depot) וצי הרכבים הזמין לביצוע חלוקת המשלוחים.

הפעולות המרכזיות שלה:

ניהול גיאוגרפי של המחסן: (btnSaveWarehouse_Click)

המחלקה מאפשרת למנהל להגדיר את נקודת המוצא הלוגיסטית של המערכת באמצעות שילוב בין רכיב מפה אינטראקטיבי (GoogleMapComponent) לבין שירות ה- MapsService המנהל מזין כתובת, והמערכת מבצעת המרה לקואורדינטות גיאוגרפיות (Geocoding) ומציגה את המיקום על גבי המפה לאימות חזותי. לאחר שמירת הנתונים, המחסן הופך לנקודת הייחוס המרכזית של כל חישובי המסלולים במערכת.

ניהול צי רכבים דינמי: (btnAddVehicle, btnAddVehicle_Click)

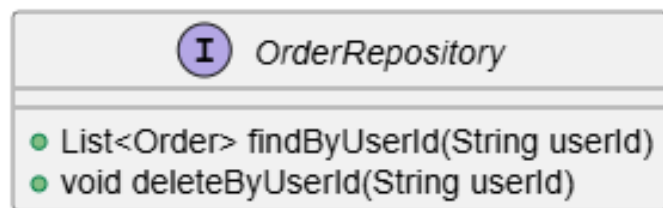
המחלקה מספקת ממשק ניהול מלא לרכבי החלוקה, כולל הוספה, עדכון והצגה של רכבים במערכת. לכל רכב מוגדרים פרטים כגון מספר רישוי וקיבולת נשיאה מקסימלית. הנתונים נשמרים באמצעות VehicleService ומוצגים בטבלה דינמית (Grid<Vehicle>) המתעדכנת בזמן אמת, כך שלמנהל קיימת תמונת מצב עדכנית של כלל המשאבים הזמינים לביצוע חלוקה.

הגדרת אילוצי המערכת לבעיה הלוגיסטית: (VRP Constraints)

מבחינה ארכיטקטונית, המחלקה אחראית על יצירת שכבת האילוצים של בעיית הניתוב. הנתונים שמוגדרים במסך זה: מספר הרכבים, קיבולות, ומיקום המחסן, מהווים את הבסיס המתמטי שעליו פועל ה- GeneticRoutingService. ללא הגדרות אלו, לא ניתן להריץ את האלגוריתם או לייצר פתרון תקף לבעיית חלוקת המשלוחים.

:Repositories

OrderRepository



הממשק OrderRepository מהווה את שכבת הגישה לנתונים (Data Access Layer) של ישות ההזמנות במערכת, ומשמש כגשר מרכזי בין שכבת הלוגיקה העסקית לבין מסד הנתונים MongoDB. באמצעותו מתבצעת אינטראקציה מלאה עם נתוני ההזמנות, ללא צורך בכתיבת שאילתות ידניות או ניהול חיבורים למסד הנתונים.

הפעולות המרכזיות שלו:

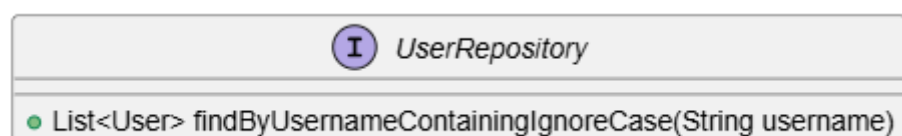
שליפה מבוססת משתמש: findById()

הממשק עושה שימוש במנגנון Spring Data שמאפשר יצירת שאילתות אוטומטיות מתוך שם המתודה. לדוגמה, המתודה findById מאפשרת שליפה של כל ההזמנות המשויות למשתמש מסוים, בהתאם למזהה שלו. מנגנון זה הוא הבסיס להצגת נתונים ממוקדים במסכים כמו myOrdersView, ומבטיח שכל משתמש רואה אך ורק את המידע האישי שלו.

מחיקה לפי משתמש: deleteById()

הממשק כולל יכולת מחיקה גורפת של כל ההזמנות המשויות למשתמש מסוים. פעולה זו חיונית במיוחד בתהליכי ניהול משתמשים, כאשר נדרש לבצע מחיקה מלאה של חשבון משתמש כולל כל ההזמנות הקשורות אליו. כך נשמרת שלמות הנתונים במסד הנתונים ונמנעת היווצרות רשומות יתומות.

UserRepository



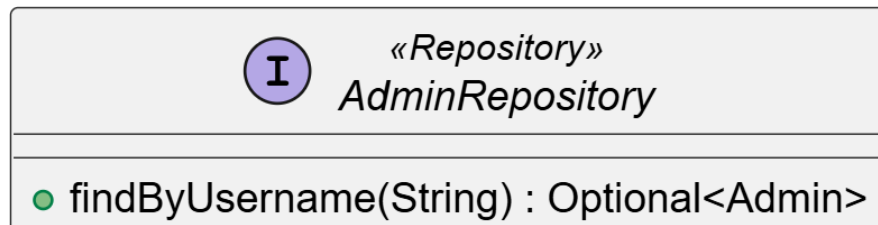
הממשק UserRepository משמש כשער הגישה המרכזי של המערכת אל אוסף המשתמשים (Users Collection) במסד הנתונים, ותפקידו ליצור הפרדה מלאה בין הלוגיקה העסקית של המערכת לבין האופן שבו הנתונים נשמרים ונשלפים בפועל.

באמצעות הממשק הזה, כל הפעולות מול נתוני המשתמשים מתבצעות בצורה מופשטת ונקייה, ללא צורך בכתיבת שאילתות מסד נתונים ידניות. בכך נשמרת ארכיטקטורה מודולרית שבה השירותים (Services) עובדים מול ממשק בלבד, ולא מול בסיס הנתונים עצמו.

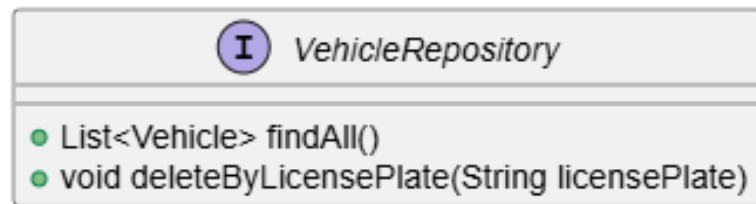
הפעולות המרכזיות שלו:

מנגנון חיפוש גמיש: `(findByUsernameContainingIgnoreCase)` מתודה זו מהווה את בסיס מנוע החיפוש במסך ניהול המשתמשים. `(UserDetailsView)` היא מאפשרת חיפוש חלקי של שמות משתמשים באמצעות `Containing`, כך שגם הזנה של תת-מחרוזת תחזיר תוצאות רלוונטיות. בנוסף, השימוש ב-`IgnoreCase` מבטל תלות ברישיות אותיות, מה שמאפשר חוויית חיפוש נוחה, אינטואיטיבית וסלחנית לשגיאות הקלדה. הממשק מדגים את היכולת של `Spring Data` לייצר שאילתות באופן אוטומטי מתוך שם המתודה בלבד. כלומר, אין צורך לכתוב קוד `MongoDB` או `SQL`, שם המתודה מתורגם בזמן ריצה לשאילתה מדויקת מול בסיס הנתונים. גישה זו מפשטת את הקוד, מצמצמת טעויות ומאיצה את תהליך הפיתוח. בנוסף למתודות המוגדרות בו, הממשק יורש באופן אוטומטי סט פעולות בסיסיות `(Update, Delete, Create, Read)`, כגון שמירת משתמשים חדשים, שליפה לפי מזהה ומחיקה. כך ניתן לבצע את כל פעולות ניהול המשתמשים דרך שכבה אחת עקבית, ללא צורך במימוש נוסף של לוגיקת גישה לנתונים.

AdminRepository



הממשק `AdminRepository` משתייך לשכבת הגישה לנתונים של המערכת, ותפקידו לנהל את התקשורת הישירה מול מסד הנתונים `MongoDB Atlas` בכל הקשור למנהלי המערכת. הפעולה המרכזית שמוגדרת בו היא חיפוש מנהל לפי שם משתמש. פעולה זו מאפשרת למערכת לבדוק האם קיים מנהל עם שם המשתמש שהוזן, ולהחזיר את הנתונים שלו אם כן. אם לא נמצא מנהל כזה, המערכת יודעת להתמודד עם זה בצורה בטוחה בלי להיכשל. מבחינת מבנה הפרויקט, הממשק עוזר לשמור על סדר במערכת בכך שהוא מפריד בין הלוגיקה של האפליקציה לבין האופן שבו הנתונים נשמרים בפועל, כך ששאר חלקי המערכת יכולים להשתמש במידע בצורה פשוטה וברורה.

VehicleRepository

הממשק `VehicleRepository` משמש כשכבת הגישה לנתונים (`Data Access Layer`) עבור ישות הרכב במערכת, והוא אחראי על התקשורת הישירה מול אוסף הרכבים במסד הנתונים `MongoDB`, באמצעותו מתבצעות פעולות של שמירה, שליפה ועדכון של צי הרכבים, המשמש כמשאב קריטי בתהליך תכנון המסלולים של המערכת. הממשק מאפשר לשכבת השירותים לעבוד מול מודל אובייקטיבי ונקי, תוך הסתרה מלאה של פרטי המימוש של מסד הנתונים, ובכך שומר על ארכיטקטורה מופרדת וברורה בין לוגיקה עסקית לבין שכבת האחסון.

הפעולות המרכזיות שלו:

שליפת כלל הרכבים: (`findAll`)

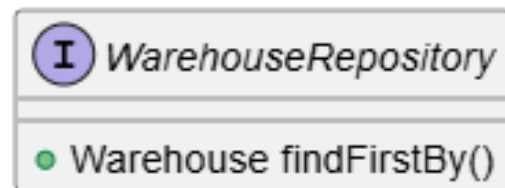
מתודה זו, המתקבלת לרוב בירושה מממשק הבסיס של `Spring Data`, מאפשרת שליפה של כל הרכבים הרשומים במערכת. פעולה זו היא שלב בסיסי והכרחי לפני הרצת האלגוריתם הגנטי, שכן היא מספקת למנוע החישוב את כלל המשאבים הזמינים: כולל מספר הרכבים והקיבולת של כל אחד מהם לצורך בניית פתרון לבעיית הניתוב.

מחיקה לפי לוחית רישוי: (`deleteByLicensePlate`)

מתודה זו מממשת גישה מבוססת מזהה עסקי ולא מזהה טכני, כאשר לוחית הרישוי משמשת כמפתח הייחודי של הרכב בעולם האמיתי. באמצעות מנגנון `Query Derivation` של `Spring Data`, שם המתודה מתורגם אוטומטית לשאילתת מחיקה מול מסד הנתונים. יכולת זו מאפשרת למנהל המערכת להסיר רכבים בצורה אינטואיטיבית, ישירות מתוך מסך הניהול, ללא צורך במזהים פנימיים.

הפשטת שכבת הנתונים: (`Abstraction Layer`)

הממשק מסתיר לחלוטין את מורכבות העבודה מול `MongoDB` ומאפשר לשכבת ה-`VehicleService` לעבוד מול אובייקטים בלבד. בכך נשמרת הפרדה ברורה בין הלוגיקה העסקית לבין תשתית הנתונים, מה שמגביר את הגמישות, מקל על תחזוקת הקוד, ומאפשר הרחבה עתידית של המערכת ללא שינוי בשכבת השירותים.

WarehouseRepository

הממשק WarehouseRepository משמש כשכבת הגישה לנתונים (Data Access Layer) עבור ישות המחסן במערכת, והוא אחראי על ניהול התקשורת מול מסד הנתונים לצורך שמירה ושליפה של המחסן המרכזי. המחסן מייצג את נקודת המוצא הלוגיסטית של כלל המערכת, ולכן הוא רכיב קריטי בכל תהליך חישוב המסלולים.

הממשק פועל כחלק מארכיטקטורת Spring Data, ומאפשר ביצוע פעולות מול בסיס הנתונים ללא צורך בכתיבת שאלות ידניות, תוך שמירה על קוד נקי, מופשט וקל לתחזוקה.

הפעולות המרכזיות שלו:

שליפת המחסן המרכזי: (findFirstBy)

מתודה זו עושה שימוש במנגנון Query Derivation של Spring Data כדי לשלוף את רשומת המחסן היחידה הקיימת במערכת. מאחר שהמערכת מבוססת על מודל של מחסן מרכזי אחד, אין צורך בחיפוש לפי מזהה ספציפי, והמתודה מחזירה באופן ישיר את נקודת המוצא הלוגיסטית. פתרון זה מבטיח שליפה מהירה, פשוטה ויעילה של נתוני המחסן בכל פעם שהמערכת נדרשת לבצע חישוב מסלולים.

אינטגרציה עם מנוע האופטימיזציה:

הנתונים המוחזרים מהממשק, בעיקר קואורדינטות המחסן, משמשים כבסיס לכל חישוב במערכת. הם מועברים אל DistanceMatrixService לצורך חישוב מרחקים, ומשם אל GeneticAlgorithmService, שם הם מוגדרים כנקודת התחלה וסיום מחייבת לכל מסלול. בכך המחסן הופך לעוגן המרכזי של כל פתרון שמייצר האלגוריתם.

הפשטת גישה לנתונים: (Data Abstraction)

הממשק מסתיר לחלוטין את פרטי העבודה מול MongoDB ומאפשר לשכבת השירותים לעבוד מול אובייקטים בלבד. גישה זו שומרת על הפרדה ברורה בין הלוגיקה העסקית לבין שכבת הנתונים, ומבטיחה שהמערכת תישאר גמישה, מודולרית וקלה להרחבה בעתיד.

Services

C DistanceMatrixService	
String apiKey	
double[][] buildDistanceMatrix(List<Order> orders, LatLng warehouseLoc)	
double[][] buildAirDistanceMatrix(List<Order> orders, LatLng warehouseLoc)	

המחלקה משמשת כמנוע הגיאוגרפי והחישובי המקדים של המערכת, ותפקידה הוא לבנות תשתית מספרית מדויקת שמייצגת את כל המרחקים בין הנקודות השונות במערכת, מחסן המוצא ונקודות החלוקה. תשתית זו מהווה בסיס הכרחי להרצת האלגוריתם הגנטי, שכן היא מתרגמת את העולם הפיזי לייצוג מתמטי שניתן לעיבוד מהיר ויעיל.

הפעולות המרכזיות שלה:

בניית מטריצת מרחקים: (buildDistanceMatrix)

זוהי הפונקציה המרכזית במחלקה, אשר מקבלת את כל נקודות היעד (הזמנות) יחד עם מיקום המחסן, ובונה מהם מטריצה דו־ממדית מסוג double. כל תא במטריצה מייצג את המרחק בין שתי נקודות שונות במערכת. מבנה זה הוא קריטי לבעיית הניתוב, מאחר שהוא מאפשר לאלגוריתם הגנטי לבצע שליפות מרחק בזמן קבוע של $O(1)$. במקום לבצע חישוב חוזר בכל גישה. בכך מושגת האצה משמעותית בביצועים כאשר האלגוריתם רץ אלפי איטרציות.

אינטגרציה עם שירותי מפות חיצוניים:

המחלקה עושה שימוש במפתח API ייעודי (apiKey) לצורך תקשורת עם שירותי מפות חיצוניים כגון GoogleMapsAPI. באמצעות חיבור זה, המערכת מחשבת מרחקים המבוססים על נתיבי נסיעה אמיתיים בכבישים, ולא רק על מרחק גיאומטרי. כך מתקבלת תוצאה מדויקת יותר שמייצגת את המציאות התפעולית של מערכת ההפצה.

מנגנון גיבוי לחישוב מרחקים: (Graceful Degradation)

במקרים שבהם לא ניתן לגשת לשירות החיצוני, או כאשר נדרש חישוב מהיר יותר ללא תלות ברשת, המחלקה מפעילה מנגנון חלופי באמצעות buildAirDistanceMatrix. מנגנון זה מחשב מרחקים אוויריים (בקו ישר) בין נקודות על בסיס קואורדינטות גיאוגרפיות. פתרון זה מבטיח שהמערכת תמשיך לפעול בצורה יציבה גם בתנאי תקלה או מגבלות חיבור, תוך שמירה על זמינות גבוהה של מנוע החישוב.

GeneticRoutingService

 GeneticRoutingService
□ <code>int POPULATION_SIZE</code> □ <code>int GENERATIONS</code> □ <code>double MUTATION_RATE</code>
● <code>Map<String, List<Order>> calculateRoutes(List<Order> orders, List<Vehicle> vehicles, Warehouse depot, double[][] matrix)</code> ■ <code>double calculateFitness(List<Order> sequence, List<Vehicle> vehicles, double[][] matrix)</code> ■ <code>Map<String, List<Order>> decodeSolution(List<Order> sequence, List<Vehicle> vehicles)</code> ■ <code>List<Order> crossover(List<Order> p1, List<Order> p2)</code> ■ <code>void mutate(List<Order> individual)</code> ■ <code>List<List<Order>> initializePopulation(List<Order> orders)</code> ■ <code>List<Order> selectParent(List<List<Order>> population)</code>

המחלקה משמשת כמנוע האופטימיזציה המרכזי של המערכת ומיישמת אלגוריתם גנטי לפתרון בעיית ניתוב הרכבים. מטרתה היא למצוא חלוקת מסלולים אופטימלית עבור צי הרכבים, תוך מזעור מרחקי נסיעה ועמידה באילוצי קיבולת וזמינות.

הפעולה המרכזית של המחלקה היא הפונקציה `calculateRoutes`, שמנהלת את כל תהליך החישוב לאורך מספר דורות. בשלב הראשון נבנית אוכלוסיית פתרונות התחלתית, כאשר כל פתרון מייצג חלוקה אפשרית של הזמנות בין רכבים. לאחר מכן מתבצע תהליך אבולוציוני איטרטיבי שבו בכל דור נבחרת איכות הפתרונות באמצעות פונקציית התאמה המודדת את איכות המסלול הכוללת לפי מרחקים ואילוצים.

במהלך הריצה מיושמים מנגנוני אבולוציה מרכזיים: פעולת `crossover` משלבת בין שני פתרונות "הורים" באמצעות שיטה מסוג `Order Crossover` השומרת על סדר תחנות תקין ומייצרת פתרונות חדשים תוך שמירה על מידע מוצלח מהדורות הקודמים. בנוסף, פונקציית `mutate` מבצעת שינויים אקראיים מבוקרים לפי שיעור `MUTATION_RATE`, כדי לשמור על שונות באוכלוסייה ולמנוע היתקעות בפתרונות לא אופטימליים (local optimal).

בסיום התהליך, הפתרון הטוב ביותר עובר פענוח באמצעות `decodeSolution` המתרגם את הייצוג הגנטי (Chromosome) לרשימת מסלולים מסודרת עבור כל רכב. תוצאה זו משמשת את שכבת התצוגה להצגה גרפית על גבי המפה ולביצוע בפועל של חלוקת ההזמנות.

DistanceMatrixService ו- MapsService

C MapsService
+getLatLngFromAddress(String) : LatLng

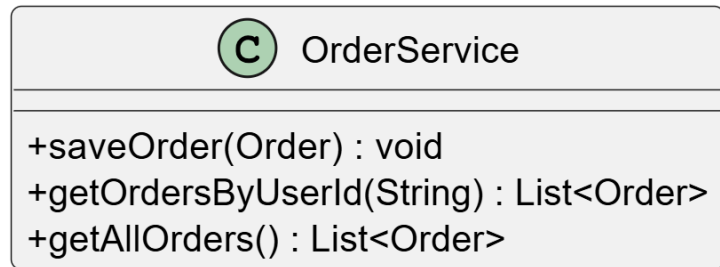
C DistanceMatrixService
+buildDistanceMatrix(List<Order>, LatLng) : double[][]

תרשים זה מציג את שכבת השירותים הגיאוגרפיים של המערכת, המשמשת כגשר בין העולם הפיזי (כתובות, מיקומים וכבישים) לבין השכבה החישובית שעליה מבוסס מנוע האופטימיזציה. שכבה זו אחראית להפוך מידע טקסטואלי שמזן על ידי המשתמש לנתונים גיאוגרפיים מדויקים שניתן לעבד אלגוריתמית.

הפעולה המרכזית של MapsService היא המרת כתובות לקואורדינטות (Geocoding) באמצעות הפונקציה `getLatLngFromService`. פונקציה זו מקבלת כתובת טקסטואלית ומבצעת קריאה לשירות מפות חיצוני, כגון Google Maps API על מנת לקבל ייצוג מדויק של המיקום בצורת קווי רוחב ואורך (latitude, longitude). תוצאה זו מאפשרת למערכת לעבוד עם נקודות גיאוגרפיות מדויקות במקום מחרוזות טקסט לא מובנות.

בנוסף, המחלקה משמשת כשכבת אימות מרכזית המוודאת שכל כתובת שהוזנה אכן קיימת וניתנת לאיתור. במקרים בהם הכתובת אינה תקינה או לא מזוהה, מוחזרת שגיאה והנתון אינו נשמר במערכת, מה שמונע חדירה של מידע שגוי למסד הנתונים ושומר על איכות הנתונים של המערכת כולה.

לבסוף MapsService, מהווה רכיב תשתיתי קריטי בשרשרת העיבוד: היא מספקת את הקלט המדויק ל-distanceMatrixService אשר מסתמך על הקואורדינטות כדי לחשב מרחקים אמיתיים בין נקודות. בכך היא מאפשרת למנוע האופטימיזציה הגנטי לעבוד על בסיס נתונים גיאוגרפיים אמיתיים ולא על שמות או כתובות טקסטואליות בלבד.

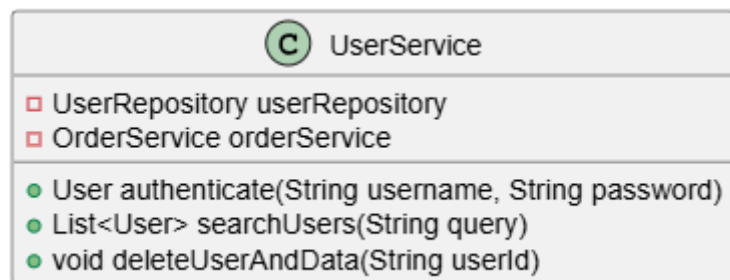
OrderService

המחלקה משמשת כשכבת הלוגיקה העסקית המרכזית לניהול ההזמנות ונקודות החלוקה במערכת. היא מהווה את המתווך בין שכבת ממשק המשתמש לבין שכבת הגישה לנתונים, ובכך מוודאת שהעבודה מול מסד הנתונים מתבצעת בצורה מבודדת, עקבית ומבוקרת.

הפעולה המרכזית של המחלקה היא `saveOrder`, שמקבלת אובייקט מסוג `Order` ושומרת אותו במסד הנתונים. פעולה זו מבטיחה שכל הזמנה שנקלטת במערכת נכתבת באופן תקין וניתן יהיה לשלוף אותה בהמשך לצורך מעקב או חישוב מסלולים.

בנוסף, הפונקציה `getOrdersByUserId` אחראית על שליפת הזמנות ממוקדת לפי משתמש. פונקציה זו מיישמת הפרדה מלאה בין משתמשים שונים, כך שכל לקוח מקבל גישה רק להזמנות האישיות שלו, והיא משמשת בסיס ישיר להצגת הנתונים במסך `MyOrdersView`.

פונקציה נוספת ומשמעותית היא `getAllOrders`, המשמשת את צד המערכת והמנהל. היא שולפת את כלל ההזמנות הפעילות במערכת ומשמשת כקלט מרכזי עבור תהליך הרצת האלגוריתם במסך `AlgorithmRuunnerView`, דרך פונקציה זו מוזרמים כל נתוני נקודות החלוקה אל מנוע האופטימיזציה לצורך בניית מסלולים אופטימליים.

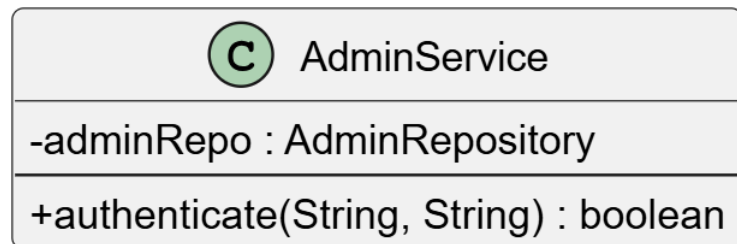
UserService

המחלקה UserService משמשת כשכבת הלוגיקה העסקית (Service Layer) המרכזית לניהול משתמשי המערכת. היא אחראית על תהליך ניהול הזהויות מקצה לקצה, ומשמשת כמתווך בין שכבת הממשק לבין שכבת הגישה לנתונים. תפקידה הוא לא רק לבצע פעולות CRUD, אלא גם לאכוף חוקים עסקיים ולשמור על שלמות ואבטחת המידע.

הפעולה המרכזית של המחלקה היא authenticate, המשמשת כמנגנון האימות של המערכת. פונקציה זו מקבלת פרטי התחברות מהמשתמש, מבצעת בדיקה מול מסד הנתונים באמצעות ה- UserRepository, ומחזירה אובייקט משתמש תקף רק במקרה של התאמה מלאה. הצלחת תהליך זה מאפשרת יצירת Session פעיל והענקת גישה למערכת, בעוד כישלון מוביל לחסימת הכניסה.

בנוסף, המחלקה מספקת את פונקציית searchUsers, המשמשת את מסך הניהול UserDetailsView. פונקציה זו מאפשרת חיפוש וסינון דינמי של משתמשים על בסיס מחרוזת חלקית, ובכך תומכת במנגנון חיפוש מהיר ואינטראקטיבי בזמן אמת עבור מנהל המערכת.

פונקציה קריטית נוספת היא deleteUserAndData, המממשת מנגנון מחיקה מקושר. בעת מחיקת משתמש, הפונקציה דואגת תחילה להסיר את כל ההזמנות המשויות אליו דרך OrderService, ורק לאחר מכן מוחקת את המשתמש עצמו. גישה זו מונעת היווצרות של נתונים יתומים ומבטיחה שמירה על עקביות וניקיון של מסד הנתונים לאורך זמן.

AdminService

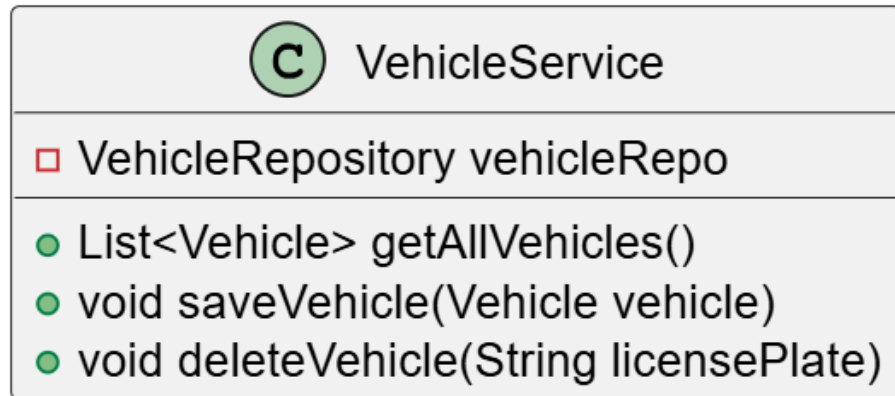
המחלקה משתייכת לשכבת הלוגיקה העסקית של המערכת, תפקידה המרכזי הוא לנהל את תהליכי האימות והגישה של מנהלי המערכת, תוך יצירת שכבת תיווך מבודדת בין ממשקי המשתמש לבין שכבת הגישה למסד הנתונים.

מבחינה ארכיטקטונית, המחלקה מיישמת את עקרון הפרדת תחומי האחריות. במקום שמסכי ה UI ייגשו ישירות למסד הנתונים, הם פונים ל-AdminService אשר אחראי על הפעלת הלוגיקה העסקית, אימות הנתונים וקבלת ההחלטות הנדרשות לפני ביצוע פעולות רגישות במערכת.

הפעולה המרכזית במחלקה היא, `authenticate(String username, String password)`, המשמשת כמנגנון ההתחברות של מנהלי המערכת. הפונקציה מקבלת את שם המשתמש והסיסמה שהוזנו במסך ההתחברות, בודקת האם קיים מנהל עם שם המשתמש הזה במסד הנתונים, ולאחר מכן משווה בין הסיסמה שהוזנה לבין הסיסמה השמורה במערכת.

אם הנתונים תואמים, הפונקציה מחזירה `true` ומאפשרת למנהל להיכנס למערכת. אם המשתמש אינו קיים או שהסיסמה שגויה, מוחזר הערך `false` והגישה נחסמת בצורה בטוחה.

מבחינה ארכיטקטונית, המחלקה מדגימה שימוש נכון בהפרדת תחומי אחריות. מסכי המערכת אינם מתקשרים ישירות עם בסיס הנתונים, אלא פונים ל-AdminService, אשר אחראי לבצע את בדיקות האימות ולקבל את ההחלטות הלוגיות הנדרשות לפני מתן גישה לממשק הניהול.

VehicleService

המחלקה משמשת כשכבת הלוגיקה העסקית (Service Layer) לניהול צי הרכבים של המערכת. היא מהווה את שכבת התיווך בין ממשקי הניהול בממשק המשתמש לבין שכבת הגישה לנתונים, ותפקידה הוא לנהל את המשאבים הפיזיים שעליהם מתבסס מנוע האופטימיזציה של המערכת.

הפעולה המרכזית של המחלקה היא `getAllVehicles`, אשר שולפת את כלל הרכבים הפעילים ממסד הנתונים. פונקציה זו אינה משמשת רק להצגה במסך הניהול, אלא גם מספקת קלט קריטי למנוע החישוב `AlgorithmRunnerView`. באמצעותה מוזרמים לאלגוריתם הגנטי אילוצי הבסיס של הבעיה, הכוללים את מספר הרכבים הזמינים ואת מגבלות הקיבולת של כל רכב, ומהווים חלק מהתנאים לפתרון בעיית הניתוב.

בנוסף, הפונקציה `saveVehicle` מאפשרת הוספה או עדכון של רכבים במערכת. היא מבטיחה שכל שינוי בצי הרכבים, בין אם מדובר ברכב חדש או בעדכון מאפיינים קיימים, יישמר באופן עקבי במסד הנתונים תוך שמירה על תקינות המידע.

פונקציה נוספת היא `deleteVehicle`, המאפשרת הסרה של רכבים מצי הפעילות על בסיס לוחית הרישוי כזיהוי עסקי. שימוש במזהה זה ולא במזהה פנימי טכני הופך את התפעול של המערכת לאינטואיטיבי יותר עבור מנהל המערכת, במיוחד בתהליכי תחזוקה שוטפת כמו השבתת רכבים או הוצאתם משימוש.

WarehouseService

C WarehouseService
❑ WarehouseRepository warehouseRepo
● Warehouse getWarehouseDetails() ● void updateWarehouseLocation(double lat, double lng)

המחלקה משמשת כשכבת הלוגיקה העסקית (Service Layer) לניהול מחסן ההפצה המרכזי של המערכת. היא מהווה את שכבת התיווך בין ממשק הניהול AdminManagementView לבין שכבת הגישה לנתונים WarehouseRepository ותפקידה הוא לנהל את נקודת העוגן הגיאוגרפית שעליה מתבסס כל תהליך האופטימיזציה במערכת.

הפעולה המרכזית של המחלקה היא getWarehouseDetails אשר שולפת את נתוני המחסן העדכניים ממסד הנתונים. נתונים אלו משמשים כרכיב יסוד בתהליך החישוב, שכן הם מגדירים את נקודת ההתחלה והסיום של כל מסלול במערכת ומהווים קלט הכרחי עבור ה-DistanceMatrixService והאלגוריתם הגנטי.

בנוסף, הפונקציה updateWarehouseLocation מאפשרת למנהל המערכת להגדיר או לעדכן את מיקום המחסן באמצעות קואורדינטות גיאוגרפיות (lat, lng) הנקלטות מתוך המפה האינטראקטיבית. הפונקציה אחראית על שמירה עקבית של הנתונים במסד הנתונים, כך שהמערכת תמיד תעבוד מול מיקום מחסן עדכני ומדויק.

מעבר לפעולות אלו, המחלקה מיישמת עקרונות של Abstraction ו-Encapsulation, בכך שהיא מסתירה את פרטי הגישה ל MongoDB משכבות ה UI. גישה זו מאפשרת גמישות ארכיטקטונית והרחבה עתידית של הלוגיקה העסקית, כגון תמיכה במספר מחסנים או הוספת בדיקות תקינות לקואורדינטות, מבלי להשפיע על ממשק המשתמש או על שאר חלקי המערכת.

12.3 מבני נתונים בהם נעשה שימוש

להלן פירוט מבני הנתונים המרכזיים אשר נעשה בהם שימוש בפרויקט:

מערך דינאמי - ArrayList

- ArrayList הוא מבנה נתונים ליניארי המבוסס על מערך פנימי, אך בניגוד למערך רגיל הוא מסוגל להתרחב ולהתכווץ באופן דינמי בזמן ריצה. המשמעות היא שאין צורך להגדיר מראש גודל קבוע, והמערכת יכולה להוסיף או להסיר איברים בהתאם לצורך. הגישה לאיברים מתבצעת ישירות לפי אינדקס, ולכן זמן הגישה הוא קבוע $O(1)$. בפרויקט, מבנה זה משמש בעיקר במחלקה GeneticRoutingService לניהול האוכלוסייה של האלגוריתם הגנטי וליצירת פתרונות חדשים. בנוסף, הוא מופיע בשכבת התצוגה, למשל ב- UserDetailsView וב-AlgorithmRunnerView, שם הוא משמש להעברת רשימות נתונים שמתקבלות מהשירותים. החשיבות של ArrayList בפרויקט נובעת מהעובדה שהאלגוריתם הגנטי עובד עם רצפים דינמיים של פתרונות ומסלולים, שבהם מספר האיברים יכול להשתנות בכל איטרציה. לכן נדרש מבנה שמאפשר גם גמישות גבוהה בהוספה ומחיקה, וגם ביצועים טובים בגישה לפי מיקום. בנוסף, בתהליכים כמו crossover ו mutation יש צורך בגישה אקראית לאיברים והחלפתם לפי אינדקסים, מה שהופך את ArrayList למבנה טבעי ונוח מאוד לשימוש במקרה זה.

טבלת גיבוב - HashMap

- HashMap הוא מבנה נתונים המבוסס על מערכת של מפתחות וערכים (Key-Value) כאשר כל מפתח עובר פונקציית גיבוב (Hash Function) שמאפשרת איתור מהיר של הערך המתאים. היתרון המרכזי שלו הוא ביצועים גבוהים במיוחד בפעולות של הכנסה, מחיקה ושליפה, שבממוצע מתבצעות בזמן $O(1)$. בפרויקט HashMap משמש בתוך המחלקה GeneticRoutingService, בעיקר במתודת C שם נוצרת מבנה עזר בשם orderToldx, אשר ממפה כל הזמנה (Order) לאינדקס שלה במסלול הנוכחי. השימוש במבנה זה הוא אופטימיזציה קריטית לביצועים של האלגוריתם. במקום לבצע חיפוש חוזר בתוך רשימה: פעולה שעלותה $O(N)$ בכל פעם שצריך למצוא מיקום של הזמנה, ניתן לגשת ישירות לאינדקס בזמן קבוע. מכיוון שפונקציית הכשירות (Fitness Function) רצה אלפי פעמים במהלך האלגוריתם הגנטי, שיפור כזה מצטבר לחיסכון משמעותי בזמן ריצה ומונע עומס מיותר על המערכת.

טבלת גיבוב מקושרת - LinkedHashMap

- טבלת גיבוב מקושרת: LinkedHashMap, הוא הרחבה של HashMap, אשר מוסיפה יכולת לשמור על סדר ההכנסה של האיברים. כלומר, בנוסף למיפוי רגיל של מפתח וערך, המבנה מחזיק גם סדר פנימי קבוע באמצעות רשימה מקושרת. בפרויקט, מבנה זה נמצא במחלקה GeneticRoutingService, בתוך מתודת decodeSolution. שם הוא משמש ליצירת מיפוי בין רכבים (למשל לפי מספר רישוי או מזהה) לבין רשימת ההזמנות שהוקצו להם. היתרון המרכזי בשימוש ב-LinkedHashMap הוא שמירה על סדר עקבי של הנתונים. כאשר המידע עובר לשכבת התצוגה (AlgorithmRunnerView) ומוצג על גבי המפה, חשוב מאוד שהרכבים יוצגו באותו סדר שבו הם חושבו על ידי האלגוריתם. ללא מבנה זה, שימוש ב-HashMap רגיל היה עלול לגרום לערבוב סדר לא צפוי, מה שהיה פוגע בקריאות ובחזיונות המשתמש.

מערך - Array

- מערך הוא מבנה נתונים בסיסי בעל גודל קבוע מראש, שבו כל האיברים מאוחסנים ברצף בזיכרון. היתרון המרכזי שלו הוא יעילות גבוהה מאוד בגישה לפי אינדקס, וכן שימוש נמוך יחסית במשאבים. בפרויקט נעשה שימוש בשני סוגים עיקריים של מערכים: במחלקה GeneticRoutingService, בתוך מתודת crossover נעשה שימוש במערך מסוג Order[] לצורך יצירת פתרון חדש (child) בגודל קבוע. בנוסף, במחלקה DistanceMatrixService קיימת מטריצת מרחקים דו-ממדית מסוג double[], אשר מייצגת את המרחקים בין כל זוג נקודות במערכת. במקרה של crossover, המערך מאפשר לבנות שלד קבוע של פתרון, שבו חלק מהתאים ממולאים לפי הורה אחד, וחלק אחר מושלם בהמשך. במטריצת המרחקים, השימוש במערך דו-ממדי מאפשר ייצוג מתמטי פשוט ויעיל של בעיית המרחקים בין כל נקודות היעד, נתון שמשמש לאורך כל ריצת האלגוריתם ללא שינוי.

12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

להלן קוד חישוב המסלולים:

```
public Map<String, List<Order>> calculateRoutes(List<Order> orders, List<Vehicle> vehicles,
Warehouse depot,
    double[][] matrix) {
    if (orders.isEmpty() || vehicles.isEmpty() || matrix == null)
        return new HashMap<>();

    List<List<Order>> population = initializePopulation(orders);
    List<Order> bestSequence = null;
    double bestFitness = -1.0;

    for (int i = 0; i < GENERATIONS; i++) {
        population.sort((a, b) -> Double.compare(calculateFitness(b, vehicles, matrix),
            calculateFitness(a, vehicles, matrix)));

        double currentBestFitness = calculateFitness(population.get(0), vehicles,
matrix);
        if (currentBestFitness > bestFitness) {
            bestFitness = currentBestFitness;
            bestSequence = new ArrayList<>(population.get(0));
        }

        List<List<Order>> nextGen = new ArrayList<>();
        for (int j = 0; j < 10; j++)
            nextGen.add(new ArrayList<>(population.get(j)));

        while (nextGen.size() < POPULATION_SIZE) {
            List<Order> parent1 = selectParent(population);
            List<Order> parent2 = selectParent(population);
            List<Order> child = crossover(parent1, parent2);
            mutate(child);
            nextGen.add(child);
        }
        population = nextGen;
    }

    return decodeSolution(bestSequence, vehicles);
}
```

הסבר על הפעולה:

הפונקציה `calculateRoutes` מהווה את מנוע האופטימיזציה המרכזי של המערכת, ומיישמת אלגוריתם גנטי (Genetic Algorithm) לצורך פתרון בעיית ניתוב הרכבים. מטרת הפונקציה היא למצוא חלוקה אופטימלית של הזמנות בין רכבים, כך שמרחקי הנסיעה הכוללים יהיו מינימליים תוך עמידה באילוצי המערכת, ובעיקר קיבולת הרכבים. התהליך מתחיל ביצירת אוכלוסייה ראשונית של פתרונות, כאשר כל פתרון מייצג סדר אפשרי של ביקור בהזמנות. פתרונות אלו נוצרים בצורה אקראית על מנת לספק מגוון התחלתי רחב למרחב החיפוש.

לאחר מכן מתבצע שלב הערכת הכשירות, שבו כל פתרון מקבל ציון בהתאם לאורך המסלול הכולל שהוא מייצר, בהתבסס על מטריצת המרחקים. ככל שהמרחק הכולל קצר יותר, כך ערך הכשירות גבוה יותר.

בהמשך נכנס לפעולה התהליך האבולוציוני, החוזר על עצמו לאורך מספר דורות מוגדר מראש. בכל דור מתבצעים שלושה מנגנונים מרכזיים: ראשית, אליטיזם, שבו הפתרונות הטובים ביותר נשמרים ומועברים ישירות לדור הבא כדי למנוע איבוד של פתרונות איכותיים. שנית, מנגנון ה crossover, שבו נבחרים זוגות פתרונות הוריים ומשולבים יחד ליצירת פתרונות חדשים תוך שימוש בטכניקת order crossover, השומרת על סדר חוקי של הזמנות. שלישית, מנגנון המוטציה, המבצע שינויים אקראיים מבוקרים במסלולים, כגון החלפת מיקומים בין הזמנות, במטרה לשמור על מגוון פתרונות ולמנוע היתקעות בנקודות אופטימום מקומי.

בסיום כל הדורות, נבחר הפתרון בעל הציון הגבוה ביותר. פונקציית decodeSolution מתרגמת את הייצוג הגנטי של הפתרון לרצף פעולות אופרטיבי, ומחלקת את ההזמנות למסלולים עבור הרכבים בהתאם למגבלות הקיבולת. התוצאה המתקבלת היא סט מסלולים סופי המוצג למשתמש באופן גרפי על גבי המפה ומהווה את הפתרון האופטימלי של המערכת.

להלן פונקציית חישוב כשירות המסלול:

```
private double calculateFitness(List<Order> sequence, List<Vehicle> vehicles, double[][]
matrix) {
    Map<String, List<Order>> routes = decodeSolution(sequence, vehicles);
    double totalDistance = 0;

    Map<Order, Integer> orderToIdx = new HashMap<>();
    for (int i = 0; i < sequence.size(); i++) {
        orderToIdx.put(sequence.get(i), i + 1);
    }

    for (List<Order> route : routes.values()) {
        if (route.isEmpty())
            continue;

        totalDistance += matrix[0][orderToIdx.get(route.get(0))];

        for (int i = 0; i < route.size() - 1; i++) {
            int fromIdx = orderToIdx.get(route.get(i));
            int toIdx = orderToIdx.get(route.get(i + 1));
            totalDistance += matrix[fromIdx][toIdx];
        }

        totalDistance += matrix[orderToIdx.get(route.get(route.size() - 1))][0];
    }
    return 1.0 / (totalDistance + 1.0);
}
```

הסבר על הפונקציה

פונקציית הכשירות, המיושמת במחלקה באמצעות המתודה `calculateFitness`, היא רכיב מרכזי בתוך מנגנון האלגוריתם הגנטי ומהווה את מנגנון ההערכה שמאפשר לאלגוריתם "להבין" מהו פתרון טוב ומהו פתרון פחות טוב. מאחר שהאלגוריתם אינו יודע באופן מובנה מהו מסלול אופטימלי, הפונקציה היא זו שמתרגמת פתרון מוצע (סידור הזמנות מסוים) לערך מספרי שמייצג את איכותו. הפונקציה משתלבת ישירות בתהליך חישוב המסלולים של `calculateFitness`, והיא מופעלת על כל פתרון אפשרי שנוצר במהלך האלגוריתם כדי לקבוע את דירוגו באוכלוסייה. תהליך החישוב מתחיל בפענוח הפתרון באמצעות `decodeSolution`, אשר ממיר את הרצף הגנטי למבנה אופרטיבי של מסלולים לפי רכבים. לאחר מכן מתבצע שלב של מיפוי הזמנות לאינדקסים, המאפשר גישה מהירה למטריצת המרחקים שנבנתה מראש. בהמשך מחושב המרחק הכולל (`totalDistance`) של כל הפתרון. חישוב זה כולל שלושה רכיבים עיקריים: מרחק מהמחסן אל התחנה הראשונה של כל רכב, מרחקי המעבר בין תחנות עוקבות בתוך

אותו מסלול, ולבסוף מרחק החזרה של כל רכב אל המחסן. סכימה של כל הרכיבים הללו עבור כלל הרכבים יוצרת את העלות הכוללת של הפתרון.

בשלב הסופי מתבצעת המרה של המרחק לציון כשירות (Fitness Score) באמצעות הנוסחה:

$$1.0 / (totalDistance + 1.0)$$

המרה זו נדרשת משום שהאלגוריתם הגנטי פועל בשיטת מקסום, ולכן הפיכת בעיית מזעור מרחקים לבעיית מקסום ציונים מאפשרת לו לפעול בצורה נכונה. ככל שהמרחק הכולל קטן יותר, כך ערך ה-Fitness גבוה יותר, ולהפך. הוספת הערך 1 במכנה נועדה למנוע מצב של חלוקה באפס במקרה קיצוני.

בסיכומו של דבר, פונקציית הכשירות היא החוליה המקשרת בין חישוב המסלולים בפועל לבין מנגנון האופטימיזציה של calculateRouts, והיא זו שמאפשרת לאלגוריתם הגנטי להשוות בין פתרונות ולהתכנס בהדרגה לפתרון האופטימלי.

להלן פונקציה לחלוקת המשלוחים בין רכבים:

```
private Map<String, List<Order>> decodeSolution(List<Order> sequence, List<Vehicle> vehicles)
{
    Map<String, List<Order>> solution = new LinkedHashMap<>();
    int totalOrders = sequence.size();

    // 1. חישוב סך הקיבולת המשותפת של כל הרכבים
    double totalCapacity = vehicles.stream()
        .mapToDouble(Vehicle::getMaxCapacity)
        .sum();

    int orderId = 0;
    for (int i = 0; i < vehicles.size(); i++) {
        Vehicle v = vehicles.get(i);
        List<Order> route = new ArrayList<>();

        // 2. חישוב כמה הזמנות הרכב הזה אמור לקבל באופן יחסי (Target Load)
        // נוסחה: (קיבולת הרכב / סך הקיבולת) * סך ההזמנות
        int targetLoad = (int) Math.round(((double) v.getMaxCapacity() / totalCapacity) *
totalOrders);

        // הגנה: שלא יחרוג מהקיבולת הפיזית שלו ושלא ינסה לקחת יותר ממה שנשאר ברצף
        int actualLimit = (int) Math.min(v.getMaxCapacity(), targetLoad);

        // אם זה הרכב האחרון, הוא לוקח את כל השארית כדי שלא נאבד הזמנות
        if (i == vehicles.size() - 1) {
            actualLimit = totalOrders - orderId;
        }

        while (orderId < totalOrders && route.size() < actualLimit) {
            route.add(sequence.get(orderId));
            orderId++;
        }
        solution.put(v.getLicensePlate(), route);
    }
    return solution;
}
```

הסבר על הקוד:

הפונקציה `decodeSolution` מהווה שלב קריטי בתהליך האלגוריתם הגנטי, ותפקידה לתרגם את הפתרון הגנטי המיוצג כרצף אחד ארוך ושטוח של הזמנות, לחלוקה אופרטיבית ומעשית של מסלולים עבור צי הרכבים. למעשה, זהו השלב שבו הייצוג המתמטי של הפתרון הופך לתוכנית עבודה לוגיסטית בפועל, תוך התחשבות באילוצי הקיבולת של כל רכב.

התהליך מתחיל בחישוב פרופורציית העבודה של כל רכב ביחס לכלל הצי. המערכת מחשבת את סך הקיבולת של כל הרכבים, ובהתאם לכך קובעת לכל רכב את החלק היחסי שלו במספר ההזמנות. כלומר, רכב בעל קיבולת גבוהה יותר יקבל באופן טבעי חלק גדול יותר מהרצף, בהתאם ליחס בין הקיבולת שלו לבין סך הקיבולת הכוללת.

בהמשך מחושב יעד עבודה לכל רכב, אשר נקבע לפי נוסחה יחסית: חלקו של הרכב בקיבולת הכוללת כפול מספר ההזמנות הכולל. מנגנון זה מונע חלוקה שרירותית של הזמנות, ומבטיח התאמה בין יכולות הרכב לבין העומס המוטל עליו.

לאחר מכן מופעל מנגנון בקרה, שמטרתו להבטיח עמידה באילוצים פיזיים של המערכת. ראשית, נמנע מצב שבו רכב מקבל יותר הזמנות מיכולת הנשיאה שלו בפועל. בנוסף, בשל פעולות עיגול מתמטיות עלולות להיווצר "שאריות" של הזמנות שלא שובצו. כדי לפתור זאת, הרכב האחרון בצי מוגדר כרכב מאזן, והוא מקבל את כל ההזמנות שנותרו, כך מובטח שכל הזמנה ברצף הגנטי תטופל ללא חריגות. לבסוף, מתבצע שיוך בפועל של ההזמנות לרכבים. הפונקציה עוברת על הרצף הגנטי ומחלקת אותו לרשימות נפרדות, כאשר כל רשימה משויכת לרכב לפי מזההו. התוצאה נשמרת במבנה נתונים מסוג Map, שבו כל רכב מקבל את רשימת התחנות שלו.

בסיכום decodeSolution היא הפונקציה שמחברת בין העולם האבסטרקטי של האלגוריתם הגנטי לבין העולם המעשי של לוגיסטיקת משלוחים, והיא זו שמאפשרת להפוך פתרון מתמטי למסלולים אמיתיים הניתנים להצגה ולביצוע במערכת.

להלן פונקציה המחשבת את המרחק בין כל שני נקודות:

```

public double[][] buildDistanceMatrix(List<Order> orders, com.google.maps.model.LatLng
warehouseLoc) {
    int size = orders.size() + 1; // +1 עבור המחסן
    double[][] matrix = new double[size][size];

    // 1. מול גוגל (Context) הגדרת ההקשר
    GeoApiContext context = new GeoApiContext.Builder()
        .apiKey(apiKey)
        .build();

    // 2. API-הכנת רשימת הכתובות/מיקומים עבור ה
    String[] locations = new String[size];
    locations[0] = warehouseLoc.lat + "," + warehouseLoc.lng;
    for (int i = 0; i < orders.size(); i++) {
        locations[i + 1] = orders.get(i).getLat() + "," + orders.get(i).getLng();
    }

    try {
        // 3. Google Distance Matrix API-ביצוע הקריאה האמיתית ל
        // אנחנו מבקשים מרחקים בין כולם לכולם (מטריצה מלאה)
        DistanceMatrix result = DistanceMatrixApi.getDistanceMatrix(context, locations,
locations)
            .mode(TravelMode.DRIVING) // WALKING או BICYCLING-אפשר לשנות ל
            .await();

        // 4. חילוץ הנתונים מהתוצאה של גוגל לתוך המערך שלנו
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                if (result.rows[i].elements[j].distance != null) {
                    // גוגל מחזירה מרחק במטרים - אנחנו נמיר לקילומטרים
                    matrix[i][j] = result.rows[i].elements[j].distance.inMeters / 1000.0;

                    // במקום מרחק, אפשר להשתמש בזמן נסיעה (שניות)
                    // matrix[i][j] = result.rows[i].elements[j].duration.inSeconds;
                } else {
                    matrix[i][j] = 9999.0; // הגנה למקרה שאין נתיב נסיעה
                }
            }
        }
    } catch (Exception e) {
        System.err.println("Google API Error: " + e.getMessage());
        // Fallback API לחישוב אווירי במקרה של שגיאת
        return buildAirDistanceMatrix(orders, warehouseLoc);
    } finally {
        context.shutdown(); // סגירת החיבור לחיפוש במשאבים
    }

    return matrix;
}

```

הסבר על הקוד:

כדי שהאלגוריתם הגנטי יוכל לבצע אופטימיזציה, הוא לא עובד ישירות מול כתובות או מול שירותי מפות, אלא זקוק לייצוג מתמטי ברור של הבעיה. הפונקציה `buildDistanceMatrix` משמשת כשכבת התיווך בין העולם הגיאוגרפי לבין מנוע החישוב, והיא ממירה את כל נקודות המערכת למטריצה דו־ממדית של מרחקים.

בשלב הראשון מתבצע תהליך הכנת נתונים, שבו נבנית מטריצה בגודל $(N+1) \times (N+1)$ כאשר N מייצג את מספר ההזמנות והאיבר הנוסף מייצג את מחסן ההפצה. כל תא במטריצה `[i][j]` מייצג את המרחק בין נקודה i לנקודה j , כך שמתקבלת תמונת מצב מלאה של כל המעברים האפשריים במערכת.

לאחר מכן מתבצעת פנייה לשירות החיצוני של `Google Distance Matrix API` באמצעות `GeoAPIContext`. בשלב זה נשלחות כל הקואורדינטות (כולל המחסן וכל נקודות החלוקה), והמערכת מקבלת חזרה נתוני מרחק וזמן נסיעה אמיתיים המבוססים על כבישים בפועל, ולא על חישוב גיאומטרי פשוט.

בהמשך מתבצע עיבוד של התשובה שהתקבלה: המרחקים שניתנים במטרים מומרים לקילומטרים, ונשמרים בתוך המטריצה הדו־ממדית בצורה מסודרת, כך שכל זוג נקודות מקבל ערך מדויק שניתן לשליפה מהירה בזמן ריצה.

בנוסף, קיימת שכבת חוסן חשובה: אם מתרחשת תקלה בתקשורת עם `Google API` או שאין זמינות לרשת, המערכת לא נעצרת. במקום זאת מופעל מנגנון חלופי שמחשב מרחקים בקו אווירי בין הקואורדינטות. כך נשמרת זמינות מלאה של המערכת גם בתנאי כשל, תוך ויתור מסוים על דיוק לטובת המשכיות תפעולית.

לבסוף, לאחר סיום החישובים, נסגר הקשר מול השירות החיצוני באמצעות `context.shutdown()` שמטרתה שחרור משאבים ושמירה על ביצועים יציבים של השרת לאורך זמן.

13. תיאור מסד הנתונים

ניהול הנתונים במערכת מתכנן משלוחים ומסלולים מתבצע באמצעות מסד נתונים מנוהל הפועל בתשתית ענן. הבחירה בפתרון זה נובעת מהצורך בזמינות גבוהה, שרידות מידע, גיבויים אוטומטיים ויכולת הרחבה מהירה של משאבי האחסון בהתאם לגידול בכמות המשתמשים והנתונים, ללא תלות בחומרה מקומית או תחזוקה ידנית מורכבת.

המערכת עושה שימוש במסד נתונים מסוג `NoSQL`, המאפשר גמישות גבוהה במבנה הנתונים והתאמה טבעית לעולם הלוגיסטיקה הדינמי. במקום מבנה קשיח, הנתונים נשמרים כאובייקטים גמישים, כך שניתן לייצג ישויות מורכבות כמו מסלולי נסיעה הכוללים רשימות משתנות של הזמנות, נקודות עצירה וחלונות זמן, ללא צורך בפעולות חיבור מורכבות בין טבלאות.

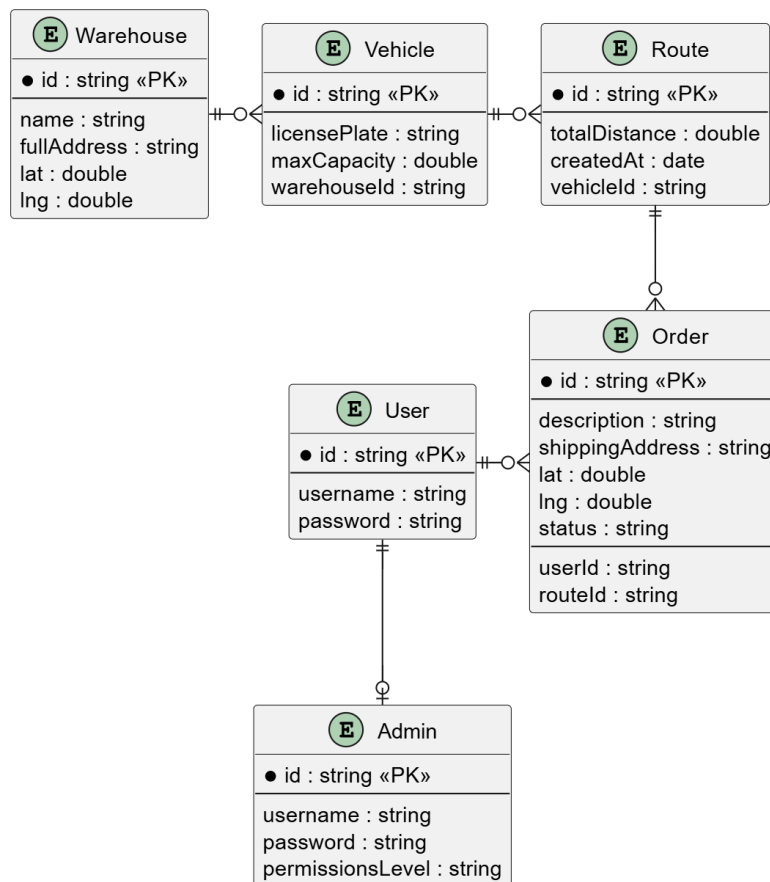
בנוסף, אחסון הנתונים בפורמט מבוסס JSON יוצר התאמה מלאה בין שכבות המערכת: הנתונים נשמרים במסד הנתונים, מועברים דרך ה-RestAPI באותו מבנה, ומעובדים ישירות בשכבת ה-Backend והאלגוריתם ללא צורך בהמרות או התאמות נוספות. הדבר מפשט את זרימת המידע ומשפר את ביצועי המערכת.

הארגון הפנימי של הנתונים מתבצע באמצעות אוספים נפרדים עבור הישויות המרכזיות כמו משתמשים, הזמנות ורכבים. מבנה זה מאפשר גמישות גבוהה בהרחבת המערכת בעתיד, כך שניתן להוסיף שדות ויכולות חדשות לכל ישות מבלי לשנות את מבנה המסד הקיים או להשפיע על נתונים קיימים.

להלן פירוט האוספים שישמרו במסד הנתונים:

שם האוסף	תיאור השדות המרכזיים	תפקיד במערכת
לקוח (User)	ID, שם משתמש, סיסמה.	יצירת הזמנות ויכולת להסתכל על הזמנות קיימות.
מחסן (Warehouse)	ID, מיקום מדויק (קואורדינטות).	הגדרת נקודות היציאה והחזרה של הצי.
רכב (Vehicle)	ID, קיבולת מטען, ID של מסלול נוכחי.	הגדרת המשאבים הזמינים לביצוע המשימות.
הזמנה (Order)	ID, כתובת משלוח, ID משתמש, סטטוס, מיקום בתוך המסלול (אם נמצאת בתוך מסלול).	תיעוד דרישות הלקוח ומיקומן ברצף החלוקה.
משתמש אדמין (Admin)	ID, שם משתמש, סיסמה.	ניהול פרטי משתמשים, מחסנים, רכבים, ונקודות יעד.
מסלול (Route)	ID, ID רכב, רשימת עצירות, מרחק.	שמירת הפלט הסופי של האלגוריתם הגנטי.

להלן תרשים ERD המציג את הקשרים והתלויות בין בסיסי הנתונים:



הסבר הקשרים בין הישויות:

Warehouse -> Vehicle (קשר של 1 ל-N)

מחסן אחד יכול לנהל מספר רכבים שונים. כל רכב במערכת משויך למחסן מסוים באמצעות מזהה המחסן.

Vehicle -> Route (קשר של 1 ל-N)

רכב אחד יכול לבצע מספר מסלולים שונים לאורך זמן. לכן כל מסלול שומר את מזהה הרכב שהוקצה לו.

Route -> Order (קשר של 1 ל-N)

מסלול אחד כולל מספר הזמנות שונות. לאחר שהאלגוריתם מסיים את תהליך האופטימיזציה, כל הזמנה משויכת למסלול המתאים לה.

User -> Order (קשר של 1 ל-N)

משתמש אחד יכול לבצע מספר הזמנות שונות לאורך זמן, ולכן לכל הזמנה נשמר מזהה המשתמש שיצר אותה.

User -> Admin (קשר של 1 ל-0 או 1)

מנהל מערכת הוא למעשה משתמש בעל הרשאות מורחבות. כלומר, כל Admin הוא גם User, אך לא

כל משתמש רגיל מוגדר כמנהל מערכת. קשר זה מאפשר ניהול הרשאות יעיל ומונע כפילות מידע במערכת.

14. מדריך למשתמש

פרק זה מתאר את תהליך השימוש במערכת מנקודת המבט של משתמשי הקצה (לקוחות) ומנהלי המערכת, ומדגים כיצד כל אחד מהם פועל בתוך זרימת העבודה הכוללת של האפליקציה. בתחילת העבודה, המשתמש מבצע התחברות דרך מסך ה-Login. לאחר אימות מוצלח של פרטי ההזדהות, מתבצע ניתוב דינמי בהתאם להרשאות: משתמש רגיל מועבר ללוח הבקרה האישי שלו, בעוד שמנהל המערכת מועבר ללוח הבקרה הניהולי. מנגנון זה מבטיח הפרדה מלאה בין רמות גישה שונות במערכת.

לאחר הכניסה למערכת, הלקוח יכול לבצע יצירת הזמנה דרך טופס ייעודי להזנת פרטי משלוח. במהלך הזנת הנתונים מתבצע תהליך אימות גיאוגרפי בזמן אמת שמוודא שהכתובת שהוזנה תקינה וניתנת להמרה לקואורדינטות מדויקות. כך נמנעת כניסה של נתונים שגויים למערכת עוד בשלב מוקדם. בהמשך, המשתמש יכול לצפות בכל ההזמנות שביצע באמצעות טבלה מרכזת. כל הזמנה מוצגת יחד עם סטטוס עדכני, כאשר שימוש בתוויות צבעוניות (Badges) מאפשר הבנה מהירה ואינטואיטיבית של מצב ההזמנה, כגון ממתין לשיבוץ, שובץ למסלול או נמסר בפועל. תצוגה זו משפרת את חוויית המשתמש ומצמצמת את הצורך בקריאה טקסטואלית מורכבת.

מן הצד הניהולי, מנהל המערכת עובד דרך ממשק ייעודי המאפשר לו לשלוט בתשתיות הלוגיסטיות של המערכת. במסך AdminManagementView הוא מגדיר את מיקום המחסן המרכזי (Depot) ומנהל את צי הרכבים הפעיל. לאחר מכן, הוא עובר למסך הרצת האלגוריתם (AlgorithmRunnerView). שם הוא מפעיל את מנוע האופטימיזציה בלחיצת כפתור אחת. התוצאה מוצגת באופן חזותי על גבי מפה אינטראקטיבית, יחד עם פירוט מסלולי הנסיעה והשיבוץ המלא של ההזמנות לרכבים.

15. בדיקות והערכה

על מנת להבטיח כיסוי בדיקות נרחב ומענה מלא לדרישות הפונקציונליות, תהליך בקרת האיכות חולק לשלוש רמות בדיקה היררכיות כמקובל בהנדסת תוכנה:

- **בדיקות יחידה:** בדיקות אלו התמקדו באימות הלוגיקה הפנימית של פונקציות הליבה במערכת, תוך בידוד מלא שלהן וללא תלות במסדי נתונים או בשירותים חיצוניים. מוקדי הבדיקה כללו את אימות פעולות החישוב של תהליכי הקרוס אובר והמוטציה המיושמים ב-GeneticRoutingService, וגם בדיקה של מנגנוני הסינון של סטטוס ההזמנות.

- **בדיקות אינטגרציה:** בדיקות אלו נועדו לבחון את תקינות האינטראקציה וזרימת הנתונים בין רכיבי המערכת השונים ובין שירותים חיצוניים. מוקדי הבדיקה כללו את התקשורת בין השרת למסד הנתונים בענן, סנכרון הנתונים עם רכיבי ה Grid הדינמיים ב-Vaadin, וכן אינטגרציה עם Google Maps API לצורך ביצוע Geocoding והפקת מטריצת מרחקים באמצעות DistanceMatrixService.
- **בדיקות קצה לקצה:** בדיקות אלו בוצעו כסימולציה מלאה של תהליך העבודה במערכת מנקודת מבטו של המשתמש הסופי ושל מנהל המערכת. התהליך כלל התחברות משתמש לקוח והזנת הזמנה חדשה דרך CreateOrderView, נעילת מיקומים על ידי המערכת, ולאחר מכן כניסת מנהל מערכת להגדרת מחסנים ורכבים ב-AdminManagementView. בהמשך הופעל האלגוריתם דרך AlgorithmRunnerView, והמשתמש צפה בתוצאות המסלולים המוצגים על גבי המפה ונשמרים בארכיון במסך SavedRoutesView.

16. מסקנות

הפרויקט הוכיח כי שימוש באלגוריתם גנטי הוא פתרון פרקטי ויעיל לבעיית ניתוב הרכבים תחת אילוצי קיבולת. בעוד שחיפוש פתרון אופטימלי מוחלט (Brute Force) אינו ישים עבור מספר רב של הזמנות עקב סיבוכיות זמן הריצה, האלגוריתם הגנטי הראה יכולת התכנסות לפתרון קרוב לאופטימלי בזמן ריצה סביר, המאפשר שימוש במערכת בזמן אמת.

אחת המסקנות הבולטות היא הפער שבין מודלים תיאורטיים ליישום מעשי. הפרויקט הוכיח ששילוב שירותי מיפוי חיצוניים בזמן אמת לחישוב מטריצת מרחקים אמיתית, משפר דרמטית את אמינות התוצאות בהשוואה לחישוב מרחקים אוויריים. המערכת מסוגלת להתמודד עם אילוצי כבישים אמיתיים ולהפיק מסלולים בני-ביצוע.

הפרויקט הוכיח את היתרון הגדול בשימוש בארכיטקטורת שכבות והפרדת תחומי אחריות. העבודה עם Spring Boot אפשרה להפריד לחלוטין בין מנוע החישוב המתמטי, ניהול הנתונים במסד הנתונים, ושכבת התצוגה. הפרדה זו הפכה את הקוד לקריא, ניתן לתחזוקה, וגמיש לשינויים עתידיים.

ברמה המערכתית, הפרויקט הוכיח כי אוטומציה של תהליך תכנון המסלולים יכולה לחסוך זמן תכנון יקר, להפחית את עלויות הדלק והבלאי של ציי רכב, ולנצל בצורה מקסימלית את קיבולת הרכבים הקיימים במחסן.

17. פיתוחים עתידיים

כדי להבטיח את הרלוונטיות של המערכת לטווח ארוך ולאפשר לה להתמודד עם אתגרי השוק הדינמיים, הוגדרו מספר צירי התפתחות מרכזיים. הרחבות אלו נועדו לשפר את ביצועי מנוע החישוב ולהעניק ערך מוסף משמעותי לחברות הפצה ולוגיסטיקה:

- **ניתוב מחדש בזמן אמת (Dynamic Re-routing)**

מנגנון שמאפשר עדכון מסלולים בזמן אמת בזמן שהנהגים כבר נמצאים בשטח. במקרה של תאונה, חסימה או שינוי ברגע האחרון בהזמנה, המערכת תחשב מסלול חלופי רק לרכב הרלוונטי ותשלח עדכון מיידי לנהג. כך נמנע צורך בחישוב מחדש של כל המערכת ונשמרת יציבות תפעולית.

- **מנוע אופטימיזציה ירוקה (Green VRPTW) וקיימות סביבתית**

הוספת שיקולים סביבתיים כחלק מהאלגוריתם של תכנון המסלולים. המערכת תחשב גם את רמת פליטת הזיהום של כל מסלול ותתחשב בסוג הרכב. רכבים חשמליים יועדפו לנסיעות עירוניות צפופות, בעוד רכבי דלק ישמשו יותר לנסיעות ארוכות ובינעירוניות. כך מתקבל שילוב בין יעילות תפעולית לבין הפחתת זיהום.

- **שילוב למידת מכונה (Machine Learning) לחיזוי עומסים דינמיים**

מעבר ממודל חישוב סטטי שמסתמך על נתוני מפות עדכניים בלבד, למודל חכם מבוסס בינה מלאכותית. המערכת תלמד מנתוני עבר של ההפצות ותזהה דפוסים חוזרים של עומסים, כמו פקקים קבועים בשעות מסוימות או עיכובים באזורים ספציפיים. המידע הזה ישולב בתוך פונקציית הכשירות של האלגוריתם הגנטי ויאפשר לו לבחור מסלולים חכמים יותר מראש, תוך הפחתת עיכובים ושיפור דיוק הזמנים.

18. ביבליוגרפיה

1. Toth, P., & Vigo, D. (Eds.). (2014). *Vehicle routing: problems, methods, and applications*. Society for Industrial and Applied Mathematic.
<https://www.jstor.org/stable/2627477>
ממקור זה למדתי את הרקע התיאורטי המקיף של בעיית ניתוב כלי רכב. הספר סיפק לי הבנה עמוקה של האילוצים השונים הקיימים בעולם האמיתי (כמו קיבולת רכבים ומספר נקודות חלוקה) וחשף אותי האלגוריתמים המקובלים כיום לפתרון בעיות לוגיסטיות מורכבות אלו.
2. Dantzig, G. B., & Ramser, J. H. (1959). The truck dispatching problem. *Management science*, 6(1), 80-91.
המאמר הזה, שהגדיר בפעם הראשונה את בעיית ניתוב כלי הרכב, עזר לי להבין את נקודת ההתחלה והבסיס המתמטי של הבעיה. דרכו למדתי כיצד מפרקים בעיה לוגיסטית של שילוח מנקודת מוצא מרכזית ליעדים מרובים למודל שניתן לפתור באמצעות חישוב, מה ששימש כבסיס ללוגיקת הניתוב במערכת.
3. Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*.
ממקור זה למדתי את עקרונות הפעולה של אלגוריתמים גנטיים, כולל מנגנוני הברירה, ההכלאה והמוטציה. ידע זה היה קריטי עבורי כדי לתכנן וליישם בפועל את מנוע האופטימיזציה ההיברידי בפרויקט, המאפשר למערכת לסרוק מרחב פתרונות עצום ולמצוא מסלולי חלוקה יעילים וקרובים לאופטימליים בזמן ריצה סביר.
4. *Spring Boot Reference Documentation*. Spring Framework. Retrieved from: <https://docs.spring.io/spring-boot/docs/current/reference/html>
התיעוד הרשמי של Spring Boot שימש אותי ככלי עבודה יומיומי לפתרון בעיות נקודתיות. ממקור זה למדתי כיצד להגדיר את סביבת הפיתוח במהירות, ליישם הגדרות אבטחה, לחשוף ממשקי REST API, ולחבר את השירותים השונים של המערכת בצורה חלקה תחת מעטפת Spring Boot.
5. *Vaadin Documentation*. Vaadin Ltd. Retrieved from: <https://vaadin.com/docs>
מתוך התיעוד של Vaadin למדתי כיצד לפתח ממשק משתמש אינטראקטיבי ומתקדם (UI) ישירות בשפת Java, ללא צורך בכתיבת קוד צד-לקוח נפרד כמו JS או HTML. המקור לימד אותי על ניהול מצבי מסך, בניית רכיבים ויזואליים, וקשירת הנתונים המחושבים בשרת ישירות לתצוגה שרואה מנהל ההפצה או הנהג.

6. *MongoDB Atlas Documentation*. MongoDB Inc. Retrieved from:

[/https://www.google.com/search?q=https://www.mongodb.com/docs/atlas](https://www.google.com/search?q=https://www.mongodb.com/docs/atlas)

ספר זה סיפק לי את ההבנה הנדרשת לעבודה עם מסד נתונים NoSQL מסוג מסמכים. למדתי ממנו כיצד לעצב את מבנה הנתונים של הפרויקט בצורה גמישה, כך שיתאים לאחסון אובייקטים מורכבים כמו נקודות ציון, פרטי נהגים ומסלולים מחושבים, וכיצד לבצע שאילתות יעילות על נתונים אלו.

7. *Google Maps Platform - Distance Matrix & Maps JavaScript API*

Documentation. Google Developers. Retrieved from:

<https://developers.google.com/maps/documentation>

ממקור זה למדתי כיצד לעבוד עם ממשקי ה-API-החיצוניים של Google, שהם לב ליבה של המערכת הגיאוגרפית. למדתי כיצד לשלוף נתוני אמת של מרחקי נסיעה וזמנים, החיוניים להפעלת אלגוריתם הניתוב, וכן כיצד להציג את המסלולים הסופיים בצורה ויזואלית וברורה על גבי מפה אינטראקטיבית בממשק המשתמש.